作品编号：**CACR2026EA16XZ**



作品类别：□ 软件设计　□ 硬件制作　■ 工程实践

□ 密码应用技术　□ 其它

2026 年第十一届全国密码技术竞赛作品设计报告

作品题目：ML-KEM 解封装实现的时间侧信道检测与常数时间防护

2026 年 10 月 21 日

中国密码学会

基本信息表

编号：CACR2026EA16XZ

作品题目：ML-KEM 解封装实现的时间侧信道检测与常数时间防护

作品类别：□ 软件设计　□ 硬件制作　■ 工程实践　□ 密码技术应用　□ 其它

作品内容摘要：

ML-KEM-768（FIPS 203）的解封装在 Fujisaki-Okamoto（FO）变换下依次执行解密、重加密、比较与隐式拒绝，四步都接触私钥或解密消息。完整解封装的耗时在五十微秒量级，真正与消息位相关的差异只有数十纳秒；单次调用不可分辨，必须靠每类一万次、三次独立运行与采集前冻结的检验家族，才能把差异从噪声里拉出来。源码写成无分支，不等于编译产物无分支，判定必须绑定具体的二进制。检测工具又各有分工：dudect 与 RTLF 只回答两类耗时是否可区分，Memcheck 与 Microwalk 只回答哪里依赖秘密，Binsec 只回答给定边界内是否常数时间。单工具结论不能互相替代，未检出也常被读成安全；没有检测预算的阴性，读者不知道它排除了多大的效应。这一步的判定止于两类分布是否可区分，不进入密钥恢复。本作品面向 ML-KEM-768 便携 C 实现，构建检测—定位—防护—复测平台。被测对象分三类：M1 与 M2 是真实历史案例，M3 与 M4 是人工校准夹具，M5 是阴性对照。平台在采集前冻结主类对、样本量、停止条件与 Holm 家族，用锁频控制器固定工作点并核对退出恢复；统计层用 Welch 检验、Holm 校正、逆方差固定效应合并与最小可检测效应量化每个阴性结论；定位层用六类工具交叉，并以三实现、五步骤、六类来源的 90 格排查逐格登记。定位、计时与防护复测绑定到同一份编译产物，产物摘要、编译条件与反汇编随每次判定登记，失败尝试与未运行项单独登记。

M1 未修版本的两类密文均值差为 32.3 ns，约占 57 μs 的 0.06%，三次运行方向一致；官方修复后区间含 0，解封装开销增加 1.51%。M2 在三处秘密除法上定位阳性，完整解封装与除法热点微基准都未检出，检测预算为 25 至 67 ns；定位阳性不等于计时阳性，修复前后也都不能说明泄漏降低。M4 的提前返回臂可区分、常数时间臂不可区分，结论只覆盖 32 字节比较夹具，不外推到完整解封装。M5 在本机、本预算下未检出，运行过的六种工具误报为 0。KyberSlash 官方修复版本仍残留与编译器引入分支同源的消息编码分支；同一份 M1 源码在 aarch64 上被编成无分支形态，机制不复现；同机网络各档在本预算下未检出，不推断真实远程。结论限定在单台 x86_64 裸金属、clang 18.1.3、本地白盒与不做密钥恢复的范围内；时间结论不跨架构、不跨环境合并，每个未执行单列，不并进阴性。题面 12 条按 10 条满足、2 条部分登记，缺口与影响写在附录 A。

关键词（五个）：ML-KEM；时间侧信道；常数时间实现；泄漏检测与定位；预注册统计

1.作品功能与性能说明

1.1 问题背景与赛题定位

ML-KEM 解封装的难点不在算法本身，而在源码级常数时间与产物级常数时间之间那条容易被忽略的缝。ML-KEM 由 CRYSTALS-Kyber 标准化而来，最终标准见 FIPS 203[1, 2]; 解封装在 Fujisaki–Okamoto 变换下依次执行解密、重加密、比较与隐式拒绝，四步都接触秘密。任何一步出现依赖秘密的分支、访存或可变时延指令，秘密就会进入时间。Guo 等对采用 FO 变换的 KEM 给出密钥恢复的计时攻击，说明比较与拒绝这一步的时延可以携带信息 [3]; KyberSlash 把秘密除法做成可复现的计时差异 [4]; clangover 表明编译器可以把源码里的无分支掩码还原成分支 [5]。

FO 变换把解密得到的消息重新加密，再与输入密文比较；比较结果决定输出真密钥还是假密钥，这条选择必须由常数时间比较与掩码完成。隐式拒绝的设计目的，是让失败路径与成功路径在外部不可区分；一旦比较或拒绝的路径依赖秘密，这个目的就落空，攻击者不需要私钥，也能从时间上判断哪一步走得更远。

32.311 ns 是 M1 未修版本在 57 µs 的完整解封装均值上多出的差，95% 置信区间 [20.282, 44.339]，占均值的 0.06%。

源码级常数时间不等于产物级。M1 的源码用掩码选择写成无分支，liboqs 0.10.0 在 x86_64 的 -Os 下仍被 clang 18 编出一条依赖消息位的条件跳转，liboqs 的安全公告把它登记为 CVE-2024-36405[6]；同一份源码在 aarch64 上被编成位域提取与条件选择，机制不复现。

M1 在 aarch64 上不复现，只说明这一条机制换了产物形态；换编译器、换优化级别、换目标架构，都要重新判定。这正是后面把产物与环境分开登记的原因。

单工具也不可靠。dudect 与 RTLF 只回答两类耗时是否可区分 [7, 8]，Memcheck 与 Microwalk 只回答哪里依赖秘密 [9, 10]，Binsec 只回答给定边界内是否常数时间 [11]；三类结论不能互相替代。M2 是最直接的例子：三处秘密除法在定位层稳定命中，完整解封装的合并差 $+4.58$ ns、区间 $[-3.08, 12.25]$ 含 0，除法热点微基准 $+0.044$ ns、区间 $[-0.430, 0.517]$ 也含 0，定位阳性与计时阳性并不重合。

还有一重困难在读者一侧。未检出常被当成常数时间，但阴性结论只在给定样本量、方差与检验水平下成立；不写检测预算，读者就不知道这个阴性排除了多大的效应。RTLF 在 42 个数据集上给出 30 个未检出、12 个检出 [8]，

M2 的检测预算为 24.7–67.1 ns，大于除法可能产生的效应；在这个预算下，未检出只能说明效应小于预算，不能说明除法没有时延。

四重困难落在同一条链上：效应小让测量难，源码与产物分离让承诺难，工具分工让判定难，误读让结论难。这决定了本作品要搭的是一套流程，而不是一个更快的实现或一个攻击工具。

题面的 12 条把这条缝拆成可检查的动作：接入标准实现、公开向量验证、可重复计时、

两类输入、统计检验、定位、防护、同环境复测、开销、环境实验、威胁模型与复现。任何一条只做一层，都会被后一条追问；本作品按检测、定位、防护、复测组织，让这些追问落在同一套证据上。

题面要的不是密钥恢复；本作品只判断两类耗时是否可区分。每次判定绑定编译产物，登记在明确的证据层上，阴性结论附该预算下的最小可检测效应。源码级常数时间不等于产物级常数时间；时间侧信道结论必须绑定编译产物、分层登记、按检测预算表述。

1.2 威胁模型与攻击面

本作品评估的是本地白盒攻击者：有源码、能编译、能重复调用、能读完整解封装耗时，但看不到私钥、共享密钥与内部验证标志。表 1 把能力假设与评估边界分成六项：能观测什么，判定止于哪一步。

表 1: 威胁模型与评估边界

项目	本作品的设定
评估者能力	持有源码与编译产物，可自选编译选项；可重复调用解封装，并读取每次调用的完整耗时
秘密对象	解封装私钥中参与解密的部分、解密得到的消息、共享密钥、比较结果与隐式拒绝标志
可观测量	每次完整解封装的耗时；两类合法等长密文的耗时分布
不观测的量	私钥内容、共享密钥、内部验证标志；不使用功耗、电磁、故障注入；不假设与被测进程共享缓存
攻击目标	判断两类密文的耗时分布是否可区分，并把差异归因到机制；不追求密钥恢复
范围之外	真实远程链路、其他架构的计时层、功耗与电磁与故障注入、密钥恢复

表注：本地白盒、可重复计时、不做密钥恢复。能力列说明攻击者能做什么，范围列说明本作品不做什么；表中没有列出的能力，本作品一律不假设。

表 1 给后续所有结论定边界：能力列是攻击者能做的事，范围列是本作品不做的判断。本地白盒意味着攻击者可以挑选编译选项并反复运行同一份产物，这比远程攻击者的条件宽松；结论按这个上限写，换到更弱的能力，同一机制可能从可区分退回未检出。

这张表也解释了为什么本地可区分不等于远程可利用：本地白盒给了编译与重复调用的自由，远程链路上的排队、协议栈与抖动不在观测范围内。

一次解封装的耗时可以写成

$$T = T_0 + \delta(\text{secret}) + \text{noise}.$$

T_0 是公开路径的基线，本机上约为 57 µs；δ 是秘密相关差异，在 M1 上只有 32.311 ns；noise 来自调度、频率、缓存与后台进程。锁频压住的是噪声里的频率项，压不住 δ 本身；要把 δ 从噪声里分出来，只能靠大样本与预注册检验。

方程的三项各有来源。T_0 由公开路径决定，换实现、换编译选项都会变；δ 是判定要找的量，它在源码里可能不存在，在产物里却可能出现；noise 不能靠一次测量估计，只能靠同一工作点下的重复采集。方程不是统计模型，它只说明看得见的部分由什么组成。

重复调用是这套模型里最容易被低估的能力。单次耗时落在噪声里，重复调用把均值差的标准误压到纳秒量级；本作品每类采 10,000 次、三次独立运行，按这个上限设计。

攻击者能控制的是密文类别与调用次数。主类对只让解密消息的位全 0 或全 1 不同，两类密文合法、等长，路径长度接近，差异只能来自消息位；随机密文、逐字节篡改、格式破损与截断作为族外对照，不进入主家族。攻击者能看到每次完整解封装的耗时，看不到私钥、共享密钥与内部验证标志，也不假设与被测进程共享缓存。

四类机制对观测的要求并不相同。分支与提前返回看的是路径长度差，除法看的是操作数相关的指令时延，查表看的是缓存行落点；同一套计时窗口能覆盖前两类，后两类还要靠定位工具确认机制存在。攻击面不是一张清单，而是机制、观测与判定三列的对应关系。

解封装链上的四类机制——消息编码里的条件分支、压缩路径上的秘密除法、秘密下标查表、提前返回比较——分别对应 M1 到 M4，M5 是对照。每类机制都要回答同一个问题：攻击者需要什么观测，本作品做到哪一层，离密钥恢复还差什么；这三问的逐机制对照见框 1。

框 1　攻击可行性对照

机制	攻击者需要的观测	本作品做到哪	离密钥恢复还差什么
秘密分支（M1）	同一密文类下，消息位不同导致耗时不同	本地锁频、完整解封装、三次运行可区分（+32.311 ns）	选择密文构造、逐位消息恢复、纠错与多轮查询；本作品不做
秘密除法（M2）	除法操作数不同导致耗时不同	定位层稳定命中三处除法；端到端与热点微基准未检出	更大样本量、针对除法的输入类、远程噪声下的可观测性；本作品不做
秘密下标（M3）	不同下标落在不同缓存行导致耗时不同	人工夹具上动态检查与轨迹互信息命中；计时层未检出	共驻缓存攻击模型、缓存驱逐与观测；本作品不做
提前返回（M4）	比较从第几个字节开始不同导致耗时不同	32 B 夹具上脆弱臂可区分，常数时间臂不可区分	完整 KEM 比较路径、远程噪声、逐字节恢复；本作品不做
阴性对照（M5）	若存在机制，应能被工具或计时发现	三种动态模式 0 报告、有界验证无秘密相关控制流、计时未检出	不是常数时间证明；需要更大预算与其他工具

框注：本作品只判断可区分性，不进行密钥恢复。表中“本作品做到哪”一列只写实测到的层，不把定位阳性写成计时阳性。

本地锁频下 32 ns 可区分，不等于远程可观测：远程还要叠加协议栈、排队与链路抖动，本作品只做同机模拟，并按检测预算报告未检出。可区分与可利用之间隔着输入构造、逐位恢复与纠错，本作品在可区分处停止。

范围之外包括四类：真实远程链路、其他架构的计时层、功耗与电磁与故障注入、密钥恢复。时间结论只来自一台 x86_64 裸金属；aarch64 只构成第二架构，不是第二台测量机；同机网络只是模拟，不是真实远程。这些结论不构成安全性认证，只说明本机、本产物、本预算下的观测；把这些任务交给本作品，会得到看似相关、实际越界的结论。

1.3 作品功能与系统组成

平台由五个被测对象、四个共享层和一条四段链组成；它的功能不是跑工具，而是把每次运行绑定到对象、产物、环境和判定。

平台对外提供六项功能。统一输入生成按公开种子派生主类对与族外输入，保证两类密文合法、等长、只差有安全意义的部分；受控测量把每次调用放进预热、乱序、固定窗口与锁频工作点，原始值全量落盘；统计检测用预注册家族、Welch 检验、Holm 校正与逆方差合并给出可区分性判定，并为每个阴性附最小可检测效应。

另外三项是函数级定位、防护与审计、可复现。函数级定位用反汇编、秘密标记、可变时延检查、轨迹互信息与有界验证把阳性落到函数与指令；防护与审计覆盖七类手段，并在产物层核对目标机制是否真的消失；可复现把计划、产物摘要、原始表与重算脚本打包成一键入口。

五个被测对象分三类。M1 与 M2 是真实历史案例：前者是 liboqs 0.10.0 在 x86_64 上被编译器引入的消息编码分支，后者是 pq-crystals Kyber768 历史快照里的秘密除法；M3 与 M4 是人工校准夹具，分别埋入秘密下标查表与提前返回比较，用来检验检测器能否识别已知机制；M5 是 mlkem-native 阴性对照，用来检查工具会不会误报。M1 对应 FIPS 203 初始公开草案（ipd），M2 是历史 Kyber768，两者都不等同于最终标准；五类对象的结果不能互相替代。

五类对象覆盖三种角色，是为了让同一套规程同时面对真实缺陷、已知机制与阴性对照。真实案例回答能否修复，人工夹具回答检测器能否识别，阴性对照回答会不会误报；三类问题不能用一个对象回答，也不能用一类对象的结论替另一类作保。

四个共享层是计时入口、锁频控制器、统计与功效、定位工具与结果登记。计时入口统一读钟与窗口，锁频控制器统一工作点与恢复核对，统计层统一家族、合并与预算口径，定位层统一工具版本与结果字段。四段链是检测、定位、防护、复测：检测回答能否分开，定位回答差异来自哪，防护回答改什么，复测回答改动是否生效。同一份原始数据可以供多层引用，结论只在其所属层成立；总体架构见图 3。

平台把判定用的二进制与计时用的二进制绑定为同一个。每次运行记录源摘要、产物摘要、编译器与选项，定位工具读的产物和计时窗口跑的产物是同一份；换编译选项等于换被测对象，必须重新登记。这条约束把源码改了与产物变了分开，也是后面所有分层结论的前提。

工作量按对象、工具、排查、采集、样本、环境、测试、审计、统计与复现十个维度登记，汇总见框 2。框 2 的用处不是罗列数字，而是让读者按维度追问：每个对象跑了几次，每格排查有没有证据，每次采集能不能回到运行编号。

框 2　工作量总览

维度	数量	说明
被测对象	5 个	2 个真实历史案例、2 个人工校准、1 个阴性对照
定位工具	9 种	45 格矩阵中 30 格有真实运行产物
全流程排查	90 格	3 实现 × 5 步骤 × 6 类；未检查 0 格
正式采集	177 次控制器运行	54 次平台家族 + 6 次 M2 微基准 + 117 次收尾
主类对样本	每类 10,000 × 3 次	换种子、换物理核；原始值全量落盘
优化级别	12 格 x86_64 + 12 格 aarch64	3 臂 × 4 级；产物判定与计时分开
网络场景	5 档 × 3 臂 × 2 次	本地直调、回环与低/中/高三档模拟
功能测试	263 项	88 + 57 + 57 + 61 四项；只验证软件流程
泄露渠道审计	128 文件 / 1,048 调用点	122 个接触秘密；0 条秘密泄露；1 条低危已修
统计复核	4 个 Holm 家族 / 21 个成员	42 个 RTLF 数据集；每个阴性附 MDE
复现材料	精简与完整两档	匿名包 5/5；42 份原始长表；逐次运行台账

框注：工作量不在跑了多少次，而在每次运行都能被追溯到对象、产物、环境和判定。

1.4 主要工作与创新点

本作品的贡献是把已有工具组织成一套以产物为中心、可审计、可分层的证据流程；它不提出新的检测原语，也不把任何观察主张为新的缺陷。四条工作各自回答四个问题：相对谁、改了什么、为什么不显然、代价是什么。

四条工作不是并列的四个功能，而是一条链上的四个约束：分层模型规定证据怎么登记，预注册规定计划什么时候冻结，多工具交叉规定阳性怎么归因，产物与环境对照规定结论能用到哪。少一条，另外三条的结论就会松。

第一条是分层证据模型：把机制存在、热点可测、端到端可测与防护有效分为四层登记，并用最小可检测效应量化未检出。相对单工具、单层结论的评估方式，它要求同一个被计时的产物同时登记定位层、热点层、端到端层与复测层；每个阴性附预算，每个未执行单列。直觉容易把定位阳性当成计时阳性，M2 正好相反：三处除法定位阳性，端到端与热点微基准都未检出。四种状态是检出、未检出、未执行、不适用；状态不是标签，决定这条结论能不能被引用。代价是采集与工具运行增加，报告必须保留四种状态。证据见表 2、表 9 与表 10。

第二条是预注册与可审计的锁频协议：采集前冻结检验计划，控制器校验计划摘要并核对频率恢复，使每次判定可追溯、可复算。相对于事后选检验、只看 p 值的计时实验，主类对、样本预算、停止条件与 Holm 家族一次冻结；放行前做 20 次频率探针，退出核对恢复。锁频的价值不是降噪：不锁频与锁频的变异系数比为 0.964、1.006、1.089，方向不一致；它提供的是确定、可核对、可恢复的工作点。

第三条是多工具交叉与 90 格排查：用 9 工具 × 5 模块矩阵和 90 格排查，把定位阳性与计时阳性分开，并识别官方修复后残留的同源分支。相对单工具评估，它要求每个存在给出函数、指令与触发输入，每个不存在写明检查手段与覆盖输入；90 格中 5 格存在、75 格不存在、10 格不适用、0 格未检查。

第四条是产物与环境的依赖对照：把同一源码在优化级别、架构、虚拟化与网络条件下的产物形态与可检出性并列，界定结论的适用边界。相对单环境、单优化级别的评估，它把

12 格优化级别、12 格 aarch64、三台 WSL2 与 5 档同机网络分开登记；时间证据不跨环境合并。直觉是分支越多越容易测到，实测 -O2 与 -O3 的命中位置更多，计时却未检出；M1 的机制在 aarch64 上不复现。

四条工作合起来，回答的是同一个问题：同一份源码在不同产物上可以有不同结论，证据必须按产物、按层、按预算登记。每条结论都能回到一个具体的产物、一个明确的证据层和一组可复算的数字；这些结论不覆盖所有实现或所有环境。

1.5 关键结果与性能概览

五个模块在同一套测量与统计规程下给出了分层结论：M1 测得出、修得掉；M2 定位阳性、计时阴性；M4 前后对照完成；M5 未误报。表 2 把五类的定位层、计时层与防护后状态并列。

五行的可比性来自同一套规程：同一计时入口、同一锁频工作点、同一预注册家族与同一功能门；不可比的是对象角色，M1 与 M2 是历史缺陷，M3 与 M4 是人工夹具，M5 是对照。表格按层读，不按行读。

表 2: 五模块分层结论

模块	对象与角色	定位层	完整解封装计时层	防护后
M1	liboqs 0.10.0 消息编码分支（FIPS 203 初始公开草案），真实历史案例	阳性：1 个秘密条件跳转	阳性：$+32.311$ ns [20.282, 44.339]；三次运行区间均不含 0	官方修复后分支消失；-8.150 ns $[-21.916, 5.616]$，未检出
M2	pq-crystals Kyber768 秘密除法（历史快照），真实历史案例	阳性：三处除法，1,280 errors / 3 contexts	未检出：$+4.58$ ns $[-3.08, 12.25]$；热点微基准 $+0.044$ ns $[-0.430, 0.517]$	官方修复后除法计数归零；计时层前后均未检出
M3	秘密下标查表，人工校准	阳性：1,280 errors / 2 contexts；互信息 1.922 bits	未检出：-0.73 ns $[-5.56, 4.10]$	off 臂：0 报告；互信息 0.722 bits
M4	32 B 提前返回比较，人工校准	阳性：32 errors / 1 context；有界验证给出反例	阳性：vulnerable 臂 -4.089 ns $[-4.215, -3.963]$	ct 臂：$+0.055$ ns $[-0.363, 0.473]$；有界验证无秘密相关控制流
M5	mlkem-native，阴性对照	未命中：三种模式各 10 次，0 报告	未检出：-5.30 ns $[-11.77, 1.16]$	无需修复；6 种工具误报 0

表注：未检出指观测效应小于该预算下的 MDE，不等于常数时间证明。M1 是 FIPS 203 初始公开草案，M2 是历史 Kyber768；M4 的计时对象是 32 B 比较调用，不与完整解封装直接比较；M5 只说明本机、本产物、本类对与本预算下未检出。

数字上的主线有四条。M1 未修合并差 $+32.311$ ns，区间 $[20.282, 44.339]$，三次运行区间均不含 0；官方修复后 -8.150 ns，区间 $[-21.916, 5.616]$ 含 0。

M2 的两个臂单独看是未修 $+12.35$ ns、已修 -11.20 ns，$I^2 = 58.3\%$，合并值只作描述；这不改变两臂都未拒绝的判定。

未检出的格子都附了预算：M2 的 MDE 为 24.7–67.1 ns，M3 为 17.7–33.7 ns，M5 为 13.5–28.3 ns。

M1 的三次运行区间均不含 0，Holm 家族中只有这一项被拒绝，这是本作品唯一在端到端层被判阳性的真实案例。

这些结论合起来说明一件事：修了与测不出是两件事。M1 的官方修复使分支消失，计时层随之回到区间含 0；M2 的除法在定位层归零，计时层却从未给出可区分的基线；M5 没有任何阳性，也只能说明本预算下未检出。三句话分开，读者才能判断每一步证据的强度。

表 2 显示同一套测量与统计规程能在五类对象上给出可比较的分层结论；未检出的格子只说明该预算下没有观测到效应，五类对象的风险并不相同。读表时先看层，再看行：定位层阳性只说明机制存在，计时层阳性才说明本机可区分，防护后一列回答改动是否生效。

性能开销按成对分块测。M1 修复使完整解封装 $+1.51\%$，M2 修复 -2.92%：修复并不总意味着变慢，更快也不等于更安全。

表 2 的作用是把测出了什么与没测出什么放在同一张表上：未检出的格子不是空白，而是附了预算的阴性；未执行与不适用不并入阴性。本章只给入口，方法见第 2 章，逐项证据与边界见第 3 章。题面 12 条按 10 条满足 + 2 条部分登记，缺口与原因见附录 A。

2.设计与实现方案

2.1 解封装流程与泄漏面

ML-KEM 解封装在 Fujisaki–Okamoto 变换下完成解密、重加密、比较与隐式拒绝四步计算，每一步接触的秘密并不相同。解密接触私钥与解出的消息，重加密把消息重新送进编码、数论变换、压缩与序列化，比较同时接触真密钥与假密钥，隐式拒绝决定失败时输出什么。把这条链笼统称为“实现有侧信道”，后续的测量与定位就没有落点；泄漏面必须按步骤拆开，才可能落到函数与指令。[1]

算法 1 ML-KEM 解封装与隐式拒绝

输入： 密文 c（1088 B），解封装密钥 $dk = (dk_{\text{PKE}}, h, z)$

输出： 共享密钥 K（32 B）

1: $m' \leftarrow$ K-PKE.Decrypt(dk_{PKE}, c) ▷ 解密，使用私钥

2: $(K', r') \leftarrow G(m' \,\|\, h)$ ▷ 派生候选密钥与重加密随机数

3: $c' \leftarrow$ K-PKE.Encrypt$(dk_{\text{PKE}}, m', r')$ ▷ 重加密

4: $\bar{K} \leftarrow J(z \,\|\, c)$ ▷ 隐式拒绝的假密钥

5: $K \leftarrow$ Select$(c' = c,\ K',\ \bar{K})$ ▷ 常数时间比较与掩码选择

6: **return** K

算法 1 回答的是标准结构中哪些数据接触秘密，不表示任何具体实现编译后的样子；M1 的对象是 FIPS 203 初始公开草案（ipd），M2 是历史 Kyber768，二者都不等同于最终标准。标准伪代码也不记录实现选择：消息编码可以逐系数写，也可以批量写；压缩可以原地进行，也

可以先拷贝。实现选择决定一条分支落在哪个函数、哪条指令，所以 90 格的“步骤”只是第一层坐标，第二层坐标必须是函数与指令。

第 1 步解密的秘密是私钥与解出的消息。消息解码函数里的一次整数除法，就是 M2 三处秘密除法中的第一处；除法指令的时延随操作数变化时，私钥系数会进入耗时。[4] 第 3 步重加密把消息重新送进消息编码、数论变换、压缩与序列化：M1 的分支出现在消息编码，编译器把源码中的掩码选择还原成按位测试后条件跳转；M2 的另两处除法出现在多项式压缩与向量压缩；M3 的人工秘密下标也插在这条路径上。[5]

第 4 步比较对应 M4 的提前返回夹具。第 5 步的隐式拒绝必须由常数时间比较与掩码选择完成；只要实现让分支或提前返回依赖比较结果，成功与失败两条路径就可以从耗时上分开，隐式拒绝的设计目的就落空。图 1 把这条对应关系画在同一张流程图上；M5 不挂在单独一步上，它作为整条链的阴性对照。

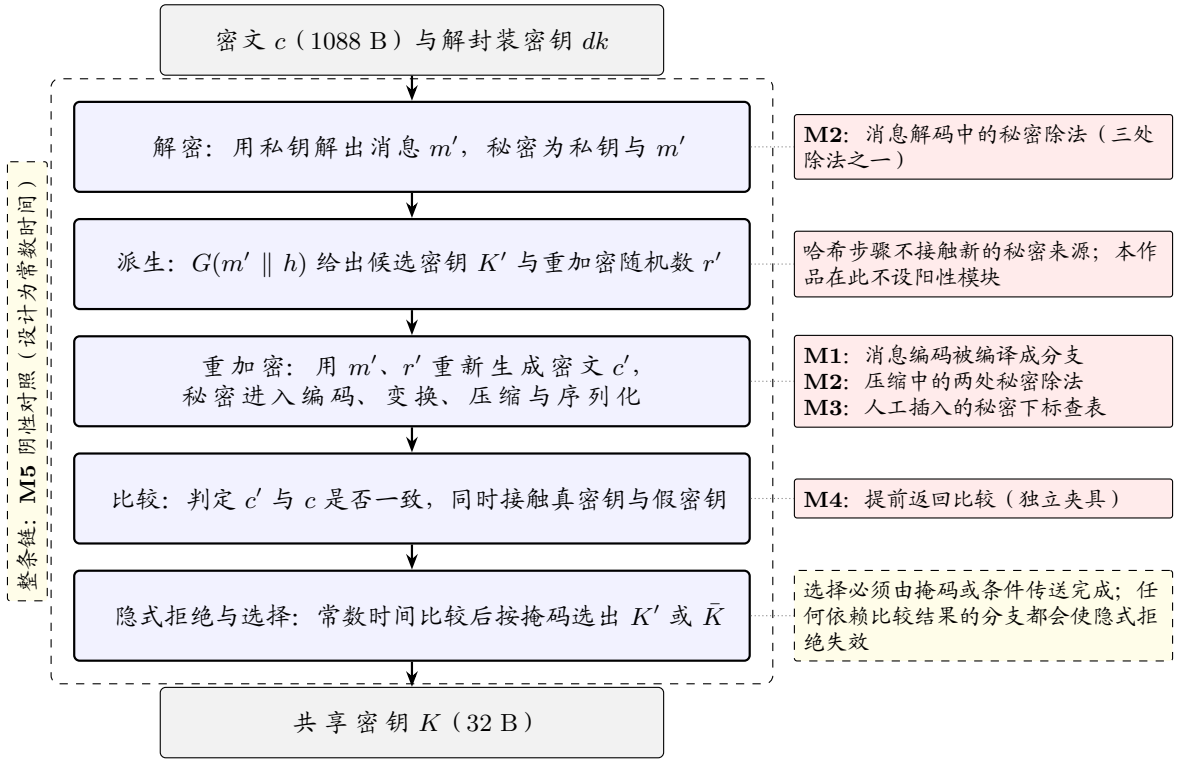


图 1: ML-KEM 解封装流程与泄漏点

解密、重加密、比较、隐式拒绝四步分别对应 M1–M5 的泄漏机制。M2 的第一处除法在解密步骤，另两处在重加密的压缩路径；M1 与 M3 都落在重加密的消息编码路径；M5 覆盖整条链，作为阴性对照。

泄漏来源按题面第 5 条分为六类，每一类都在解封装链上有明确的作用点。

1. 敏感条件分支：消息位或比较结果决定执行路径，两条路径的指令数或访存不同。
2. 提前返回：比较在发现第一个不同字节时结束，函数返回时刻取决于失配位置。
3. 可变时间比较与可变时延指令：逐字节比较提前退出，或整数除法等指令的时延随操作数变化。

4. 秘密下标访存：查表下标含秘密位，不同下标落在不同缓存行。

5. 数据相关循环：循环次数或迭代上界随秘密变化。

6. 异常处理路径：成功与失败走不同的分支、日志与返回码。

六类来源的检查手段不同：前三类靠反汇编与秘密标记，第四类靠执行轨迹与秘密的互信息，第五类靠控制流核对，第六类靠泄露渠道审计。工具放在 2.7 节，这里只固定分类口径，避免同一现象被两个名字重复登记。

这六类不是互斥的标签，而是按“秘密以什么方式影响执行”划分的检查口径。同一条指令可能同时属于两类，例如逐字节比较既是可变时间比较，也可能带提前返回；排查时按“最早把秘密带入可观测差异的那一步”归类，避免同一现象在两格重复计数。归类规则写进排查表的使用说明，换人核对时按同一口径。题面列出的六类来源覆盖了从控制流、访存到错误处理的主要通道；功耗、电磁这类物理通道不在本作品范围内，因为平台只测时间。

把这六类来源分别放到三个实现、五个解封装步骤上，就得到 $3 \times 5 \times 6 = 90$ 格的全流程排查表。五个步骤是解密、哈希派生、重加密、密文比较与隐式拒绝；三个实现是 M1、M2 与 M5，人工夹具 M3、M4 不进入这张表。每一格只允许四种判定：存在、不存在、不适用、未检查。判定规则在排查前写定：判“存在”必须给出可指认的函数、指令或源码位置以及触发它的输入；判“不存在”必须写明用了哪种检查、覆盖了哪些输入；判“不适用”要求该机制在这一步骤上不可能出现，例如该步骤没有异常处理；判“未检查”单列，不得并入阴性。

逐格结论与证据编号见 3.5 节与附录 F，本节只定义方法与判定规则。

每一格的记录不只写判定，还要写证据。字段包括：实现与产物摘要、步骤、来源类别、判定、函数名或指令地址、触发输入、使用的检查手段、证据编号。判“存在”的格子必须有触发输入；没有触发输入的“存在”只能算“疑似”，在汇总时降级为“未检查”。判“不存在”的格子要写覆盖了哪些输入；只跑过一个输入的阴性不足以支撑“不存在”。这套字段让 90 格既是排查表，也是审计清单：换一个人按同样字段复核，结论应当一致。

五步的划分比算法 1 多出“哈希派生”一步。算法 1 把派生与重加密写在一起，排查时若合并，哈希路径上的问题会被记到重加密名下；拆开后，派生步骤只检查哈希调用本身，重加密步骤只检查编码、变换、压缩与序列化。代价是排查表多出一列，但定位结论可以直接落到“哪一步、哪个函数”。

如果不按步骤拆，M1 的消息编码分支与 M2 的压缩除法都会被归到“重加密有问题”这一个大类里。重加密路径覆盖编码、变换、压缩与序列化多段代码，定位只能停在这一层，修复也无法核对到底改了哪条指令。90 格把每一格绑定到“实现 × 步骤 × 来源”，任何一格判“存在”都附一个可复核的代码位置，修复后同一格可以被复查为“不存在”。修复后按同一张表复查，目标格从“存在”变为“不存在”，其余格保持不变，修复的副作用也能被看见。代价是排查前要为每个实现准备五步的证据口径，排查后要维护 90 条带证据编号的记录；这部分工作量换来的是定位结论可追溯。

2.2 泄漏机理与攻击可行性

时间侧信道的“攻击”在本作品里只到“两类时间分布是否可区分”，不到密钥恢复。攻击者能控制的是密文类别与调用次数，能看到的是每次完整解封装的耗时；看不到私钥、共享密钥与内部验证标志。这个边界决定了实验设计：输入类必须合法、等长、只让有安全意义的部分不同；判定必须是两类分布的统计比较；结论必须带样本量与检测预算。

一次解封装的耗时可以写成

$$T = T_0 + \delta(\text{secret}) + \text{noise},$$

其中 T_0 是公开路径的平均耗时，δ 是秘密相关部分，noise 来自调度、频率、缓存与地址对齐。M1 未修版本在 57 µs 的均值上只多出 32.3 ns，δ/T_0 约为 0.06%；本地锁频压住的是噪声中的频率项，压不住 δ 本身。要在噪声里分出 δ，只能靠大样本与预注册检验；每类 10,000 次、三次独立运行，是把可检出下界压到几十纳秒量级的直接原因（2.5、2.6 节）。图 2 把观测模型与泄漏方程画在一起：左侧是攻击者能控制的输入与能看到的耗时，右侧是判定，底部是边界。

算法 2 攻击者的计时观测与可区分性实验

输入： 解封装预言机 $\mathcal{O}$，两类合法等长密文 C_0, C_1，每类样本 n，显著性 α

输出： 可区分 / 未检出（附效应量与区间）

for $b \in \{0,1\}$ **do**
  for $i = 1$ **to** n **do**
    $c \leftarrow \text{Sample}(C_b)$ ▷ 由公开种子派生，长度合法
    $t_{b,i} \leftarrow \text{Time}(\mathcal{O}(c))$ ▷ 只测一次完整解封装
  end for
end for
$\Delta \leftarrow \text{mean}(t_0) - \text{mean}(t_1)$
$CI \leftarrow \Delta$ 的 95% 置信区间
if CI 不含 0 **then**
  return 可区分 (Δ, CI)
else
  return 未检出 (Δ, CI, MDE)
end if

算法 2 回答的是给定观测模型与样本量时，两类密文的时间分布能否被分开；它不回答密钥如何恢复，也不回答远程链路上是否可观测。

四类机理可以用最小形态写清楚。代码 1 到代码 4 只保留结构，不写完整实现，也不构成可直接复用的攻击代码。

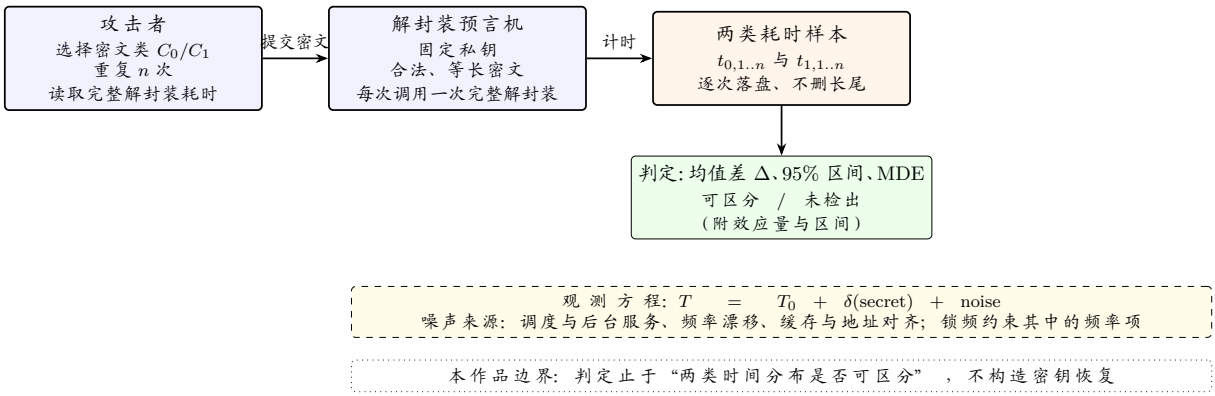


图 2: 攻击者观测模型与泄漏方程

判定止于两类时间分布是否可区分，不进入密钥恢复。观测方程把耗时拆成公开路径、秘密相关部分与噪声三块；锁频约束噪声中的频率项，样本量决定 δ 的可检出下界。

代码 1: 秘密相关分支的最小形态

```c
/* 消息位决定走哪条路径；两条路径的指令数或访存不同 */
int16_t select_coeff(const uint8_t *msg, size_t i)
{
    if ((msg[i / 8] >> (i % 8)) & 1)
        return COEFF_ONE;
    return 0;
}
```

代码 1 对应 M1 的消息编码：源码本用掩码，编译器在特定选项下把它还原成按位测试后条件跳转。边界是：这段代码只说明机制，不代表上游源码的原样写法；分支是否出现取决于编译器与优化选项，必须看实际产物。

代码 2: 秘密相关除法的最小形态

```c
/* x 来自解密消息；整数除法的时延可能随操作数变化 */
int16_t compress_coeff(int16_t x)
{
    return (int16_t)((((int32_t)x << 4) + (Q / 2)) / Q);
}
```

代码 2 对应 M2 的三处除法，Q 为公开模数。边界是：除法是否真的有可测时延取决于处理器与编译选项；本作品在特定产物上定位到三处整数除法，但端到端与热点微基准都未检出。

代码 3: 秘密下标查表的最小形态

```
/* 下标来自消息位；不同下标落在不同缓存行 */
int16_t lookup_coeff(unsigned secret_index)
{
    static const int16_t table[16] = { /* 16 个公开常数 */ };
    return table[secret_index & 15];
}
```

代码 3 对应 M3 的人工校准夹具：查表插在常数时间基线的重加密路径上，on/off 两版输出逐字节相同。边界是：这是人工埋点，不是上游漏洞；它检验检测器能否识别目标机制，不证明真实库存在同样问题。

代码 4: 提前返回比较的最小形态

```
/* 发现第一个不同字节就返回；耗时取决于从哪里开始不同 */
int compare_bytes(const uint8_t *a, const uint8_t *b, size_t len)
{
    for (size_t i = 0; i < len; i++)
        if (a[i] != b[i])
            return 0;
    return 1;
}
```

代码 4 对应 M4 的脆弱臂：常数时间臂改为逐字节异或累积、循环结束后统一判断。边界是：M4 是 32 B 独立比较夹具，不挂到完整解封装上；它证明检测链路能走通，不证明任何真实库有同样缺陷。

从“可区分”到“可利用”还隔着几道关。攻击者要构造能触发目标机制的合法密文，要把逐次观测拼成逐位或逐系数的恢复，要处理错误与多轮查询；换到远程，还要叠加协议栈、排队与链路抖动。本作品只做本地锁频下的可区分性判定，并在同机网络场景给出检测预算，不构造密钥恢复。M2 是最清楚的例子：三处除法在定位层稳定命中，端到端完整解封装与除法热点微基准都未检出；同一份实现，dudect 的最大 t 还落在已修臂上。机制存在、计时可区分、可利用是三件事，证据必须分层登记。

主类对只让解密消息的位全 0 或全 1 不同，其他条件相同；这样两类的路径长度接近，差异只能来自消息位。随机密文、逐字节篡改、格式破损与截断属于族外对照：随机密文检验一般路径，篡改与破损检验隐式拒绝，截断检验公开长度检查。它们不进入主家族，因为它们的路径长度或返回位置本来就不同，混在一起会把“路径不同”误读成“秘密泄漏”。族外对照的结论只作描述，不进入 Holm 家族；它们回答“其他输入是否也异常”，不回答“主类对是否可区分”。本作品不构造密钥恢复：密钥恢复需要选择密文预言机与逐位纠错，而检测平台的目标是判断可区分性并定位机制；走完攻击链不会让检测结论更强，反而会把工程边界与攻击能力混在一起。

框 3　攻击可行性对照

机制	攻击者需要的观测	本作品做到哪	离密钥恢复还差什么
秘密分支（M1）	同一密文类下，消息位不同导致耗时不同	本地锁频、完整解封装、三次运行可区分（+32.3 ns）	选择密文构造、逐位消息恢复、纠错与多轮查询；本作品不做
秘密除法（M2）	除法操作数不同导致耗时不同	定位层稳定命中三处除法；端到端与热点微基准未检出	更大样本量、针对除法的输入类、远程噪声下的可观测性；本作品不做
秘密下标（M3）	不同下标落在不同缓存行导致耗时不同	人工夹具上动态检查与轨迹互信息命中；计时层未检出	共驻缓存攻击模型、缓存驱逐与观测；本作品不做
提前返回（M4）	比较从第几个字节开始不同导致耗时不同	32 B 夹具上脆弱臂可区分，常数时间臂不可区分	完整 KEM 比较路径、远程噪声、逐字节恢复；本作品不做
阴性对照（M5）	若存在机制，应能被工具或计时发现	三种动态模式 0 报告、有界验证无秘密相关控制流、计时未检出	不是“无泄漏”的证明；需要更大预算与其他工具

框注：本作品只判断可区分性，不进行密钥恢复。表中“本作品做到哪”一列只写实测到的层，不把定位阳性写成计时阳性。

如果只讲“定位阳性”而不讲“计时是否可检出”，读者会把机制存在误读为本机可利用。M2 的三处除法定位阳性，端到端合并差 $+4.58$ ns、区间 $[-3.08, 12.25]$ 含 0，热点微基准合并差 $+0.044$ ns、区间含 0；只拿定位结论写“M2 有泄漏”，就会与计时结果冲突。框 3 把“本作品做到哪”和“离密钥恢复还差什么”分列，正是为了消除这种误读：每类机制都回答了“攻击者需要什么观测”，但没有一类被写成“可以恢复密钥”。

2.3 总体架构与证据分层

平台的价值不在单个工具，而在把冻结计划、被测对象、测量、分析、登记串成一条可追溯的流水线。任何一个环节可以换工具，换不掉的是“结论必须能回到运行编号与证据编号”这条约束。

冻结计划层在采集前写定主类对、样本量、停止条件与 Holm 家族，并生成计划摘要；测量层每次运行校验摘要，防止采集开始后改规则。被测模块层固定五个对象，记录源摘要、产物摘要、编译器、选项与反汇编，判定与计时使用同一个二进制。测量层执行计时循环与锁频控制，每次调用逐次落盘，失败样本单列。分析层三路独立：统计判定回答“是否可区分”，定位工具回答“哪里依赖秘密”，族外检验回答“换个统计口径是否还成立”。结果登记层把每条结论写成四种状态之一，附运行编号与证据编号，未执行与不适用不并入阴性。图 3 给出五层的顺序与共用的规程。

五层的顺序不是排版顺序，而是防篡改顺序。计划在最前，后续每一层都只能按计划执行；被测模块在测量之前固定，避免测到一半换产物；分析在测量之后，三路互不覆盖；登记放在末端，任何一条结论都必须带运行编号与证据编号。探索性分析可以做，但不能进主家族，只能作为族外观察或后续工作登记。

登记层的每条记录都有唯一编号，编号由运行编号、证据编号与计划版本三段拼接；引用时只写编号，不再复述结论。这样第 3 章引用第 2 章的方法时，读者可以顺着编号回到原始记录，而不必在两章之间寻找同一句话。

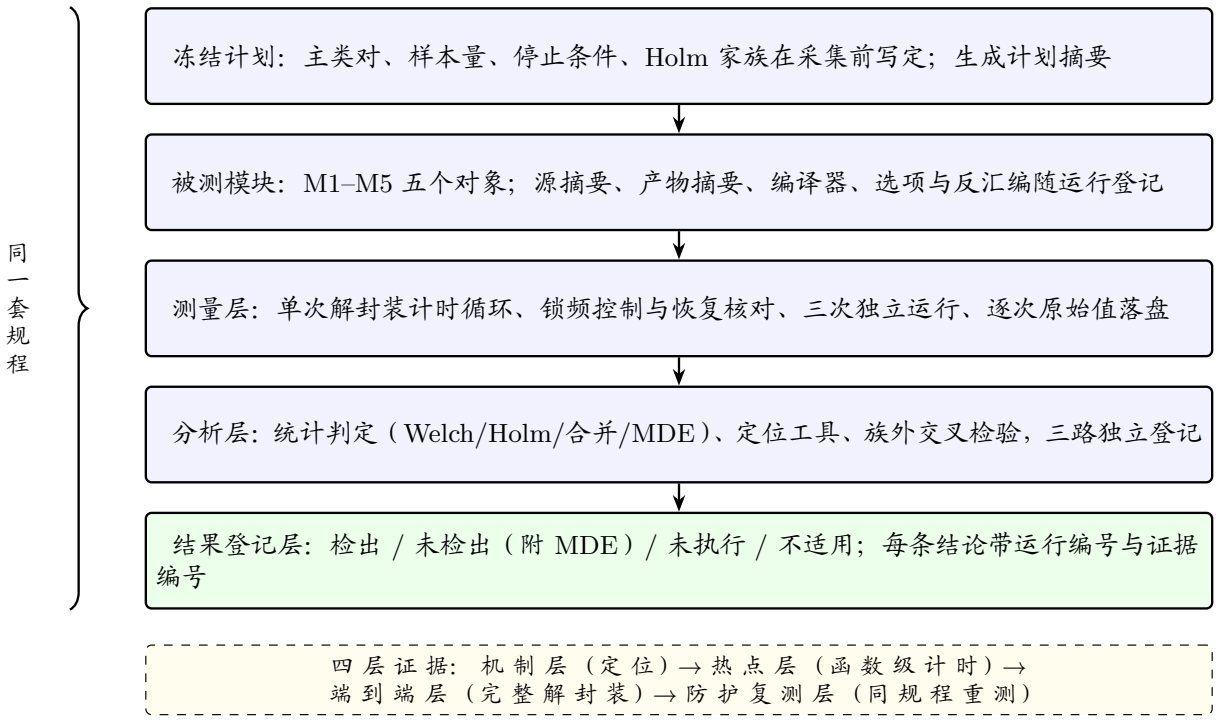


图 3: 平台总体架构与证据分层

冻结计划、被测模块、测量层、分析层与结果登记层共用一套规程。结果登记层的四种状态互斥，未执行不并入未检出；每条结论带运行编号与证据编号。

计划摘要还有一个副作用：任何一次运行都能被独立复算。审计者拿到计划文件、运行记录与原始值，可以重算摘要、重跑统计、核对恢复记录；不需要信任报告里的叙述。被拒绝的运行同样登记，编号连续，审计时能看到“尝试过但没通过”的记录，而不是只看到成功的样本。

四层证据是登记时的四个高度。机制层看源码、反汇编与动态标记能否指出秘密相关行为；热点层看函数级微基准或工具定位到的热点是否显示时间差；端到端层看完整解封装的类对检验；防护复测层看修复后在同一规程下重测的结果。

四层的判据不同：统计层的“检出”是预注册检验拒绝，“未检出”是观测效应小于 MDE；定位层的“检出”是工具稳定命中且能归因到函数、指令或源码行，动态工具的“未检出”只覆盖执行过的输入；防护复测层的“检出”是修复后目标机制仍在或计时差仍可区分。四种状态互斥：检出、未检出、未执行、不适用；未执行是没做，不适用是机制与步骤的组合不可能出现。

四种状态里最容易混淆的是“未检出”和“未执行”。未检出有样本、有检验、有 MDE；未执行只有原因。登记层要求每个未执行项写清“缺什么条件、影响哪条结论”，例如第二台测量机未部署、真实远程网络未搭建；这些条目在附录 A 与 5.2 节逐条列出，不计入阴性。

如果不分层，M2 的“定位阳性、计时阴性”会被写成自相矛盾的一句话。分成机制层与端到端层登记后，两句话分别属于不同层，可以同时为真，而且各自带各自的判据与预算。代

价是同一模块要做多次登记，报告必须保留“未执行/不适用”行，篇幅增加；分析层三路还要各自维护输入清单，换工具就要重跑对应格。换来的是任何一条结论都不会被另一层的证据误读。

2.4 被测对象与模块设计

五个对象不是“五个例子”，而是三类角色：真实历史案例、人工校准与阴性对照。每个角色回答不同的问题：真实案例检验平台对真实编译产物能不能检出，人工校准检验工具能不能识别目标机制，阴性对照检验会不会冤枉好人。表 3 列出五个对象的固定版本、编译条件、泄漏机制与修复或对照。

表 3: 被测对象与编译条件

模块	对象与固定版本	编译条件	泄漏机制	修复或对照
M1	liboqs 0.10.0，ML-KEM-768 初始公开草案的可移植 C 路径	clang 18.1.3，面向体积优化，关闭向量化与 AVX2 分发	编译器把消息编码还原为按位测试后条件跳转	官方修复提交 982c762
M2	pq-crystals Kyber768 参考实现，未修快照 a621b8d	clang 18.1.3，最低优化级别（-Oz）	三处秘密被除数除以公开模数	官方修复提交 272125f；另一提交 dda29cc 只修第一处
M3	基于 mlkem-native，在消息编码路径插入秘密下标查表	与 M5 相同	秘密下标访存（人工埋入）	on/off 两臂，输出逐字节相同
M4	独立 32 B 比较函数	独立编译，i386，-O1	发现首个不同字节即返回（人工埋入）	vulnerable/ct 两臂
M5	mlkem-native 固定版本 b3ba7b3	参考 C 路径	设计为常数时间	阴性对照

表注：五个对象的版本、编译条件、泄漏机制与修复一一对应，判定与计时都绑定到表内同一个产物。提交号仅用于定位历史版本，不表示对这些版本的推荐或背书。

M1 与 M2 是真实历史案例。M1 的可移植 C 路径在特定编译器与选项下被还原出分支，M2 的历史 Kyber768 参考实现出现三处秘密除法 [2]；两者都有上游修复可比对，因此能同时回答“检出”与“修复后是否消失”。代价是真实案例不能任意控制变量：M1 的构建要额外处理运行时分发，M2 的除法只在特定优化级别出现。

M3 与 M4 是人工校准夹具：M3 在常数时间基线的消息编码路径上插入秘密下标查表，M4 用独立的 32 B 比较函数对比提前返回与常数时间两臂；两版输出逐字节相同，确保校准的是检测器而不是算法语义。人工夹具回答“工具能不能识别目标机制”，不回答“真实库有没有这个缺陷”。M5 是阴性对照，设计为常数时间，用来估计工具误报；它在六种工具上的

误报为 0，这个数字只说明这六种工具在这个对象、这些输入上没有报假阳性，不说明 M5 已达到常数时间。

人工夹具的设计要点是“只差目标机制”。M3 的 on/off 两版除查表外完全相同，M4 的两臂除比较方式外完全相同；输出逐字节相同是先决条件，否则计时差异可能来自语义变化而不是泄漏。代价是夹具只能覆盖一种机制，不能代表完整解封装的交互；真实案例补上这一点，但真实案例又不能任意开关机制。三类角色合起来，才覆盖“能检出、能识别、不误报”三个问题。

选择历史 Kyber768 而不是最终标准作为 M2 的对象，是因为 KyberSlash 的两处修复提交落在历史代码上；最终标准改了消息编码与压缩的实现细节，同一机制在最终标准里可能已被间接消除。要检验平台能否识别这个机制，必须在机制存在的版本上做；把 M2 写成 ML-KEM-768 会把对象口径弄错。

判定用的二进制与计时用的二进制必须同一。每次运行记录源摘要、产物摘要、编译器版本、选项与反汇编摘要；采集前核对产物摘要，分析时再核对一次。M1 的数据是前期封存的，但封存于同一台机器、同一套锁频规程；它不是与 M2–M5 同一批采集，引用时单独标注。产物绑定还要求分析脚本拒绝没有摘要的输入：缺摘要的记录不能进入统计，避免手工拷来的数据混入。

两个构建坑直接决定结论是否成立。M1 只关闭可移植构建开关不够，运行时仍可能分发到 AVX2 实现；必须同时关闭 AVX2 指令与对应的实现开关，并用反汇编核对实际调用的消息编码函数。M2 的除法只在最低优化级别的构建中出现，换优化级别重建可能让机制静默消失；采集程序因此记录并核对二进制摘要，产物一变就拒绝沿用旧结论。

版本固定还意味着第三方代码只读：上游文件不改，修复以补丁形式叠加，补丁前后的源摘要与产物摘要都登记。这样一来，“官方修复”与“团队修改”不会混在同一次构建里，审计时可以分别核对。两次构建核对都进运行记录，缺一份就不能出具判定。

M1 的对象是初始公开草案，不是最终标准；草案与最终稿在密钥生成的域分隔上有差异，功能测试里 keygen 0/25 由此而来，不能记为实现错误。这个差异同时提醒：对象版本写错，功能结论与泄漏结论都会跟着错。

如果不绑定产物，就会出现“判定的是 A 二进制、计时的是 B 二进制”的情况：定位工具在带调试信息、未优化的构建上命中，计时却在发布构建上采集，两条结论无法拼到一起。产物绑定把每次判定压到一个摘要上，任何一次重跑都能回答“当时测的是哪个文件”。代价是每换一次编译器或选项都要重新构建、重新记录摘要，跨环境实验的构建次数因此成倍增加。

2.5 测量层设计

测量层的目标不是“跑得快”，而是让每次调用的耗时都能被复算、被追溯、被边界约束。计时窗口里多放一次内存分配或一行日志，32 ns 量级的效应就会被淹没；窗口外少做一次预

热，前几百次调用会把缓存与分支预测的冷启动写进均值。

算法 3 单次解封装的计时循环

输入： 被测二进制，类别集合 $\mathcal{C}$，每类样本 n，种子 s，CPU 核 k

输出： 逐次记录与元数据

1: 生成输入流；按 s 做可复现的类别乱序；记录输入流摘要
2: 在计时窗口外预热 100 次
3: **for** 每个（类别，密文）按乱序 **do**
4: $t_0 \leftarrow$ Clock()
5: Decaps(密文) ▷ 窗口只包住一次完整解封装
6: $t_1 \leftarrow$ Clock()
7: 写入运行编号、实现、类别、序号、$t_1 - t_0$ 与单位
8: **end for**
9: 记录元数据：源摘要、产物摘要、编译器、选项、时钟、频率、CPU、计划摘要
10: 失败样本单列；不删普通长尾；不扣读钟开销

算法 3 回答的是一次调用的耗时怎样被测到、怎样被落盘；它不回答这个耗时是否可区分，也不回答测量环境是否受控，后者由算法 4 与独立运行设计负责。

计时循环还有一个容易忽略的约束：每次调用后必须消费返回值。编译器可能把没有用到的解封装调用整个删掉，或者把循环折叠成一次调用；把返回值写入一个随后被校验的累加器，才能保证每次调用真的发生。代价是多一次内存写，但它在窗口之外。

输入生成、内存分配、结果消费、日志与 I/O 全部在窗口外，窗口内只有一次完整解封装。类别乱序按种子生成，避免“先测完 A 再测 B”时环境漂移造成假差异；预热 100 次让缓存与分支预测进入稳定状态。原始值全量落盘，普通长尾保留，异常样本规则在计划中预先写定，不在看到数据后删点。代价是每次运行产生的逐次记录与样本量同阶，存储与整理成本随运行次数线性增长；收益是任何一次判定都能从原始值复算。

读钟开销不扣：扣掉一个常数不改变两类均值差，却会让原始值失去可复算性；读钟开销约 17.4 ns，已在分辨率说明里单列。窗口内的调用返回值必须被消费，窗口外的日志与内存分配全部移走；这两条比“换一个更快的时钟”更能决定数据质量。预热次数也是计划里写定的；它只负责让缓存与分支预测进入稳定状态，继续增加预热不会改变分布，只会增加机时。

算法 4 锁频控制与恢复核对

输入： 目标频率 $f_0 = 4.30$ GHz，CPU 核 k，探针次数 $m = 20$

输出： 放行 / 拒绝；进入、采中、退出三段记录

保存睿频开关、调速策略、能效偏好与频率上下限的原始快照
关闭睿频；调速策略设为性能模式；能效偏好设为性能模式；频率下限与上限均设为目标频率
用绑核工具把被测进程绑定到核 k
等待 1.0 s
for $i = 1$ **to** m **do**
    读当前频率；间隔 50 ms
end for
if 全部 $|f - f_0|/f_0 \le 2\%$ 且 $\max - \min \le 50000$ kHz **then**
    放行
else
    拒绝，恢复快照，记录原因
end if
采集结束后恢复快照；读取并比较恢复核对结果

算法 4 回答的是“这次采集的频率工作点是否可核对、可恢复”；它不回答物理频率是否在每次调用中恒定，也不回答后台服务是否影响了单次调用。

探针的判据有两个:全部采样落在目标频率 ±2% 内，且最大值与最小值之差不超过 50,000 kHz。前者防止工作点偏到另一个频率档，后者防止采集期间来回跳档。任一条不满足就拒绝本次采集，恢复快照并记录原因；拒绝的运行同样进台账并保留原因。20 次探针、50 ms 间隔覆盖约 1 s，这个窗口足以看到调速器是否稳定，但不足以保证整段采集不变频；采中的跨度因此单独记录。

放行判据作用于 20 次探针，不作用于采集中的每一次调用；采集中记录频率跨度，单独登记。恢复核对只证明设置恢复到进入前的状态，不证明物理恒频。不锁频与锁频的 A/B 对照里，变异系数比为 0.964、1.006、1.089，方向不一致；锁频的价值是确定、可核对、可复算的工作点，不是“降噪”。本机未配置 isolcpus，SMT 开启，后台约 39 个服务在运行；这些条件如实记录，量化尺度约 10 ns。

计时精度决定了什么样的效应可以被讨论。读钟开销约 17.4 ns，最小步进 10 ns；时间戳计数器的分辨率下界为 42–43 个计数、约 9.8–10.0 ns；计数器速率为 4,291,962 kHz。32 ns 的效应只有 2–3 个分辨率单位，单次调用不可分辨，必须靠每类 10,000 次、三次运行的大样本。1 ms 短区间的频率极差为 348 kHz，相对极差 8.1×10^{-5}，相对标准差 5.4×10^{-6}，说明短窗内频率稳定；长期稳定性未认证，只测了短区间与逐次运行中位频率。

M4 的比较调用改用不做频率调整的原始单调时钟，窗口只包住一次 32 B 比较；它与完

整解封装使用不同的时钟与不同的窗口，数值不可直接比较。这一点在表 5 的表注里单独说明，避免把 -4.09 ns 与 $+32.3$ ns 放在同一把尺子上。

每个模块做三次独立运行：r1 种子 31337、CPU2；r2 种子 424242、CPU2；r3 种子 31337、CPU6。换种子检查结论是否依赖特定输入，换核检查是否依赖特定物理核。每类 10,000 次、三次运行合计 30,000 条；两臂合计 60,000 条，M5 单臂 30,000 条。

三个编号不是三次重复同一件事：r1 与 r2 同核不同种子，r3 同种子不同核；三者一致说明结论不依赖输入与物理核，三者不一致则提示环境漂移或输入依赖，按预注册规则登记，不选取对结论更有利的一次。选 CPU2 与 CPU6 是为了覆盖两个物理核，不是两台机器；同一台机器上的两个核共享内存控制器与电源域，核间一致只能说明结论不依赖某个核的局部状态。第二台测量机未部署，所有时间结论只来自这一台 x86_64 裸金属。

主类对是同一份确定性密文的 10,000 次重复，单次运行的区间反映的是测量噪声，不是密文间的变异；跨种子复测部分弥补这一点，但不等价于在每类内部使用大量不同密文。如果不锁频、不核对，频率漂移可以伪造类间差异：按 57 µs 的均值换算，1% 的频率偏差约对应 570 ns 的均值移动，比 32 ns 的效应大一个数量级以上；锁频协议把工作点固定并留下可复核的记录。如果 I/O 留在窗口内，一次日志写入的微秒级开销同样会淹没信号；窗口设计把这类开销全部移到计时之外。

失败样本的来源分三类：调用返回错误、读钟异常、进程被调度打断。前两类单列；第三类不删，但按计划中写定的规则标记，敏感性分析时分别用截尾均值与中位数复核。规则在采集前写定，看到数据后不再改；这条约束与统计层的预注册是同一条纪律在测量层的体现。失败率与失败原因一并进台账，便于判断采集是否整体失效。

2.6 统计层设计

统计层的目标不是“找到一个 p 值”，而是让“检出”和“未检出”都有可比较的含义。一个只报 p 值的实验，看到不显著可以加样本，看到显著可以停；这条路径最终一定通向“显著”，却说明不了效应大小与可检测下界。

每个模块的检验在采集前归为一个家族，全族 $\alpha = 0.05$，做 Holm 校正：M1、M2、M3、M4 各 6 项（3 次运行 × 2 个版本或臂），M5 3 项，共 21 项。主类对、样本量、停止条件与家族成员一次冻结，控制器每次运行校验计划摘要；停止条件固定为每类 10,000 次 × 3 次运行，不因中间结果延长或缩短。M1 的家族沿用前期封存计划，其余四份计划在采集前由同一提交冻结。

预注册的载体是一份计划文件：主类对的定义、每类样本量、三次运行的种子与核、家族成员、停止条件、异常样本规则都写在里面，采集前生成摘要。控制器每次启动校验摘要，摘要不符就拒绝运行；审计时独立复算摘要，确认计划没有被改过。计划文件与运行记录一起归档，任何一条判定都能回答“当时按哪份计划做的”。三次独立运行而不是一次，是为了估计运行间异质性：单次运行只能给测量噪声，三次才能算 I^2，并在合并时看出漂移。运行

次数再增加会提高机时，按预注册的停止条件不再延长。

算法 5 统计判定与合并

输入： 两组样本 t_0, t_1；家族大小 m；$\alpha = 0.05$；功效 $1 - \beta = 0.80$

输出： Δ、95% 区间、p、Holm 判定、合并 Δ、I^2、MDE

$\Delta \leftarrow \text{mean}(t_0) - \text{mean}(t_1)$

$SE \leftarrow \sqrt{\text{var}(t_0)/n_0 + \text{var}(t_1)/n_1}$

$t \leftarrow \Delta/SE$；自由度按 Welch–Satterthwaite 计算

$p \leftarrow 2(1 - T_{df}(|t|))$；$CI \leftarrow \Delta \pm z_{1-\alpha/2}SE$

家族内 m 个 p 升序排序；Holm 逐步比较 $\alpha/(m - i + 1)$

for 每次独立运行 i **do**

    $SE_i \leftarrow (CI_{\text{high}} - CI_{\text{low}})/(2z)$

    $w_i \leftarrow 1/SE_i^2$

end for

$\Delta_{\text{pool}} \leftarrow \sum w_i\Delta_i / \sum w_i$；$SE_{\text{pool}} \leftarrow \sqrt{1/\sum w_i}$

$I^2 \leftarrow \max(0, (Q - (k - 1))/Q)$，$Q$ 为 Cochran 异质性统计量

$\text{MDE} \leftarrow (z_{1-\alpha/2} + z_{1-\beta})\sqrt{s_0^2/n_0 + s_1^2/n_1}$

算法 5 回答的是给定样本如何得到均值差、区间、校正后的判定、合并估计与检测预算；它不回答输入类是否设计得对，也不把统计显著自动翻译成“可利用”。

选 Welch 而不是合并方差的学生 t，是因为两类的方差不一定相等：主类对只让消息位不同，路径长度接近，但缓存与分支预测的状态可能让两臂方差拉开。Welch–Satterthwaite 自由度不假设方差相等，代价是自由度略小、区间略宽；在每类 10,000 次下，这个代价可以接受。

统计量包括均值差 Δ、Welch t 与自由度、95% 正态近似区间、Holm 逐步校正、逆方差固定效应合并、I^2 与 MDE。区间与 p 值用大样本正态近似（$z = 1.9599639845$），不称为有限样本精确的 Student-t 区间。三次运行按逆方差固定效应合并；M2 的 I^2 为 58.3%，说明运行间存在未受控漂移，此时合并区间只作描述，判定以预注册的 Holm 结果为准。MDE 按 $(z_{1-\alpha/2} + z_{1-\beta})\sqrt{s_0^2/n_0 + s_1^2/n_1}$ 计算，Holm 口径取 α/m；“未检出”的确切含义是观测效应小于 MDE，不是效应为 0。

Holm 是逐步下降的校正：把家族内 m 个 p 值升序排列，第 i 个与 $\alpha/(m-i+1)$ 比较，一旦某个不拒绝就停止，后面的全部不拒绝。它比 Bonferroni 均匀地更有功效，同时仍然控制全族第一类错误。家族按模块划分，不把五个模块混成一个大家族：模块之间的效应机制不同，混在一起会让门槛过严，也会让一个模块的阳性掩盖另一个模块的问题。M1 的家族大小为 6，最小 p 的门槛是 $0.05/6 \approx 0.00833$；门槛在计划里写定，不随观测到的 p 值变化。

逆方差固定效应合并把三次运行看成同一真值的三次估计，权重取标准误平方的倒数；它假设运行之间只差抽样噪声。I^2 衡量这个假设是否成立：M2 的 58.3% 说明运行间还有未受

控漂移，此时合并值只作描述，不能拿来替代单次判定。合并的代价是它会把异质性平均掉，所以每次合并都同时报告 I^2。

报告里同时给 Δ、区间与 p 值，是因为三者回答不同问题：Δ 给效应大小，区间给估计的不确定度，p 值只回答“在零假设下观察到这么大数据有多罕见”。只给 p 值，读者无法判断效应是否值得修；只给 Δ，读者无法判断它是否超过噪声。

各家族的 MDE（Holm 口径）为：M1 31.39 ns，M2 25–67 ns，M3 18–34 ns，M5 14–28 ns，M4 最坏 1.47 ns。MDE 是事后功效描述，不改变已提交的判定；它回答的是“这个样本量下能看见多大的效应”，不是“真实效应有多大”。

MDE 与样本量的平方根成反比：要把 MDE 减半，样本量要翻四倍。每类 10,000 次、三次运行的选择，是在机时与可检测下界之间取的折中；M2 的 MDE 25–67 ns 大于除法可能带来的效应，这是“定位阳性、计时未检出”的直接解释，也是 5.2 节把 M2 列为局限的原因。

族外检验不并入 Holm 家族。RTLF 在 $\alpha = 0.01$、固定种子 2026101605、bootstrap 10,000 次的设置下对 M2–M5 的 42 个数据集给出 30 个未检出、12 个检出 [8]；dudect 报告在线最大 t[7]；Mona 做箱体分离。三者与 Welch 检验比较的对象不同：Welch 比较均值，RTLF 比较分布（含分位与尾部），dudect 与 Mona 是二值判据。它们的二值结论、最大 t 或箱体结论都不转换成 p 值参与校正，只作独立交叉印证。

族外检验的二值结论不转成 p 值，原因有三：零假设不同（分布差异对均值差异）、错误控制方式不同（RTLF 用 bootstrap 控制第一类错误，dudect 用最大 t 的经验阈值）、样本使用方式不同（在线检验与固定样本）。把三者并入同一个 Holm 家族，会得到一个既不是 0.05 也不是 0.01 的混合错误率。它们的位置是交叉印证：与主检验同向时增加信心，反向时提示输入类或环境有问题，而不是投票。

样本量不是先验拍定的：每类 10,000 次是在预实验中按观测标准差与目标 MDE 反推后冻结的，冻结之后不再调整。这样做的代价是方差估计本身也有不确定性，MDE 只能当作量级参考，不能当作精确的边界；正文提到 MDE 时都给区间或“最坏”值。

正态近似不是有限样本精确 t 区间；MDE 是事后描述；I^2 高时合并区间只作描述。三条限定都写在表注与 3.7 节。如果不预注册，看到不显著就加样本、换分组，最终一定能“显著”；预注册把这条路径堵死，代价是失败尝试也要登记、机时成本上升，而且不能在看到更好结果时改计划。

另一条代价是探索性发现进不了主结论：90 格里后来注意到的现象只能作为族外观察登记，等下一轮计划再升级为正式检验。预注册的另一个效果是让阴性结果可以比较：未检出不是失败，而是带预算的结论。统计层的输出全部落成表格，任何读者都能用附录 C 的公式与原始值复算。

2.7 定位层设计

定位层的目标不是“工具说有问题”，而是把问题落到函数、指令或源码行，并说明工具的覆盖边界。定位阳性只说明机制在执行过的输入上存在，不说明计时上可检出，更不说明可利用；每条定位结论都带这条边界。

算法 6 秘密标记动态检查

输入： 被测二进制，私钥 dk，秘密片段 $S = dk[0{:}1152] \cup z[0{:}32]$，公开输入集合

输出： 错误数、上下文数、源码位置或指令

1: 在动态检查工具下启动被测程序
2: 把 S 标记为未初始化（秘密）；公开输入保持已初始化
3: 运行完整解封装，覆盖接受与隐式拒绝两类路径
4: 收集“以秘密值为分支条件”与“以秘密值为访存下标”两类报告
5: 收集进入可变时延指令的秘密操作数报告
6: 归因到函数、源码行、调用栈与指令地址
7: 对未修与已修两臂重复；比较错误数与上下文数

算法 6 回答的是秘密是否进入了分支条件、访存下标或可变时延指令，以及落在哪个源码位置；它只覆盖实际执行过的输入，0 条报告不等于实现是常数时间。秘密标记的范围是私钥中参与解密的 1152 B 与隐式拒绝秘密的 32 B；公开输入保持已初始化。运行覆盖接受与隐式拒绝两类路径，报告按上下文归并：同一函数、同一源码行的多次命中记一个上下文，上下文数比错误数更能说明问题分布。

算法 7 Binsec 有界常数时间检查

输入： 抽取内核，入口与终点，秘密数组 s，公开数组 p，公开长度范围 1–16 B

输出： 不安全（附反例）/ 有界安全 / 未知

1: 固定目标架构、优化级别与编译选项；记录二进制摘要
2: 声明 s 为秘密；p 与长度为公开
3: 做关系符号执行，检查控制流与访存是否只依赖公开输入
4: 若存在反例，输出反例地址与路径
5: 检查待定、截断、中止与求解器未知四类状态是否为 0
6: 负对照：同一配置下已知脆弱内核必须判为不安全

算法 7 回答的是给定入口、终点与长度范围内，控制流与访存是否只依赖公开输入；它不覆盖范围外的输入，也不检查运行时时序。有界验证的结论只在入口、终点与公开长度范围内成立；路径完整性要单独检查，搜索被截断时工具可能返回未知而进程仍然正常退出。负对照用同一配置下的已知脆弱内核，确保工具不会把脆弱代码判成安全。工具版本、补丁与调用参数在环境表里固定；换版本要重新跑受影响的格子，不能把旧结论直接搬过来。

五类工具各自回答不同的问题。反汇编回答“机器码里有没有目标指令形态”，不回答运行时是否真的触发；秘密标记的动态检查沿用 ctgrind 的思路，回答“秘密是否进入分支条件或访存下标”，不回答可变时延指令 [9]；可变时延检查回答“秘密是否进入除法等指令”，不回答缓存与分支；轨迹互信息回答“哪些指令与秘密的关联最强”，不回答时序可利用性 [10]；有界符号执行回答“给定范围内是否存在秘密相关路径”，不回答范围外的行为 [11]。工具的版本在环境表中固定，输出摘要进附录 E。

五类工具的输出先归一化成同一种证据记录：工具与版本、输入集合、命中位置、上下文数、处置结论。归一是为了交叉：同一位置被两类工具命中记两条证据，只有一类命中时保留但不升级为“确认”。处置结论只取四种状态，与 2.3 的登记口径一致。

归因要求可复现：同一位置至少在三次运行中的两次出现，且换种子后仍出现；只出现一次的命中记为“待复核”，不进入结论。反汇编命中的指令地址要与源码行对上，动态工具命中的源码行要与反汇编对上；对不上时保留两种坐标，不强行合并。

9 种工具 × 5 个模块共 45 格，30 格有真实运行产物，15 格未运行；未运行的格子写明原因，不填阴性。M5 上运行过的 6 种工具误报 0；这个数字支持“阴性对照没有被冤枉”，不支持“M5 已达到常数时间”。定位结论与计时结论分开登记：M2 的三处除法在定位层命中，端到端与微基准未检出；M1 的分支在定位层命中，端到端可区分；M4 的比较在定位层与计时层都命中。

如果只用一个工具，M2 会被读错两次。单看 dudect，未修臂最大 t 为 4.68、3.05、3.66，已修臂反而为 36.27、30.94、61.91，阳性落在已修臂上；单看 RTLF，两臂都可能被判为可区分。只拿其中一个结论写报告，会得出“修复引入泄漏”或“两臂都有泄漏”，与端到端 Welch 检验的未检出冲突。交叉后才知道 M2 是定位阳性、计时未检出，dudect 的阳性不能归因于除法。多工具交叉的代价是安装、版本固定与人工核对；15 格未运行也必须保留。另一条代价是工具之间的结论可能冲突，冲突本身要写进报告，不只保留对结论有利的一条。

动态工具只覆盖执行过的输入：秘密标记看到的是这一组公开输入下的执行路径，轨迹互信息看到的是这一批轨迹；换一组输入，未执行的路径不会被报告。有界验证覆盖给定范围，但范围外的输入不在证明里。两类证据都不能外推成“实现没有泄漏”，只能写成“在本输入集、本范围内未见秘密相关行为”。

2.8 防护设计与泄露渠道审计

防护不是“改源码”，而是“改完以后在机器码、功能、计时三个层面都能核验”。源码里把分支写成掩码，编译器仍可能还原成分支；功能测试全过，日志或退出码仍可能泄露验证结果。

算法 9 常数时间防护与产物核验

输入： 脆弱实现，目标机制（分支、除法、查表、提前返回或错误路径）

输出： 修复后二进制，定位层、功能层与计时层记录

选择防护：掩码选择替换分支；乘法与移位替换秘密除法；去掉秘密下标；比较遍历全部字节；成功与失败走同一路径
编译；记录编译器、选项与二进制摘要
反汇编：检查目标分支、除法或查表是否消失
动态检查：目标错误数应降为 0
功能核验：修复前后输出逐字节一致；隐式拒绝仍然生效
同规程复测：端到端可区分性、族外检验与性能开销

算法 9 回答的是“改了什么、在产物里是否生效、功能是否仍等价、同规程复测是否完成”；它不回答修复是否在所有编译器与所有输入上都成立。表 4 把七类手段与对应的产物级核验并排列出。七类手段对应题面第 6 条列出的方向：消除数据相关分支、掩码选择、消除数据相关除法、消除数据相关访存、常数时间比较、固定循环上界、统一错误处理。每一类都给出实现方式与产物级核验，避免“源码改了就算防护”的写法。

表 4: 防护手段与产物级核验

防护手段	对应模块	实现方式	产物级核验
消除数据相关分支	M1	先清零系数，再用 16 位条件传送辅助函数按消息位做掩码选择	反汇编中按位测试后条件跳转消失；秘密标记报告 1→0
掩码选择	M1、M5	以取负、异或、与、再异或完成选择；隐式拒绝对两个候选密钥做字节掩码	机器码为取负/异或/与序列；解封装函数体内无依赖比较结果的跳转
消除数据相关除法	M2	用乘法与移位替代除以公开模数	三处整数除法计数 1→0；可变时延指令报告 1280/3→0/0
消除数据相关访存	M3	off 臂去掉秘密下标查表	秘密标记 1280/2→0/0；互信息 1.922→0.722 bit
常数时间比较	M4	逐字节异或后按位或累积，循环结束统一判断	秘密标记 32/1→0；有界验证由不安全变为有界安全
固定循环上界	M4、M5	比较循环始终遍历公开长度	反汇编与有界验证
统一错误处理	M2、M5	比较失败时返回假密钥，返回路径与成功路径一致	1000 例额外输入拒绝 1000/1000；泄露渠道审计

表注： 七类手段都在机器码层面留下可核验的痕迹，只改源码不足以说明防护生效。真实缺陷的修复来自上游补丁；团队自己写的修复只落在 M3、M4 两个人工夹具上。

M1 的防护用掩码选择替换分支：先清零系数，再由 16 位条件传送辅助函数按消息位在 0 与公开常数之间选择；辅助函数以取负、异或、与、再异或完成选择，不产生依赖秘密的跳转。M4 的常数时间比较把逐字节异或按位或累积，循环始终遍历公开长度，结束后统一判断。

代码片段：掩码选择替换分支

```
/* 先清零，再按消息位做掩码选择；不产生依赖秘密的分支 */
r->coeffs[8*i + j] = 0;
cmov_int16(r->coeffs + 8*i + j, (Q + 1) / 2, (msg[i] >> j) & 1);

void cmov_int16(int16_t *r, int16_t v, uint16_t b)
{
    b = -b;
    *r ^= b & ((*r) ^ v);
}
```

代码片段：常数时间比较

```
/* 逐字节异或后按位或累积；循环次数只取决于公开长度 */
unsigned int difference_bits = 0;
for (i = 0; i < len; i++)
    difference_bits |= (unsigned int)(secret_bytes[i] ^ candidate_bytes[i]);
return difference_bits == 0;
```

两段代码的边界相同：它们只保证在这两个对象、这两个产物上目标机制消失；真实缺陷的修复来自上游补丁，团队自己写的修复只落在 M3、M4 两个人工夹具上。

泄露渠道审计回答“防护过程本身是否通过日志、异常或返回码泄露秘密”。对 128 个源文件中的 1,048 个输出或返回调用点做静态扫描，其中 122 个接触秘密相关数据，占 11.6%；秘密内容泄露 0 条。审计发现 1 条低危问题：功能自检分支使计时程序在不同构建下退出码不同，构成可观察信号；修复后该类问题由 1 条降为 0 条。动态差分对 7 种输入模式各运行 120 次，返回码、标准输出与标准错误逐字节一致。

那条低危问题的修复方式是把功能自检改成仅在显式参数下执行：计时模式下标准错误恒空，退出码只反映参数、I/O 或内存分配失败。修复后重新跑动态差分，7 种输入模式各 120 次的返回码、标准输出与标准错误逐字节一致。静态扫描负责找调用点，动态差分负责验证可观察行为；两者都过，才把该类问题记为 0。

审计不覆盖上游第三方代码、网络路径以及进程级元数据（打开的文件描述符、内存映射、资源用量），这些需要单独专项检查。M1 所用版本的掩码辅助函数未带值屏障，在本作品主构建的解封装函数体内未观测到由此产生的分支，但在其他编译器与选项下仍是未经验证的可移植性隐患。真实缺陷的防护均采用上游官方修复，平台的作用是核验修复在产物层面是否生效。

如果只看源码，编译器可能把掩码还原成分支：M1 的源码本来用掩码，特定编译器选项下仍被还原成按位测试后条件跳转，只有反汇编与动态标记才能发现。如果只看功能测试，日志与退出码仍可能泄露验证结果：那条低危问题就是功能自检留下的可观察信号，功能测试全过并不能排除它。防护核验因此必须在机器码、功能与计时三层同时留下记录。

2.9 运行结果

运行结果只交代“跑了多少、频率是否受控、失败是否登记”，统计结论留给第 3 章。M1 的数据为前期封存：同一台机器、同一套锁频规程，但不是与 M2–M5 同一批采集。M2–M5 的正式采集共 54 次控制器运行，另有 6 次 M2 微基准；收尾阶段 117 次控制器运行（格式破损对照 18 次、性能开销 54 次、优化级别对照 36 次、锁频对照 9 次），三类合计 177 次。

“控制器运行”指一次经过计划校验、频率放行与退出恢复的采集单元；一次运行包含若干次调用，不等于一次解封装。M1 的封存数据不计入 177 次，统计口径仍按当时封存的家族。如果把拒绝的运行从台账里删掉，频率核对就只剩成功样本；本作品保留原编号与原因，批次按全部运行计。

以性能开销实验的 54 次运行为例，逐次实测中位频率的中位数为 4,309,722.5 kHz，相对名义 4.30 GHz 的偏差为 +0.005%；其中 1 次运行在采集期间的频率跨度为 50,310 kHz，略超 50,000 kHz 的跨度阈值，20 个采样值仍在 ±2% 带内，已如实登记。全部正式采集通过放行探针与恢复核对；失败尝试与异常样本单独登记，不并入阴性。频率偏差 +0.005% 是按逐次运行中位频率的中位数与名义值比较得到的，不是单次采样。频率核对的记录分进入、采中、退出三段，任何一段缺失都视为该次运行不合格。三次独立运行的原始值、失败样本与元数据一并归档；性能与优化级别的运行各自成批，不与主家族混算。

2.10 技术指标

表 5 用一张表给全平台的技术指标，便于对照题面第 2、8、9 条。指标均为本机实测值；未检出项附 MDE。如果只报已检出最小效应而不报 MDE，未检出会被读成没有效应；表中两者并列，未检出项都附预算。

表中每一行对应题面的一条或一组要求：测量对象与样本规模对应第 2、3 条，已检出最小效应与 MDE 对应第 4 条，定位覆盖对应第 5 条，防护开销与功能正确性对应第 6–8 条，复现测试对应第 11 条。未列出的项目不等于已满足，缺口见附录 A。表中未列远程网络与第二台测量机，这两项的边界分别见 3.11 节与 5.2 节。指标随产物与环境变化，换机器要重测，不能把本表当作 ML-KEM 实现的通用指标。

表 5: 技术指标

指标	实测值
测量对象	一次完整 ML-KEM-768 解封装（M1 未修均值 57,010.7 ns）
计时分辨率	读钟开销约 17.4 ns；最小步进 10 ns；时间戳计数器分辨率下界约 9.8–10.0 ns
样本规模	每类 10,000 次 × 3 次独立运行；窗口外预热 100 次
已检出最小效应	M1 +32.3 ns（约 0.06%）；M4 脆弱臂 −4.09 ns
未检出项的 MDE（Holm 口径）	M2 25–67 ns；M3 18–34 ns；M5 14–28 ns；M1 31.39 ns；M4 最坏 1.47 ns
阴性对照误报	M5 上运行过的 6 种工具误报 0
定位覆盖	90 格全流程排查，未检查 0 格；9 工具 × 5 模块矩阵中 30 格有真实产物
防护开销（解封装）	M1 +1.51% [1.36, 1.65]；M2 −2.92% [−3.06, −2.76]
功能正确性	M2 官方 KAT 100/100；M3、M5 各 80/80；M1 60 例通过 35（keygen 0/25 为初始公开草案与最终稿的域分隔差异）
复现测试	干净 clone 263 项通过；匿名包 CTest 5/5（软件流程验证，不产生计时结论）

表注：指标均为本机实测值；未检出附 MDE。M4 是 32 B 比较调用，其效应与开销不可与完整解封装直接比较；M1 的效应为 +32.3 ns（约 0.06%），修复开销为 +1.51% [1.36, 1.65]。

3.系统测试与结果

3.1 测试方案与环境

测试分六组，每组回答题面条目中的一个问题；环境是一台 x86_64 裸金属，所有时间结论只在这台机器上成立。六组是功能与语义等价、检测与定位、防护复测、功效与稳健性、性能、跨环境与多工具。分组按“回答哪一个问题”划分，不按工具划分：同一份原始数据可以在不同组里被引用，结论只在其所属组内成立。

六组与题面的对应关系：功能与语义等价对应第 1 条，检测与定位对应第 3–5 条，防护复测对应第 6、7 条，功效与稳健性对应第 4、8 条，性能对应第 8 条，跨环境与多工具对应第 9、11 条；第 2、10 条贯穿所有组。

表 6 给出七项环境要素。处理器、操作系统、编译器、动态分析、形式化、计时源与频率控制各自固定；换掉其中任何一项，时间结论都要重测。

表 6: 测试环境

项目	配置
处理器	AMD Ryzen 9 9950X（x86_64），SMT 开启，未配置 isolcpus
操作系统	Ubuntu 24.04.4 LTS，Linux 6.17，amd_pstate 频率驱动
编译器	clang 18.1.3（主构建）；gcc 13.3.0 已安装但未用于对照实验
动态分析	Valgrind 3.22.0（Memcheck）；Valgrind 3.25.1 + KyberSlash varlat 补丁；Intel Pin 3.28（Microwalk）
形式化	Binsec 0.11.3 + Z3 4.15.3
计时源	单调时钟（完整解封装）；原始单调时钟（M4 比较调用）
频率控制	关闭睿频，性能策略，频率上下限均为 4,300,000 kHz，绑核 CPU2 / CPU6

表注：单台 x86_64 裸金属、clang 18.1.3、锁频 4.30 GHz。gcc 只安装未参与对照，结论不覆盖 gcc；SMT 开启且未配置 isolcpus，后台约 39 个服务在运行，单次调用的量化尺度约 10 ns。

表 6 证明每组实验都有固定的处理器、编译器、工具版本与频率工作点；它不证明这些条件覆盖其他机器。

操作系统与频率驱动最容易被忽略: 锁频协议通过 sysfs 写频率上下限，换内核或换驱动，20 次探针的读数就可能落在不同工作点。编译器版本同样是背景信息: M1 的分支是 clang 18 在特定选项下的产物，换编译器等于换被测对象。环境表的作用是让“未检出”有明确的工具边界。

图 4 把五个模块、三类对象与七类实验环境排在同一张矩阵上。矩阵的行是实验类别，列是模块；空格不是遗漏，是该模块在该类实验中没有对象或不适用。

矩阵的每一格都绑定产物摘要、种子、CPU 与判定层。图 4 证明每一项结论都有对应的对象、环境与判定层；它不证明这些组合覆盖了所有环境，矩阵之外的组合没有数据。

正式采集分三批：平台家族 54 次控制器运行、M2 微基准 6 次、收尾 117 次（格式破损 18、性能 54、优化级别 36、锁频对照 9），合计 177 次。一次控制器运行是一次经过计划校验、频率放行与退出恢复的采集单元；M1 的封存数据不计入这 177 次，统计口径仍按当时冻结的家族。

被拒绝的运行与未执行项分开登记。优化级别实验中有 1 次运行因 20 次探针的频率跨度 54,629 kHz 超过 50,000 kHz 阈值被控制器拒绝，换名重跑后计入；性能批次有 1 次运行跨度 50,310 kHz 略超阈值，20 个采样值仍在 ±2% 带内，已如实登记。未执行项包括 GCC 对照、-O1/-Oz/-Og 三个优化级别、第三架构、isolcpus 与关闭 SMT、真实远程网络、第二台测量机、Microwalk 扩展到 M1/M2、Binsec 覆盖完整解封装、面向 CI 的回归门禁。这些项目只写“未执行”，不并进阴性。

M1 的数据是同一台机器、同一套锁频规程下的前期封存结果，不是与 M2–M5 同一批采集；引用时单独标注。第二台真实测量机没有部署，所有时间结论只来自这一台 x86_64 裸金属；aarch64 只构成第二架构，不是第二台测量机（3.11）。这两条决定适用范围。

	M1	M2	M3	M4	M5
对象类别	真实历史	真实历史	人工校准	人工校准	阴性对照
主类对家族	3 次 × 10^4/类	3 次 × 10^4/类	3 次 × 10^4/类	3 次 × 10^4/类	3 次 × 10^4/类
定位层	反汇编、Memcheck	varlat、反汇编	Memcheck、Microwalk	Memcheck、Binsec	Memcheck、varlat、Binsec
90 格排查	30 格	30 格	—	—	30 格
优化级别	4 级 × 2 版	—	—	—	4 级
aarch64	4 级 × 2 版	—	—	—	4 级
WSL2 虚拟化	4 级 × 2 版	—	—	—	4 级
同机网络	2 版 × 2 次	—	—	—	2 次
性能开销	解封装	解封装	小内核	小内核	解封装
工具矩阵	适用格	适用格	适用格	适用格	适用格

运行批次：平台家族 54 次 + M2 微基准 6 次 + 收尾 117 次 = 177 次控制器运行；每格登记产物摘要、种子、CPU 与判定层。

图 4: 实验矩阵与运行批次

图注：五个模块、三类对象、锁频/不锁频、优化级别、架构与网络各自独立登记。矩阵的空格表示该模块在该类实验中没有对象，不表示未执行；未执行项在正文单列。

3.2 功能与语义等价测试

功能测试先证明“被测对象仍是 ML-KEM”，再谈侧信道；修复版本必须与未修版本逐字节等价。被测对象的口径并不相同：M1 是 FIPS 203 初始公开草案的可移植 C 路径，M2 是历史 Kyber768 参考实现，M5 对应最终标准。三者的功能向量不能混用同一套期望值；M1 的密钥生成差异就是口径差异的直接结果。[1]

表 7 把五组功能结果列在一起。

表 7: 功能正确性与语义等价

对象	测试向量	结果与说明
M1 两版	NIST ACVP 转换的 ML-KEM-768 向量	执行 60 例通过 35：encaps 25/25、decaps 10/10（含 5 例隐式拒绝）通过；keygen 0/25 未通过，原因是 liboqs 0.10.0 实现 FIPS 203 初始公开草案，其 K-PKE.KeyGen 的哈希输入不含最终稿新增的参数集域分隔字节；另有 20 例密钥校验因封装层无对应入口记为不适用
M2 两版	Kyber round3 官方 KAT	100/100；普通构建与 ASan/UBSan 构建测试各 7/7
M3、M5	NIST 独立向量	各 80/80（含隐式拒绝用例）
修复等价性	同一驱动、同一公开输入	三对“未修 / 已修”实现的逐例输出摘要 diff 全空
额外输入	一半随机密文、一半逐字节篡改	隐式拒绝 1000/1000 按 FIPS 203 或 round3 规范生效

表注：修复前后功能等价；M1 keygen 0/25 来自 ipd 与最终稿的域分隔差异，不是实现错误。

keygen 0/25 需要单独解释。liboqs 0.10.0 实现的是 FIPS 203 初始公开草案，草案的 K-PKE.KeyGen 在哈希输入中没有最终稿新增的参数集域分隔字节；用最终稿生成的向量去核对，密钥生成必然对不上。这是标准版本差异，不是实现错误。0/25 是标准版本差异，35/60 是可核对的通过数；两种简化都会改变对象口径。

这 60 例的构成需要写清楚。encaps 25/25 覆盖封装路径的接受分支；decaps 10/10 里含 5 例隐式拒绝，覆盖比较失败后返回假密钥的路径；keygen 0/25 全部落在密钥生成；另有 20 例密钥校验因为该封装层没有对应入口，只能记为不适用。35 通过、25 未通过，合计 60；另有 20 例不适用，单列，不与这 60 例混算。这个口径与台账的四种状态一致：不适用不是失败，也不是通过。

20 例密钥校验记为不适用，是因为该封装层没有对应入口，不是测试没做。不适用与未通过的区别在报告里必须写清楚：前者是“这个接口不提供这项检查”，后者是“提供了但结果不对”。把 20 例不适用并进 35/60 的通过率，会把分母弄错；把 0/25 的 keygen 并进“实现错误”，会把标准版本差异写成缺陷。

M2 用 Kyber round3 官方 KAT，100/100；普通构建与 ASan/UBSan 构建各 7/7。M3 与 M5 用 NIST 独立向量，各 80/80，含隐式拒绝用例。三套对象的向量来源不同，是因为它们的标准版本不同：M2 是历史 Kyber768，只能用 round3 的 KAT；M5 对应最终标准，用 NIST 向量；M3 基于 M5 基线，向量与 M5 同源。把三套向量混在一起统计“总通过率”没有意义，表 7 因此按对象分行。

三对未修/已修实现的逐例输出摘要 diff 全空，说明修复没有改动算法语义。修复涉及三类改动：把分支改成掩码选择、把除法改成乘法与移位、把人工夹具的查表或提前返回改成固定路径；改动只落在执行路径上，不落在输出上。逐例摘要 diff 比“再跑一遍 KAT”更强：

KAT 只比较最终共享密钥，摘要 diff 还把中间输出与返回码一起比较；任何一处字节变化都会留下痕迹。

1000 例额外输入（一半随机密文、一半逐字节篡改）中，隐式拒绝 1000/1000 按 FIPS 203 或 round3 规范生效；这条测试覆盖的是失败路径，不是接受路径。随机密文检验一般路径上的解析与拒绝，逐字节篡改检验比较逻辑在单个字节不同时的行为；两类输入都不接触私钥内容，只检验失败路径是否按规范返回假密钥。1000/1000 说明失败路径没有被修复破坏，不说明失败路径的耗时与成功路径不可区分；后者属于 3.3 的计时层。

ASan/UBSan 构建的 7/7 是另一条证据。普通构建只证明功能正确，消毒器构建还要证明没有越界、未定义行为与内存错误；7/7 说明修复没有引入这类问题。消毒器构建的性能不适合计时，功能测试只在普通构建上做计时，两者分开登记。

表 7 证明三对修复没有改动算法语义，M1 的 0/25 是标准草案版本差异；它不证明任何实现是常数时间，也不证明这些向量覆盖了全部输入。功能等价是侧信道结论的前提：如果修复改变了输出，“修复后不可区分”就可能只是算法变了，而不是泄漏消失了。功能测试的另一条边界是向量集：NIST ACVP 与 round3 KAT 覆盖规范路径，不覆盖所有畸形输入；畸形输入由 3.8 的两组对照单独处理。向量集的来源与版本见附录 B，功能结论只覆盖这些向量集。

3.3 端到端可区分性

预注册 Holm 家族中，只有 M1 未修主类对与 M4 vulnerable 臂被拒绝；M2、M3、M5 的家族均未拒绝任何检验。图 5 把七个类对的均值差与 95% 区间放在同一把尺子上：M1 未修与 M4 vulnerable 的区间不含 0，其余五个区间含 0。

家族的构成决定了门槛。M1、M2、M3、M4 各 6 项，M5 3 项；每个模块的 6 项来自 3 次独立运行 $\times$ 2 个版本或臂。Holm 是逐步下降校正：家族内 p 值升序排列，第 i 个与 $\alpha/(m-i+1)$ 比较，一旦某一步不拒绝就停止。M1 的 $m=6$，最小 p 的门槛是 $0.05/6 \approx 0.00833$；M4 的 $m=6$，门槛相同；M5 的 $m=3$，门槛是 0.0167。门槛在采集前写进计划，不随观测到的 p 值变化。

先看 M1。三次独立运行的均值差为 +39.337、+31.117、+22.337 ns，三个 95% 区间都不含 0；逆方差合并为 +32.311 ns，区间 [20.282, 44.339]。Holm 家族 $m=6$，最小 $p=1.30\times10^{-5}$，低于阈值 0.00833，是家族中唯一被拒绝的检验。效应占完整解封装均值 57,010.7 ns 的 0.0567%，约 0.06%；32 ns 在 10 ns 的最小步进下只有 2–3 个分辨率单位，单次调用不可分辨，结论靠每类 10,000 次、三次运行。dudect 的最大 t 为 26.41、40.64、14.92，RTLF 在未修臂报告差异、在已修臂未报告差异，方向与 Welch 检验一致。

三次运行的方向一致，比单次拒绝更重要。三次分别换种子、换物理核：同核不同种子两次，同种子不同核一次；三次都给出正号、区间都不含 0，说明结论不依赖某一组输入或某一个核的局部状态。反过来说，如果三次方向不一致，即使合并后拒绝零假设，也要按预注册

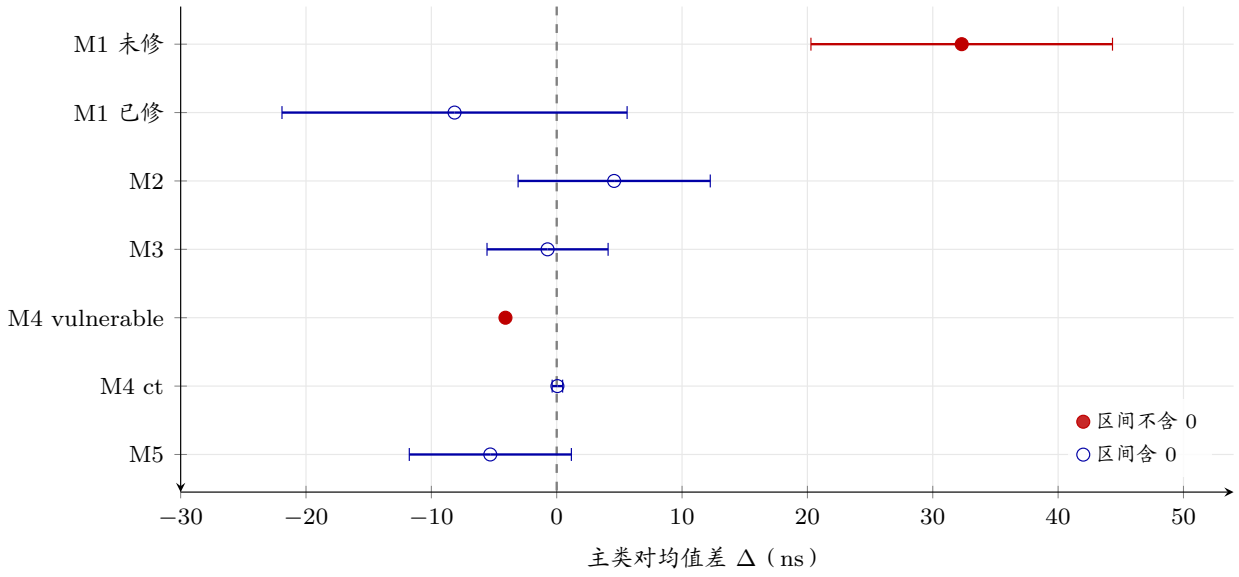


图 5: 五模块主类对均值差与 95% 区间

图注：M1 与 M4 vulnerable 臂区间不含 0，其余模块未检出。区间为三次独立运行的逆方差合并结果；M4 是 32 B 比较调用，数值不可与完整解封装直接比较。

规则登记为不稳定，不能挑一次最有利的运行写进结论。M1 的三次结果没有触发这条规则。

效应大小与统计判定要分开读。32.311 ns 只占 57,010.7 ns 的 0.06%，在绝对值上很小；它的统计判定来自每类 10,000 次、三次运行与预注册家族，不来自效应大。把“拒绝零假设”当成“效应大”会误导读者：这个效应在单次调用上不可分辨，在本地锁频、大样本下才可分辨；远程或小样本下可能完全看不到。

M4 的阳性更强。vulnerable 臂三次运行的原始 p 为 0、9.23×10^{-25}、3.14×10^{-22}；0 是双精度下溢，正文写“小于最小可表示值”。Holm 家族 $m = 6$，拒绝 3 项，全部来自 vulnerable 臂；合并差 -4.089 ns，区间 $[-4.215, -3.963]$。ct 臂合并差 +0.055 ns，区间 $[-0.363, 0.473]$，原始 p 为 0.386、0.438、0.399，三项都不拒绝。dudect 的最大 t 从 vulnerable 臂的 356.6、396.9、395.6 降到 ct 臂的 2.06、1.90、2.27。M4 是 32 B 比较调用，用原始单调时钟计时，数值不可与完整解封装的 32 ns 直接比较。

M4 的区间窄得几乎看不见，是因为比较调用的方差远小于完整解封装。32 B 比较的基线远小于完整解封装，单次调用没有 NTT、采样与序列化的开销，噪声源少，标准误小；同样的每类 10,000 次，M4 的 MDE 只有 0.19–1.19 ns，而完整解封装的 MDE 在几十纳秒。区间窄不等于效应大：-4.089 ns 在绝对值上只有 M1 的八分之一，但它的信噪比高，所以在图上反而更醒目。

效应绝对值与可检出性强弱并不同向：M4 只有 4 ns，MDE 低到 1.47 ns，信噪比高；M1 的 32 ns 更大，完整解封装的方差也大，MDE 高到 31.39 ns，信噪比反而低。判断可区分性要看区间与 MDE，不看点的位置。

M2 的家族没有拒绝项。完整解封装合并差 +4.58 ns，区间 $[-3.08, 12.25]$，含 0；按臂描

述，未修臂 +12.35 ns、已修臂 -11.20 ns，$I^2 = 58.3\%$。两个方向相反、异质性偏高，读作臂间漂移，不读作除法泄漏。按臂合并值不属于预注册的 6 个家族成员；家族成员是三次运行的逐次检验，按臂值只作描述。

主类对不是为除法操作数设计的：message-zero 与 message-ones 只让解密消息的位不同，除法的被除数是否随消息位系统变化没有控制；这一条在 3.7 的检测预算里再展开。M2 的 dudect 阳性落在已修臂（4.68、3.05、3.66 对 36.27、30.94、61.91），不能归因于除法；3.12 把它作为单工具误读的例子。

M2 的按臂描述还暴露了主类对的设计问题。未修臂 +12.35 ns、已修臂 -11.20 ns，两个方向相反；如果主类对真的触发了除法，两个臂应该同向，只是幅度不同。方向相反说明两臂的差异更多来自运行间的漂移，而不是同一个机制。$I^2 = 58.3\%$ 是这种漂移的量化：合并估计只作描述，判定仍以预注册的 Holm 结果为准。

M3 的合并差为 -0.73 ns，区间 $[-5.56, 4.10]$，Holm 拒绝 0；M5 的合并差为 -5.30 ns，区间 $[-11.77, 1.16]$，家族 $m = 3$，拒绝 0。M5 是阴性对照，未检出只说明在本机、本编译产物、本类对与每类 10,000 次的预算下没有看到可区分差异；它不是安全性结论。M3 是人工夹具，on/off 两臂只差人工查表，计时层未检出说明该机制在本机没有被端到端计时分辨出来，不说明查表没有风险。M5 的 dudect 最大 t 为 2.10、1.98、2.82，也在阈值 10 以下，与 Welch 检验同向。

M5 的 3 项家族成员没有一项接近门槛：合并差 -5.30 ns，区间跨 0，dudect 最大 t 只有 2.10、1.98、2.82，远低于阈值 10。三个来源都指向“本预算下未检出”，但没有一个能证明“没有机制”。阴性对照的价值在于检查工具会不会冤枉好人，不在于证明好人一定没问题。

端到端 Δ 的符号不能按分支代价模型解释。符号受二进制布局、对齐与指令调度影响；主实验的 +32.3 ns 与优化级别实验的 -26.16 ns 采用不同的 Δ 定义方向，同一机制在不同构建中给出相反符号。本作品只按“区间是否含 0”判定可区分性，不解释符号，也不把符号当作机制证据。

图 5 的纵轴按模块排列，不按 p 值排列。按 p 值排列会让读者以为“上面的模块更重要”；按模块排列，读者可以先看同一模块的未修/已修或不同臂，再看跨模块的比较。M4 的两行挨在一起，就是为了让 vulnerable 与 ct 的区间直接对照。

族外检验给出另一层信息。RTLF 在 $\alpha = 0.01$、固定种子与 10,000 次 bootstrap 的设置下，对 M2–M5 的 42 个数据集给出 30 个未检出、12 个检出；检出的 12 个集中在 M2 主类对与 M4 vulnerable 臂及其对照，M3 的 12 个与 M5 的 6 个数据集全部未检出。RTLF 与 Welch 检验比较的对象不同：前者比较分布与分位，后者比较均值；两者不互相替代，也不并入同一个 Holm 家族。[8]

族外检验不并入 Holm，是因为零假设与错误控制方式不同。RTLF 用 bootstrap 控制第一类错误，dudect 用最大 t 的经验阈值，Mona 用箱体分离；把三者塞进同一个 Holm 家族，会得到一个既不是 0.05 也不是 0.01 的混合错误率。它们的位置是交叉印证：同向时增加信心，

反向时提示输入类或环境有问题，而不是投票。

dudect 也属于族外。它用在线 t 检验，阈值是上游默认的 10，不转换成 p 值参与 Holm；M1 未修在三批内停止并报告泄漏，已修跑满批次仍为“可能常数时间”；M4 vulnerable 三次都报告泄漏，ct 臂三次都未报告；M5 三次都未报告。[7]

主类对是同一份确定性密文的 10,000 次重复，单次运行的区间反映测量噪声，不是密文间的变异；跨种子复测部分弥补这一点，但不等价于在每类内部使用大量不同密文。10 ns 的量化尺度下，32 ns 的效应只有 2–3 个分辨率单位；这不是把 32 ns 说小，而是说明为什么必须用大样本与预注册家族。图 5 不能回答“效应从哪来”。区间只说明两类密文的均值差在本预算下是否可区分，不说明差异来自哪条指令；M1 的差异要结合 3.4 的分支定位才能归因，M4 的差异要结合 3.6 的 ct 臂对照才能确认。把图上的点直接读成“某条指令的代价”，会把统计证据与机制证据混在一起。

图 5 证明在本机、本预算下哪些类对可区分；它不证明机制存在或不存在，不证明可远程利用，也不把 M5 的未检出写成安全。

3.4 函数级泄漏定位

定位层把每个阳性落到函数、指令或源码行；定位阳性与计时阳性是两件事，必须分开登记。同一个模块可以在定位层阳性、在计时层未检出，M2 就是这种组合；反过来，计时层阳性也必须能回到一条具体指令，否则无法判断修复是否生效。

动态检查的工作方式决定了它能回答什么。秘密标记把私钥中参与解密的 1152 B 与隐式拒绝秘密的 32 B 标成“未初始化”，公开输入保持已初始化；程序在标记下跑完整解封装，工具报告“以秘密值为分支条件”与“以秘密值为访存下标”两类事件。错误数是动态命中次数，上下文数是去重后的源码位置数；同一个循环执行 1280 次会报 1280 个错误，但只算 1 个上下文。报告里同时给两个数，是为了让读者看到“问题分布”而不是“执行次数”。

M1 未修版本的消息编码函数里，编译器把源码中的掩码选择还原成按位测试后条件跳转：先测试消息位，再由条件跳转决定是否写入系数。私钥标记下，Memcheck 报告 1 个秘密条件跳转。修复版本改成置位加条件传送：同一个按位测试之后用置位指令物化消息位，再调用 16 位条件传送辅助函数完成选择；反汇编里条件跳转消失，Memcheck 报告 0。两版的功能输出逐字节相同，说明消失的是执行路径，不是语义。[5]

反汇编上的差别可以逐条读。未修版本是“按位测试 + 条件跳转”：测试结果决定是否跳过写系数；修复版本是“按位测试 + 置位 + 调用条件传送辅助函数”：测试结果被物化成 0 或 1，辅助函数用取负、异或、与、再异或完成选择，不产生依赖秘密的跳转。两条路径的指令数不同，但都不依赖秘密；修复的代价是多了几条算术指令与一次调用，收益是分支消失。

M2 的三处整数除法分别落在消息解码、多项式压缩与向量压缩。未修版本在可变时延检查下报告 1280 errors / 3 contexts；修复版本用乘法与移位替代除法，除法计数从 3 降到 0，可变时延报告 0/0。关闭可变时延检查时，两版都报告 0，说明普通 Memcheck 看不到除法：它

只检查秘密是否进入分支或访存下标，不检查指令时延是否随操作数变化。1280 次错误来自同一源码位置的多次执行，3 个上下文才是问题分布。[4]

三处除法各自对应一个上下文：消息解码一处、多项式压缩一处、向量压缩一处。三处的被除数都来自解密消息或重加密中间值，除数都是公开模数；修复用乘法与移位替代除法，三处的除法计数同时降到 0。varlat 的 1280 次错误分布在三个上下文上，说明三处都被执行到；如果只修一处，另外两处仍会报错。

可变时延检查的边界也要写清楚。它标记的是“秘密操作数进入可变时延指令”，不是“这条指令一定产生可测差异”。除法在 x86_64 上的时延是否随操作数变化，取决于处理器微架构、操作数大小与流水线状态；工具能指出“这里有秘密操作数”，不能指出“这里一定泄漏多少纳秒”。M2 的定位阳性与计时未检出并不矛盾：前者是机制层，后者是端到端层，两层的判据不同。

M3 的 Memcheck 报告 1280 errors / 2 contexts，全部落在人工插入的秘密下标读取；off 臂去掉查表后为 0/0。Microwalk 在同一对象上给出互信息 1.922 bits，是全场最高；off 臂为 0.722 bits。Microwalk 只有 5 个公开 testcase、没有重复、没有统计检验，互信息只作定位层的独立佐证，不升级为泄漏量。[10]

互信息 1.922 bits 的含义是“看过这条指令的访存地址后，对秘密的不确定性平均减少 1.922 bit”。它是关联强度，不是泄漏量；1.922 bits 不能换算成“恢复多少位密钥”。Microwalk 的 5 个 testcase 只覆盖 5 组公开输入，没有重复、没有统计检验；互信息高只说明这条指令与秘密关联强，不说明攻击者能利用它。

M4 的 Memcheck 报告 32 errors / 1 context，落在提前返回的比较行。Binsec 在同一对内核上判 vulnerable 为 insecure，反例地址对应提前返回行；ct 臂判 bounded secure，路径探索完整。把搜索深度限为 5 时，工具返回 unknown，而进程退出码仍是 0；只看退出码会把 unknown 读成安全。M4 只开了控制流与访存检查，未开乘除法检查；它的结论只覆盖这对 32 B 比较内核。[11]

Binsec 的反例地址对应提前返回行，说明工具找到了一条“从第几个字节开始不同”的秘密相关路径。ct 臂的 bounded secure 不是“所有输入都安全”，而是“在入口、终点与公开长度 1–16 B 的范围内，没有找到秘密相关路径”；范围外的输入不在证明里。深度限 5 的 unknown 是另一个边界：工具没有证明不安全，也没有证明安全，退出码却是 0。

M5 在三种动态模式下各做 10 次完整解封装：普通 Memcheck、关闭可变时延、开启可变时延，三者都是 0 errors / 0 contexts。重加密路径的 Binsec 检查中，比较步骤 1088/1088 条路径完成，消息编码 1/1 条路径完成，没有秘密相关控制流；同一配置下 M4 的脆弱内核报告 insecure，说明工具在这套配置里不会把已知脆弱代码判成安全。Binsec 只覆盖重加密比较与消息编码，不覆盖完整解封装的所有性质。

三种动态模式的分工不同：普通 Memcheck 看分支与访存下标，关闭 varlat 看同一组输入下是否有普通标记错误，开启 varlat 看除法等可变时延指令。三者都是 0/0，说明在这组输入

上没有这三类问题；换一组输入，未执行的路径不会被报告。Binsec 的 1088/1088 与 1/1 是路径计数，不是“所有路径都安全”的证明；它只覆盖重加密比较与消息编码两个内核。

定位层的输出要归一化成同一种证据记录：工具与版本、输入集合、命中位置、上下文数、处置结论。同一位置被两类工具命中记两条证据，只有一类命中时保留但不升级为“确认”；归因要求同一位置至少在三次运行中的两次出现，且换种子后仍出现，只出现一次的命中记为“待复核”。

五类工具的回答范围可以排成一条链：反汇编回答“机器码里有没有目标指令形态”，不回答运行时是否真的触发；秘密标记回答“秘密是否进入分支或访存下标”，不回答可变时延指令；可变时延检查回答“秘密是否进入除法等指令”，不回答缓存与分支；轨迹互信息回答“哪些指令与秘密的关联最强”，不回答时序可利用性；有界符号执行回答“给定范围内是否存在秘密相关路径”，不回答范围外的行为。链条上的每一环都只回答一个问题，合起来才覆盖机制层的几个侧面。

这组结果证明机制在该产物、该输入、该工具下存在或不存在；它不证明完整解封装一定可测，也不证明未命中的输入没有机制。M4 是 32 B 比较夹具，不外推到完整解封装；Binsec 是有界验证，只覆盖给定内核与公开长度范围；动态工具只覆盖执行过的输入，0 条报告不是安全性结论。定位阳性与计时阳性分开登记：M1 两层都阳性，M2 定位阳性、计时未检出，M3 定位阳性、计时未检出，M4 两层都阳性，M5 两层都未检出。

3.5 泄漏源全流程排查

90 格排查把“六类来源 × 五个步骤 × 三个实现”逐格判定。5 个“存在”之外，还有一处更需要解释：官方修复后仍残留的同源分支。判定只有四种：存在、不存在、不适用、未检查。判“存在”必须给出可指认的函数、指令或源码位置以及触发它的输入；判“不存在”必须写明用了哪种检查、覆盖了哪些输入；判“不适用”要求该机制在这一步骤上不可能出现；判“未检查”单列，不并入阴性。

五个步骤是解密、哈希派生、重加密、比较与隐式拒绝。把哈希派生从重加密里拆出来，是因为哈希路径上的问题如果被记到重加密名下，定位只能停在“重加密有问题”这一层；拆开后，每一步都有自己的函数与指令，修复后可以逐格复查。六类来源是敏感分支、提前返回、可变时延指令、秘密下标、数据相关循环与异常路径；三个实现是 M1、M2 与 M5，人工夹具 M3、M4 不进入这张表，因为它们只覆盖单一机制，放进 90 格会把夹具结论与真实实现混在一起。

算法 8 90 格泄漏源排查

输入： 实现集合 {M1, M2, M5}；步骤集合 { 解密, 哈希派生, 重加密, 比较, 隐式拒绝 }；六类来源

输出： 90 格判定与证据

1: **for** 每个实现、每个步骤、每类来源 **do**
2: 检查源码与反汇编
3: 运行 Memcheck、varlat、Microwalk、Binsec（适用时）
4: 判定：存在 / 不存在 / 不适用 / 未检查
5: 记录函数、源码行或指令地址、证据编号
6: **end for**
7: 汇总：存在 5 / 不存在 75 / 不适用 10 / 未检查 0

算法 8 回答的是每一格在给定产物与检查手段下是否存在机制；它不回答该机制是否能被端到端计时分辨。表 8 给出汇总。

表 8: 全流程排查汇总

实现	存在	不存在	不适用	未检查	小计
M1 liboqs 0.10.0（未修 / 982c762）	1	26	3	0	30
M2 Kyber768（a621b8d / 272125f）	3	24	3	0	30
M5 mlkem-native b3ba7b3	1	25	4	0	30
合计	5	75	10	0	90

表注： 90 格 = 5 存在 + 75 不存在 + 10 不适用 + 0 未检查；M1 1、M2 3、M5 1。分模块行按 30 减去存在与不适用导出（B-06），全表见附录 F。

5 个“存在”逐格如下。M1 未修/重加密/敏感分支：消息编码函数中的按位测试后条件跳转，Memcheck 1 个上下文。M2 未修/解密/可变时延指令：消息解码中的整数除法。M2 未修/重加密/可变时延指令：多项式压缩与向量压缩中的整数除法。M2 未修与已修/重加密/敏感分支：消息编码函数中的同一分支在两版中都存在。M5/隐式拒绝/提前返回：上游设计行为，按有界例外登记。

这 5 格不是按“看起来可疑”选的，而是按“有可指认的函数、指令或源码位置，且有触发输入”选的。判“存在”的门槛比判“不存在”高：后者只要检查手段覆盖了输入且没有命中，前者必须给出触发路径。5 格里有 4 格在 M1/M2，1 格在 M5；M5 的那一格是上游设计行为，按有界例外登记，不算缺陷。

“不适用”的 10 格不是“没查到”，而是机制与步骤的组合不可能出现。例如某一步没有异常处理，异常路径一列就判不适用；某个函数不可能失败，提前返回一列就判不适用。判

“不适用”要求给出“为什么不可能”的理由，而不是因为没时间查。10 格不适用集中在 M1 与 M2 的异常处理组合，M5 有 4 格，是因为它的错误路径设计与上游不同。

第 4 格是排查中最需要解释的一格。KyberSlash 官方修复提交只消除了除法，消息编码函数中的按位测试后条件跳转仍在；未修与已修两侧的指令字节逐字节相同，秘密标记 Memcheck 在两侧各报 256 errors / 1 context。该函数体与 M1 未修版本相同，即与 clangover 同源。主类对 message-zero 与 message-ones 恰好使该分支在每个消息位上走系统性不同的路径，这为 RTLF 在 M2 两臂主类对上均报告分布差异、而在格式破损对照类对上未报告提供了一个机制层面的候选解释。[1]

这条观察有三条边界。第一，本作品没有做逐指令时序归因，不能把 RTLF 观测到的差异直接归于该分支。第二，已修臂三个种子的类间偏移符号翻转（+21.7、−17.1、−15.5 ns），离散度约为未修臂的 1.9 倍，未受控漂移仍是并列因素。第三，在同一分支存在的情况下，M2 预注册家族仍为 Holm 拒绝 0，其原因（-Oz 与 -Os 的代码布局差异、M2 噪声水平更高）尚未分离，列为待补实验。

90 格的证据字段包括实现与产物摘要、步骤、来源类别、判定、函数名或指令地址、触发输入、检查手段与证据编号。换一个人按同样字段复核，结论应当一致；判“存在”的格子必须有触发输入，没有触发输入的“存在”只能算“疑似”，汇总时降级为“未检查”。这套字段让 90 格既是排查表，也是审计清单。

三条边界指向同一个结论：这是候选解释，不是因果证明。要把候选解释升级为因果，需要逐指令计时、把分支单独抽出来做微基准，或者在关闭分支的构建上复测；本作品没有做这三件事。把“观察到的同源分支”写成“该分支导致了计时差异”，会把机制层观察与因果归因混在一起；把“Holm 拒绝 0”写成“分支不存在”，则是把统计结论与产物结论混在一起。

这是历史提交上的机制层观察，不是新漏洞；未修复、未复测，列为后续工作第一位。当前上游代码是否仍有此问题不在本作品验证范围内；这条观察停在机制层，不构成新漏洞的结论。

把这条观察放进报告，是因为它能回答“官方修复是否等于产物常数时间”。答案是否定的：修复消除了除法，但没有消除同源分支；分支是否可被计时利用，本作品没有做因果归因，只把它登记为候选解释。这个区分很重要：机制层观察可以写，因果结论不能写。

排查表的“不存在”也不是“永远不存在”。它只在给定产物、给定输入、给定工具下成立；换编译器、换优化级别、换输入类，结论可能变化。3.10 的 12 格就是例子：同一份源码在 -Os 有分支，在 -O0 没有；如果只查一个级别，就会把“这个级别没有”写成“这个实现没有”。

表 8 证明在给定产物、输入与工具下 90 格中没有发现其他机制，且没有把未做写成阴性；它不证明这些实现是常数时间。90 格的“不存在”只在所用检查手段的覆盖范围内成立：反汇编看不到运行时时序，动态工具只覆盖执行过的输入，有界验证只覆盖给定范围。源码层

[1]官方修复提交 272125f；另一提交 dda29cc 只修第一处。提交号仅用于定位历史版本。

面的常数时间写法，经过编译器、链接器与指令选择之后不一定仍是常数时间产物；判定必须落在实际二进制上，并随编译条件、架构与测量预算一起表述。

3.6 防护前后复测

M1 与 M4 在定位层和计时层都实现了“修复前可区分、修复后不可区分”；M2 只能在定位层说“机制消失”。表 9 把四个模块的防护前后放在同一张表上，证据层一列写明结论属于哪一层。

表 9: 防护前后对比

模块	证据层	防护前	防护后
M1	完整解封装（合并）	+32.311 ns [20.282, 44.339]，3/3 区间不含 0	−8.150 ns [−21.916, 5.616]，3/3 区间含 0
M1	dudect 最大 t / RTLF	26.41、40.64、14.92 / 有差异	1.15、1.12、1.31 / 无差异
M1	定位	按位测试后条件跳转；Memcheck 1	置位 + 条件传送；Memcheck 0
M2	定位	三处整数除法；varlat 1280 / 3	除法计数 0；varlat 0/0
M2	完整解封装（按臂描述）	+12.35 ns [2.98, 21.71]	−11.20 ns [−24.56, 2.15]
M3	定位	Memcheck 1280 / 2；1.922 bits	Memcheck 0/0；0.722 bits
M4	比较调用	−4.089 ns [−4.215, −3.963]；dudect 356.6–396.9	+0.055 ns [−0.363, 0.473]；dudect 1.90–2.27

表注：M1、M4 两层前后对照完成；M2 仅定位层机制消失。M2 的计时层在防护前就未检出，没有“降低”的基线可比较；M2 的按臂值只作描述，不参与 Holm。

复测的前提是同一规程。防护前后的计时使用同一个计时循环、同一个预注册家族、同一个功能门；修复后的二进制重新记录摘要，但输入类、样本量、种子与核不变。换一样东西，“修复后不可区分”就可能来自环境变化，而不是防护生效。M1 的修复版与未修版在同一台机器、同一锁频工作点上各跑三次；M4 的 ct 臂与 vulnerable 臂在同一驱动下各跑三次；M2 与 M3 同样成对。

四层前后对照的判据要分开写。定位层的完成判据是“目标机制在产物中消失且动态检查为 0”；计时层的完成判据是“修复后区间含 0 且族外检验不再报告”；功能层的完成判据是“输出逐字节相同”；性能层的完成判据是“开销有区间、可复算”。四层都过，才能说“这个机制在修复后不可区分”；只过一层，只能写那一层。M1 与 M4 四层都过，M2 只过定位层，M3 过定位层与功能层、计时层未检出。

M1 的修复把消息编码分支换成掩码选择。计时层三次运行有三次区间含 0，合并 −8.150 ns、区间 [−21.916, 5.616]；dudect 最大 t 从 26.41、40.64、14.92 降到 1.15、1.12、1.31；定位层从 1 个秘密条件跳转降到 0。功能层三对输出 diff 全空，性能开销 +1.51% [1.36, 1.65]。四

层放在一起，才能说“这个机制在修复后既不在产物里，也不在计时上可区分”。

M1 修复后的点估计是 -8.150 ns，不是 0；区间含 0，说明这个负值在噪声范围内。把点估计写成“修复后快了 8 ns”会误读：区间跨 0，不能拒绝“没有差异”。修复后的 dudect 最大 t 也只有 1.15、1.12、1.31，远低于阈值 10；三个来源都指向“不可区分”，但没有一个能证明“绝对没有差异”。

M4 的 ct 臂把提前返回改成遍历全部字节、循环结束统一判断。计时层合并 +0.055 ns、区间 $[-0.363, 0.473]$; dudect 最大 t 从 356.6、396.9、395.6 降到 2.06、1.90、2.27; 定位层 Memcheck 从 32/1 降到 0/0，Binsec 从 insecure 变为 bounded secure；两臂输出逐字节相同。M4 的两层都阳性、修复后两层都阴性，是四个模块里最完整的正对照；它的边界是 32 B 比较夹具，不外推到完整解封装。

M2 只能写到定位层。除法计数从 3 降到 0，varlat 从 1280/3 降到 0/0；计时层修复前就是 +4.58 ns、区间含 0，修复后没有“降低”的基线可以比较。按臂描述 +12.35 与 -11.20 的方向相反，$I^2 = 58.3\%$，读作漂移而不是修复效果。M2 的修复前基线本来就是未检出，写“泄漏降低”没有可比的基线。

M2 的 dudect 阳性落在已修臂，不能算作修复效果。已修臂最大 t 36.27、30.94、61.91，比未修臂的 4.68、3.05、3.66 高得多；如果只看 dudect，会得出“修复引入泄漏”。把 dudect 与 Welch、varlat 交叉，才知道阳性落在已修臂、且没有定位到除法；这不是修复效果，是另一种差异。

M3 的 on/off 两版输出逐字节相同，这是先决条件：如果输出不同，计时差异可能来自语义变化，而不是查表。off 臂 Memcheck 0/0，互信息从 1.922 bits 降到 0.722 bits；0.722 仍高于 0，说明即使去掉人工查表，其他指令与秘密仍有弱关联，它是基线，不是“零”。计时层两臂都未检出。M3 的防护是去掉人工插入的查表，不是修上游代码；它证明检测链路能识别并确认一个已知机制，不证明真实库有同样缺陷。

四条防护路径的代价不同。M1 的掩码选择只多几条算术指令与一次调用，对完整解封装的开销是 +1.51

四条路径也说明“防护”不是一个动作。M1 改的是编译器还原出的分支，M2 改的是源码里的除法，M3 改的是人工插入的查表，M4 改的是比较函数的控制流；四条路径分别落在产物、源码与夹具层。把它们放在同一张表上，是为了看“同一套复测规程能不能覆盖不同层的修复”，而不是把四种修复当成同一种技术。

防护复测还包括泄露渠道审计。对 128 个源文件中的 1,048 个输出或返回调用点做静态扫描，其中 122 个接触秘密相关数据，占 11.6%；秘密内容泄露 0 条。审计发现 1 条低危问题：功能自检分支使计时程序在不同构建下退出码不同，构成可观察信号；修复后该类问题由 1 条降为 0 条。动态差分对 7 种输入模式各运行 120 次，返回码、标准输出与标准错误逐字节一致。

防护手段与核验要一一对应。M1 用掩码选择替换分支，核验是反汇编里的条件跳转消失

与 Memcheck 1→0；M2 用乘法与移位替换除法，核验是除法计数 3→0 与 varlat 1280/3→0/0；M3 去掉秘密查表，核验是 Memcheck 1280/2→0/0 与互信息 1.922→0.722；M4 用遍历全部字节替换提前返回，核验是 Memcheck 32/1→0/0 与 Binsec insecure→bounded secure。每条核验都落在产物或动态报告上，不落在“源码改了”这句话上。

泄露渠道审计与计时复测是两条独立的证据链。审计回答“日志、异常、返回码是否泄露秘密”，计时回答“耗时是否可区分”；一条过、一条不过，不能互相替代。那条低危问题由审计自己发现并修掉，说明防护过程也被审计；它不说明其他渠道没有遗漏，审计不覆盖上游第三方代码、网络路径与进程级元数据。

表 9 证明在同一规程、同一家族、同一功能门下，修复前后的可区分性发生了变化；它不证明其他环境、其他输入类也如此，也不证明修复后的实现没有其他机制。防护复测的前提是功能等价：如果输出变了，“不可区分”可能只是算法变了。真实缺陷的修复采用上游官方补丁，团队自己写的修复只落在 M3、M4 两个人工夹具上；M2 的残留分支未修复、未复测，见 3.5。

防护复测的结论只覆盖同一规程。换一台机器、换一个编译选项、换一组输入，修复后的可区分性可能不同；M1 的修复版在 -O0 到 -Os 四个级别都是 0 个秘密相关分支（3.10），这是产物层的跨级别证据，不是计时层的跨环境证据。把“四个级别都是 0”写成“所有环境都不可区分”会越过层级。

3.7 检测预算与稳健性

“未检出”的确切含义是观测效应小于该预算下的 MDE，不是效应为 0；稳健性分析用来检查结论是否依赖均值型检验。表 10 给出各家族的 MDE，按 $\alpha = 0.05$ 与 Holm 口径两列并排。

表 10: 各家族最小可检测效应

家族	MDE（$\alpha = 0.05$）	MDE（Holm 口径）	判定
M1 未修	25.27 ns	31.39 ns	观测 +39.34 ns（r1）超过 MDE，可检出
M1 已修	28.63 ns	35.56 ns	观测 −13.21 ns（单次运行），小于 MDE，未检出
M2 端到端	19.9–54.0 ns	24.7–67.1 ns	6 项观测值均小于 MDE，未检出
M3 端到端	14.2–27.1 ns	17.7–33.7 ns	两臂观测值均小于 MDE，未检出
M4 比较调用	0.19–1.19 ns	最坏 1.47 ns	vulnerable 臂远超 MDE；ct 臂 0/3 超过
M5 端到端	11.7–24.5 ns	13.5–28.3 ns	观测值小于 MDE，未检出

表注： MDE 是事后功效描述，不改变已提交的判定。“未检出”的含义是观测效应小于 MDE，不是效应为 0。判定列的观测值取单次运行，合并值见 3.3 与 3.6。

MDE 按 $(z_{1-\alpha/2}+z_{1-\beta})\sqrt{s_0^2/n_0+s_1^2/n_1}$ 计算，Holm 口径把 α 取为 α/m；每类 $n=10{,}000$，功效 $1-\beta=0.80$。MDE 与样本量的平方根成反比：要把 MDE 减半，样本量要翻四倍。M2 的 MDE 24.7–67.1 ns 大于除法可能带来的效应，这是“定位阳性、计时未检出”的直接解释；主类对不是为除法操作数设计的，MDE 只能说明这个预算下看不见多大的效应，不能说明除法效应不存在。

公式里有三个假设：方差估计来自样本、均值差近似正态、功效取 0.80。每类 10,000 次下，正态近似可用；方差估计本身有不确定性，MDE 只能当量级参考，不能当精确边界。把功效从 0.80 提到 0.90，MDE 会变大；把 α 从 0.05 降到 0.01，MDE 也会变大。表 10 给的是 0.80 与 Holm 口径，换参数要重算。公式里的 SD 来自两臂合并的样本方差；两臂方差相差很大时，Welch 的 SE 比合并方差更稳，这也是主检验用 Welch 的原因。

逐行读表可以看到三种不同的“未检出”。M1 未修的观测 +39.34 ns 超过 MDE 31.39 ns，所以可检出；M1 已修的观测 -13.21 ns 小于 MDE 35.56 ns，所以未检出。M4 的 MDE 只有 0.19–1.19 ns，vulnerable 臂的观测远超它，ct 臂的观测 0/3 超过；M4 的预算足以分辨亚纳秒效应，是因为比较调用的方差小。M2、M3、M5 的观测都小于各自 MDE，未检出的含义是“这个预算下看不见”，不是“效应为 0”。

M4 的 MDE 只有 0.19–1.19 ns，是因为比较调用的方差小；这不代表 M4 的结论比 M1 强，只代表它的预算能看见更小的效应。M1 的 MDE 31.39 ns 比观测 39.34 ns 小，所以可检出；如果观测降到 25 ns，同样的预算就检不出。MDE 与观测的关系是“能不能看见”，不是“效应有多大”。

M2 的 MDE 跨度很大，是因为 6 个家族成员的方差不同。未修臂与已修臂的方差不同，三次运行的方差也不同；MDE 按每个成员自己的方差算，报告给区间而不是一个数。把 MDE 写成“25 ns”会掩盖这个跨度；写成 24.7–67.1 ns，读者才知道最坏情况下这个预算只能看见 67 ns 的效应。M2 的除法效应是否落在这个范围内，本作品没有独立估计；这正是把 M2 列为局限的原因。

稳健性分析对 64 个类对（其中 21 个为冻结家族成员）另用 p99.9 截尾均值差、中位数差与 Hodges–Lehmann 位置差三种方法复核。全样本方法与冻结家族的 Δ 最大绝对偏差为 0 ns；64 个类对中有 12 个在不同方法下发生判定翻转，家族成员 7 个、族外对照 5 个，但没有任何一个翻转改变 Holm 判定。

12 个翻转的分布说明主结论不依赖某一种统计量。家族成员的翻转没有改变 Holm，说明主家族的门槛足够严；族外对照本来就不进 Holm，翻转只作描述。翻转集中在边缘类对，主类对的区间离 0 足够远，换方法也翻不过来。翻转来自均值型与位置型检验的功效差异：均值型对长尾敏感，位置型对长尾稳健但功效低；在 10 ns 量化下，秩方法会把大量并列值当成同秩，功效进一步下降。把 12 个翻转原样列出，比只报“稳健”更有信息量。

表 10 与这组敏感性分析证明“未检出”有预算、结论不依赖单一均值检验；它们不证明所有方法都同意，也不证明未检出等于没有效应。MDE 不改变已经提交的 Holm 判定；把 MDE

当成精确边界，会把功效计算的不确定性藏起来。

MDE 是事后功效描述，不是事前承诺。采集前冻结的是样本量、家族与停止条件，MDE 是采集后按观测方差算出来的；它回答“这个样本量能看见多大的效应”，不回答“真实效应有多大”。把 MDE 当成“效应上限”会误读：观测效应小于 MDE，只说明这个预算下看不见，不说明效应不存在。

MDE 与观测效应的关系可以写成一句话：观测小于 MDE 时，检验没有足够的功效拒绝零假设；观测大于 MDE 时，检验才有机会拒绝。M1 未修的观测 39.34 ns 大于预算 31.39 ns，所以三次运行都能拒绝；M1 已修的观测 −13.21 ns 小于预算 35.56 ns，所以三次都不拒绝。把这两个关系倒过来读，会得出“M1 已修的效应更小所以更安全”的结论，但预算不同，不能直接比。

稳健性分析的 64 个类对不只是主类对。它们包括主类对、格式破损对照、截断对照与工具矩阵里的其他类对；21 个冻结家族成员是预注册的，43 个族外对照是描述性的。把 64 个都放进同一张敏感性表，是为了看“换一种统计量，结论会不会翻”；不是把 64 个都当成正式检验。族外对照的翻转不改变任何 Holm 判定，也不进入正文的主结论。

样本量的选择也是折中。每类 10,000 次、三次运行，是在机时与可检测下界之间取的；要把 MDE 减半，样本量要翻四倍，机时也翻四倍。M2 的 MDE 25–67 ns 大于除法可能效应，说明这个预算对 M2 不够；但加大样本量能否检出除法，本作品没有做，不能承诺。

3.8 异常输入与接口边界

格式破损与截断是两类不同的输入：前者进入 KEM 走完整重加密，后者在封装层被公开长度检查拒绝。两组对照都在 M1 两版与 M5 上做，三个种子、每类 10,000 次、18 次控制器运行全部通过核对。

对照 A 比较合法密文与长度正确但内容破损的密文：u 字段前 960 B 置 0xFF，v 字段后 128 B 置 0x00。9 个 Δ 在 −3.2 到 +9.8 ns 之间，9/9 区间含 0，原始 p 均不小于 0.28。破损输入仍然通过长度检查，进入解封装后比较失败，按隐式拒绝走完整重加密；耗时与合法输入接近，符合设计预期。

对照 A 的 9 个 Δ 来自 3 个对象 × 3 个种子，没有一组种子给出可区分差异；最小的一组 p 也远离 0.05，不是“边缘可拒绝”，而是完全在噪声内。破损输入走完整重加密，路径长度与合法输入相同，差异只能来自内容对计算的影响；这个影响小于 MDE。

这组对照的设计意图是让“路径长度”保持不变。u 与 v 的字段长度没有变，变化的是内容；解封装仍然走解密、重加密、比较、隐式拒绝四步，路径长度与合法输入相同。如果隐式拒绝的实现正确，两类输入的耗时差异只应来自内容对计算的影响，而不应来自路径；9/9 区间含 0 说明在这个预算下没有看到路径级差异。它不说明内容对计算没有影响，只说明影响小于 MDE。

对照 B 比较合法密文与截断为 1087 B 的密文。截断输入的单次调用耗时 23.1–23.4 ns，与

实现无关，说明它在进入 KEM 前就被封装层的公开长度检查拒绝；Δ 约为 29.3 μs（M5）与 55.1 μs（M1）。长度是公开信息，按固定路径拒绝是接口语义，不构成秘密相关泄漏，因此这组对照不进入主检验。

截断与破损的区别在接口层，不在 KEM 层。破损输入的长度合法，封装层放行，进入 KEM 后比较失败；截断输入的长度不合法，封装层直接拒绝，根本不进入 KEM。前者的耗时与合法输入接近，后者的耗时只有 23 ns；把两者混为一类，会把“公开长度检查”误读成“隐式拒绝”。

23.1–23.4 ns 是拒绝路径的耗时，不是解封装耗时。它比完整解封装小三个数量级，说明截断输入根本没有进入多项式运算；如果实现先把密文读进缓冲区再检查长度，耗时会更长，且可能随长度变化。长度检查放在封装层、按固定路径拒绝，是接口设计的一部分；把它写成“泄漏”会把公开信息与秘密信息混在一起。

这组对照回应题面第 3 条“异常密文”，证明破损与截断走了两条不同的路径；它不证明所有异常输入都安全，也不把“未检出”写成“没有风险”。格式破损的 9 个 Δ 都在噪声内，说明隐式拒绝没有在端到端计时上留下可区分差异；截断的差异来自公开长度，不是秘密。

这组对照与主家族分开登记，是因为它的输入类不是主类对。主类对只让消息位不同，破损与截断改变的是长度或内容；它们的路径长度或返回位置本来就不同，混进主家族会把“路径不同”误读成“秘密泄漏”。异常输入的样本量与主类对相同（每类 10,000 次），但输入类不同；把它并入主家族会让家族门槛与输入类不匹配。族外对照只作描述，回答“其他输入是否异常”，不回答“主类对是否可区分”；两者分开登记，读者才不会把“破损输入未检出”当成“隐式拒绝没有泄漏”。

3.9 性能开销

防护开销必须用成对分块测量，不能用“前后各跑一次”的差值；更快不等于更安全，更慢也不等于泄漏。表 11 给出五个对象的均值、中位数、p90、p99 与按名义频率换算的周期。

表 11: 防护前后解封装运行时间

版本	均值（ns）	中位数（ns）	p90（ns）	p99（ns）	均值（周期）
M1 未修	57,010.7	56,916.0	57,157.0	58,729.0	245,146
M1 已修	57,873.3	57,798.0	57,959.0	59,522.0	248,855
M2 未修	57,878.5	57,809.0	58,039.0	59,762.0	248,878
M2 已修	56,188.0	55,995.0	56,656.0	58,088.0	241,608
M5	29,455.7	29,404.0	29,455.0	31,077.0	126,659

表注：周期按名义 4.30 GHz 换算（B-02），仅用于跨机器粗略对齐，不代表指令数。

表 11 的均值与中位数相差不到 100 ns，p90 与中位数相差不到 300 ns，p99 的抬升也只有几百纳秒到 1.7 μs；分布的主体很窄，长尾没有把均值拉走。M1 未修与已修的均值差约 862.6

ns，M2 未修与已修的均值差约 1,690.5 ns；这两个差值与图 7 的相对开销对应。M5 的均值只有 M1/M2 的一半左右，是因为 mlkem-native 的参考路径更短；跨对象比较均值没有意义，只有同一对象的防护前后比较才有意义。

图 7 把相对开销画成成对条形：M1 修复 +1.51% [1.36, 1.65]，M2 修复 −2.92% [−3.06, −2.76]，M3 off 相对 on +3.60% [0.36, 9.98]，M4 ct 相对 vulnerable 首字节 +26.21%、末字节 +6.85%。

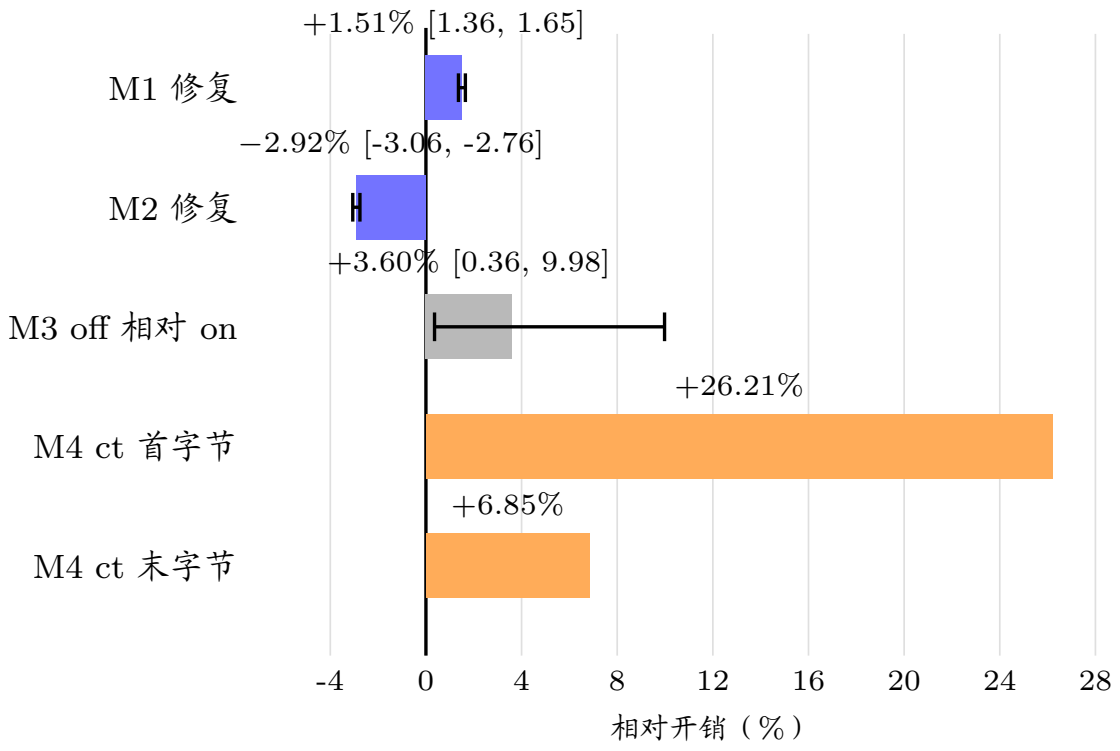


图 7: 防护前后成对相对开销

图注：M1 +1.51%，M2 −2.92%，M4 小内核 +26.21%/+6.85%。M3 的区间宽，只作描述；M4 是 32 B 比较调用，绝对差只有数纳秒，不能外推到完整解封装。

开销测量不是“前后各跑一次”。decaps 每臂 6 块 × 25,000 次，keygen 与 encaps 各 30,000 次，两臂按 ABAB 交错，相对开销用 bootstrap 10,000 次给区间。交错是为了让频率漂移、缓存状态与后台服务的影响同时落在两臂上；分块是为了让每块内部的顺序随机化。一次“修复前跑一遍、修复后跑一遍”的差值，无法区分防护开销与环境漂移。

M1 的 +1.51% 可以拆成“多出的指令”与“多出的调用”。掩码选择把一条分支换成置位、取负、异或、与、再异或，指令数增加几条；16 位条件传送辅助函数在 -Os 下没有内联，多一次调用与返回。两部分加起来是 862.6 ns，占 57,010.7 ns 的 1.51%。这个拆法只作解释，不作为精确归因；指令级开销要看反汇编与微基准。

M1 的 +1.51% 是四次测量中最稳定的一个：区间窄，方向单一，与掩码选择多出的几条指令相符。M2 的修复反而更快，M3 的区间很宽，M4 的相对值很大但绝对差很小；三个对

象的解释分别落在下面三段。

M2 的负开销来自编译器。除法被乘法与移位替代后，指令数减少，流水线停顿也减少；修复版反而比未修版快 2.92%。这不等于“修复更安全”，只说明“更安全”与“更快”可以同向，也可以反向。把负开销写成“优化”会偏离主题：本作品测的是防护开销，不是性能优化。

M3 的 +3.60% 区间 [0.36, 9.98] 很宽，是因为 off 臂的方差大、样本块少；这个区间只作描述，不用它推断总体开销。M3 是人工夹具，on/off 两臂只差一张查表；查表的开销在完整解封装里占比很小，测量噪声容易淹没它。把 3.60% 写成“查表的开销”会高估精度。

M4 的 +26.21% 与 +6.85% 是相对值，绝对差只有 4 ns 量级。首字节与末字节的差异不同，是因为比较提前返回的位置不同：vulnerable 臂在第一个字节不同时最快返回，在最后一个字节不同时最慢；ct 臂始终遍历全部字节。两个百分比不能平均，也不能与完整解封装的百分比比较。

把相对开销换算成绝对时间：M1 每解封装多 862.6 ns，M2 少 1,690.5 ns；M3 的 +3.60% 按 29,455.7 ns 的参考均值换算约 1.06 μs，但区间跨到 9.98%，绝对量的不确定性也很大；M4 的 +26.21% 只在小内核的比较调用上成立。绝对量与相对量一起看，才能判断这个开销在实际使用中的位置。

M1 的 +1.51

M4 的 +26.21

开销的置信区间来自 bootstrap，不来自正态假设。M1 的区间 [1.36, 1.65] 窄，说明 6 块 × 25,000 次的成对测量稳定；M3 的区间 [0.36, 9.98] 宽，说明 off 臂的方差大、块数少。区间宽时只作描述，不用它推断总体；区间窄时也不把点估计当成精确值，只说明这次测量的相对开销落在那个范围。

表 11 同时给均值与分位数，是为了让读者看到分布的形状。均值对长尾敏感，分位数对长尾稳健；两者相差不大时，均值可以作为摘要；相差大时，分位数更能代表典型调用。这批数据的均值与中位数相差不到 100 ns，说明分布没有被长尾拉偏；如果只给均值，读者无法判断这一点。

性能开销不是本作品的目标，防护效果才是。开销测量的作用是回答“防护的代价有多大”，不是“哪个实现更快”；把 M2 的 −2.92% 写成“性能优化”会偏离主题。报告给开销区间，是为了让读者判断这个代价是否可以接受，而不是为了给实现排名。

五个对象的行不是同一把尺子。M1 与 M2 是完整解封装，可以互相比均值；M5 也是完整解封装，但实现不同，均值的差异主要来自参考路径，不来自防护；M3 与 M4 是小内核，只用于相对开销，不进入这张表。表 11 只列能放在同一窗口里比较的对象，跨对象的均值差不在结论里。

周期列只用于跨机器粗略对齐。周期按名义 4.30 GHz 换算，而实测频率中位数是 4,309,722.5 kHz，偏差 +0.005%；用名义频率换算会带来同量级的系统误差。周期列不写进结论，也不与

别的机器比较指令数；它只是让读者对“57 µs 在 4.30 GHz 下大约是多少个周期”有个量级概念。

keygen 与 encaps 也做了测量，各 30,000 次，但表 11 只列 decaps。decaps 是防护的主要对象：M1 的分支、M2 的除法、M3 的查表、M4 的比较都在解封装路径上；keygen 与 encaps 的防护开销不是本轮重点。把 decaps 的开销写成“整个 KEM 的开销”会夸大适用范围。

开销测量与功能测试使用同一份产物；换构建要重测，这条约束与产物绑定一致。成对分块与 ABAB 交错解决的是“环境漂移”与“顺序效应”。如果先跑未修臂再跑已修臂，频率漂移、缓存升温与后台服务变化都会落在已修臂上；ABAB 让两臂在时间上交错，漂移同时落在两臂上。bootstrap 10,000 次给区间，不假设正态；区间窄说明相对开销稳定，不说明绝对耗时稳定。

表 11 的均值与图 7 的相对开销要一起读。均值给绝对量级，相对开销给可比的比例；只看均值，M1 与 M2 的差异只有 862.6 ns，容易被当成噪声；只看比例，+1.51% 又缺少绝对量级。两者并排，读者才能判断“这个开销在 57 μs 里占多少”。

表 11 与图 7 证明防护在本机带来的实测开销，以及成对设计能把环境漂移压到区间里；它们不证明开销低就一定安全，也不证明开销高就一定泄漏。M2 修复更快，不能写成“更安全”；M4 的百分比不能与完整解封装的百分比直接比较。

3.10 编译选项与测量条件

同一份源码在不同优化级别与测量条件下会呈现不同的产物形态与可检出性；结论必须绑定编译条件。图 8 给出 12 格的合并均值差与 95% 区间：M1 两版与 M5 各四个优化级别。

12 格来自 3 个臂 × 4 个级别。3 个臂是 M1 未修、M1 已修与 M5；4 个级别是 -O0、-O2、-O3、-Os。每个格做产物判定（反汇编、除法计数、Memcheck）与锁频计时（每格 3 个种子），两列证据分开登记。

把产物判定与计时判定分开，是为了避免“分支存在但计时没测到”被读成“分支不存在”；-O2 与 -O3 正是这种情况。每格三个种子的二进制摘要逐字相同，判定用的产物与计时用的产物对齐；这一条是 12 格可比的前提；换优化级别或编译器都要重新登记产物，这条约束不随环境改变。

12 格是族外探索，不进任何 Holm 家族。它的作用是看“产物形态与可检出性是否随编译条件变化”，不是给每个优化级别下安全结论；每格 3 个种子，样本量不足以做家族检验，也不足以估计总体。结论止于“该级别下没有观察到可区分差异”。

M1 未修的 -Os 格合并 -26.16 ns，区间 $[-37.51, -14.81]$，是 12 格中唯一不含 0 的区间；同一版本的 -O2 与 -O3 分别合并 -10.86 与 -7.55 ns，区间都含 0；-O0 没有秘密相关分支，源码的算术掩码被原样保留。产物侧，-O2 与 -O3 的秘密相关分支位置数为 8 与 15，比 -Os 的 1 个更多；分支更多，计时反而未检出。修复版四个级别都是 0 个秘密相关分支，M5 四个级别都是 0，12 格都没有整数除法指令。

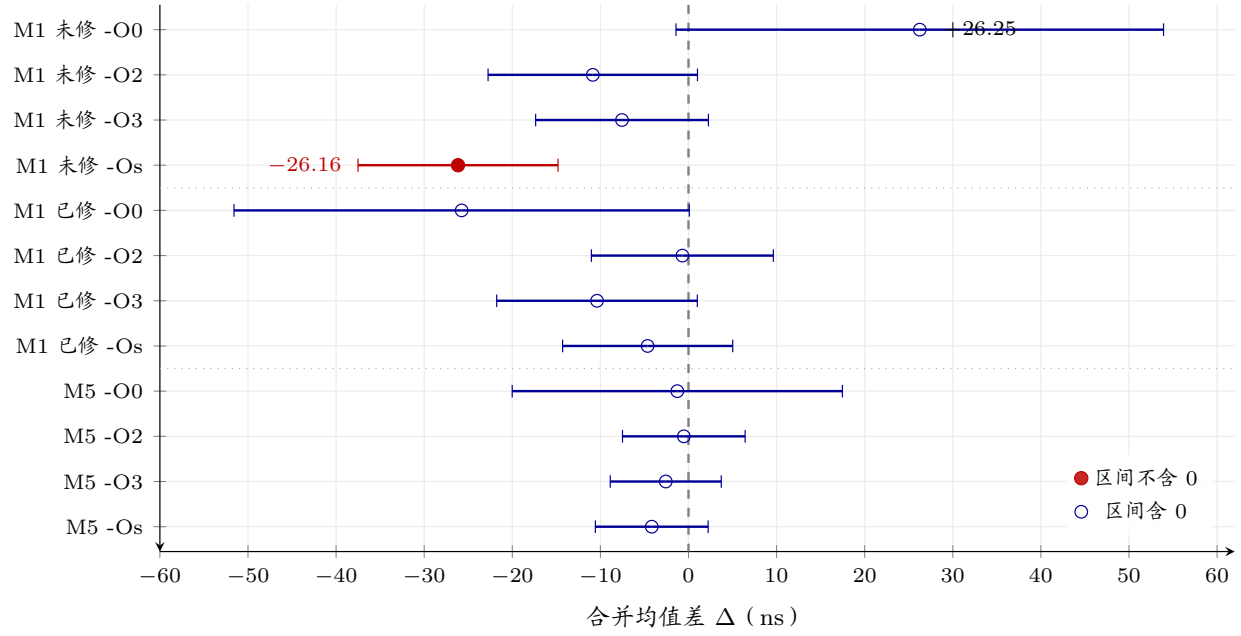


图 8: 优化级别 12 格合并均值差与 95% 区间

图注：只有 M1 未修 -Os 格区间不含 0；-O2/-O3 分支更多但计时未检出。区间为三个种子的逆方差合并；同一格三个种子使用同一份二进制。

其余 11 格的区间同样列出。M1 未修 -O0 合并 +26.25 ns，区间 [−1.42, 53.92]；-O2 −10.86 [−22.75, 1.02]；-O3 −7.55 [−17.35, 2.26]。M1 已修 -O0 合并 −25.75 [−51.59, 0.09]，上界紧贴 0；-O2 −0.69 [−11.02, 9.63]；-O3 −10.38 [−21.77, 1.00]；-Os −4.64 [−14.29, 5.02]。

M5 四格都在 −5 ns 以内，区间都含 0。只有 -Os 未修格的距离离 0 足够远。

“分支更多反而未检出”有两层解释。产物层：-O2/-O3 的每比特测试是“内存操作数加条件跳转”，形态与 -Os 的寄存器位测试不同；分支数量多，不等于单条分支的代价大。计时层：分支的代价被流水线、分支预测与乱序执行吸收，端到端均值差可能小于 MDE；-O2/-O3 的区间都含 0，说明这个预算下没有分辨出来。两层解释都指向同一个结论：不能从“分支数量”推断“可检出性”。

锁频与不锁频的对照给出另一个反直觉结果。三个对象的 CV 比为 0.964、1.006、1.089，SD 比为 0.741、0.757、0.956；不锁频的标准差反而更低。M1 未修在两种条件下都是 3/3 检出，M5 都是 0/3；判定一致，稳定性指标的方向却不一致。

CV 比的方向不一致，说明“锁频降噪”这个直觉在这台机器上不成立：M1 未修 0.964、M1 已修 1.006 小于或等于 1，M5 1.089 大于 1；如果锁频一定降噪，三个比值应该都大于 1。SD 比都小于 1，是因为不锁频时频率更高、单次调用更短，绝对标准差随均值一起缩小；换成 CV 后，两者的相对离散度接近。锁频真正提供的是可核对的工作点：每次运行记录 20 次探针、退出核对恢复，审计者能确认“这次采集在什么频率下完成”。

-O0 格没有秘密相关分支，是“该级别下这个机制没有出现”，不是“这份源码安全”。源码的算术掩码被原样保留，编译器没有把它还原成条件跳转；换一个编译器或换一个 -march，

-O0 也可能出现分支。把 -O0 的 0/0 写成“最安全的级别”会越过产物边界；本作品只写“该级别下没有观察到该机制”。

修复版四个级别都是 0 个秘密相关分支，说明官方修复在不同优化级别上都生效；它不说明修复版在计时层也不可区分。M1 已修的四个级别合并值分别为 -25.75、-0.69、-10.38、-4.64 ns，区间都含 0；这是产物层与计时层分开登记的结果。产物层 0 个分支、计时层区间含 0，两者同向，但不能互相替代。

三台 WSL2 虚拟化环境的复现把“机制”和“可检出性”分开。确定性证据与主平台一致：M1 未修 -Os 1 个、-O2 8 个、-O3 15 个、-O0 0 个 Memcheck 上下文，修复版与 M5 全为 0，反汇编形态逐项相同。时间证据不同：三台机器上 M1 未修 -Os 的合并区间都含 0，M4 vulnerable 的合并区间都不含 0（-7.13、-11.97、-22.37 ns）。机制存在与否可以跨环境复现，数十纳秒效应的可检出性依赖测量环境；WSL2 数据不进任何 Holm 家族，也不与裸金属数据合并。

WSL2 的三台机器不是三台测量机。它们是虚拟化环境，不锁频，时间数据的标准误在数百纳秒；三台机器的一致之处是确定性证据（反汇编、Memcheck），不是时间证据。M4 vulnerable 在三台机器上都给出不含 0 的区间，说明管线在效应足够大时仍能检出；M1 未修 -Os 的区间都含 0，说明 32 ns 效应在虚拟化噪声下不可检出。

WSL2 的结果还说明“效应足够大时管线仍能检出”。M4 的 4 ns 效应在虚拟化噪声下仍能给出不含 0 的区间，是因为比较调用的方差小、信噪比高；M1 的 32 ns 效应在 WSL2 上反而测不到，是因为完整解封装的方差大、虚拟化噪声高。可检出性不是只由效应绝对值决定，还由噪声与样本量决定；同一台机器上，M4 的 MDE 比 M1 小两个数量级，原因就在这里。

计时精度决定了什么样的效应可以被讨论。读钟开销约 17.4 ns，最小步进 10 ns；时间戳计数器的速率为 4,291,962 kHz，单次读数的分辨率下界为 42–43 个计数、约 9.8–10.0 ns；1 ms 短区间的频率极差为 348 kHz，相对标准差 5.4×10^{-6}。逐调用物理恒频不可主张：锁频协议固定的是工作点，不是每一次调用的频率。

时间戳计数器的读数不是物理时间的直接测量。TSC 速率 4,291,962 kHz 是按名义频率校准的；单次读数 42–43 个计数对应 9.8–10.0 ns，是分辨率下界，不是精度。1 ms 短区间的频率极差 348 kHz、相对标准差 5.4×10^{-6}，说明短窗内频率稳定；长期稳定性没有认证，逐调用物理恒频不可主张。

锁频/不锁频对照每组只有 3 次运行，只作描述。3 次运行不足以估计总体方差，也不足以推断其他处理器或调速策略；CV 比 0.964–1.089 只能说明“这台机器上方向不一致”，不能说明“锁频在所有机器上都不降噪”。把这条对照写成普遍结论，会超出样本边界。

图 8 与这组对照证明产物与测量条件会改变可检出性；它们不证明某个优化级别绝对安全，也不证明其他编译器、其他 CPU 会给出同样的结果。本机未配置 isolcpus，SMT 开启，后台约 39 个服务在运行；这些条件与 10 ns 量化一起，构成时间证据的环境边界。

3.11 架构与网络边界

M1 的泄漏机制是 x86_64 特定的编译器产物；同机网络各档在本预算下未检出，不能外推真实远程。图 9 把架构对照与网络检测预算放在一起：左侧是同一份源码在两个架构上的产物形态，右侧是五档网络分辨同一量级效应所需的样本数。

aarch64 的实验条件是“真指令集、原生二进制”。它在 Apple Silicon 的容器内运行，运行时报告的机器类型是 aarch64，不是 QEMU 翻译，也不是交叉编译后在 x86_64 上跑；同一份 liboqs 0.10.0 源码、同一个 clang 18、同一组优化级别，只换架构。这个条件让“产物形态不同”可以归因到架构，而不是源码或编译器版本。

容器环境的限制要写明：没有锁频，没有裸金属，CPU 与其他容器共享；这些条件不影响确定性证据（反汇编、Memcheck），但影响时间证据。aarch64 的 12 格只做产物判定与 Memcheck，不做锁频计时；把它的时间数据与 x86_64 合并会引入不可控的环境差异。

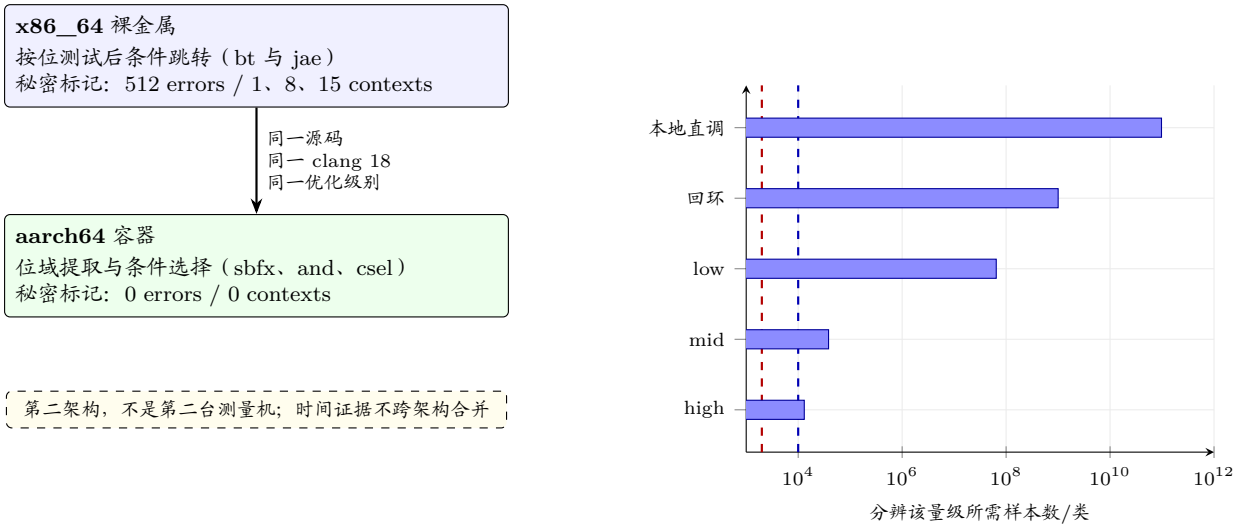


图 9: 架构与网络边界

图注：x86_64 的 bt/jae 在 aarch64 上变为 sbfx/csel；网络各档在本预算下未检出。右侧虚线为本轮实际样本数（网络每类 2,000，本地直调每类 10,000）。

aarch64 的 12 格全部是 0 errors / 0 contexts；x86_64 同格的 M1 未修是 512 errors / 8、15、1 个上下文。反汇编上，x86_64 是按位测试后条件跳转（bt 与 jae），aarch64 是位域提取、与掩码、条件选择（sbfx、and、csel），消息位的选择全部走无分支算术。同一份源码、同一个 clang 18、同一个优化级别，产物性质不同；M1 的机制不是“ML-KEM 参考实现在别的架构上也泄漏”，而是 x86_64 编译器在特定选项下的产物。

x86_64 的 512 errors / 8、15、1 个上下文，对应 -O2、-O3、-Os 三个级别；aarch64 同格都是 0/0。-O0 在两个架构上都是 0，因为源码的算术掩码被原样保留。三个不一致的格子全部落在“x86_64 加 M1 未修”，这个分布说明差异绑定的是架构与产物的组合，不是探针或输入。

12 格里 9 格一致、3 格不一致，不一致的全部集中在 x86_64 的 M1 未修臂。这个分布本身就是证据：如果差异来自探针或环境，应该在各臂上随机出现；差异只在“x86_64 加 M1 未修”这一组出现，说明它绑定的是产物形态，不是测量环境。aarch64 上消息位选择走无分支算术，Memcheck 没有秘密相关分支可报，0/0 是产物性质的结果。

aarch64 上 0/0 的原因可以直接从反汇编读出来：消息位的选择走位域提取、与掩码、条件选择，三个步骤都不产生依赖秘密的跳转；Memcheck 没有秘密条件跳转可报，也没有秘密下标可报。0/0 不是“工具没跑”或“输入没覆盖”，而是“产物里没有这个机制”；同一份探针在 x86_64 上报 512 errors，说明探针本身有效。

aarch64 跑在容器内，未锁频，不是裸金属；它只构成第二架构，不是第二台测量机。跨架构的时间证据不合并，也不要求同号；两个架构的探针实现不完全相同，M5 的跨架构结论只能由“两架构都是 0”弱支撑，不能写成强结论。把 aarch64 写成“第二台测量机”会夸大证据：它没有锁频，没有裸金属，时间数据不进入任何家族。

两个架构的探针实现不完全相同：x86_64 的 M5 四格用另一个探针，aarch64 用同一份历史探针；M5 的跨架构结论只能由“两架构都是 0”弱支撑。两架构都是 0 只支持“未检出”；探针不同，不能升级为常数时间结论。

同机网络场景用裸金属、匿名网络命名空间、自有 veth 与内核 netem 构造两端 TCP，分五档：本地直调、回环、low、mid、high。本地直调档 M1 未修两次运行一次可分辨（+37.1 ns，区间 [20.4, 53.8]）、一次未检出（-7.9 ns，区间 $[-25.0, 9.2]$）；网络各档全部未检出。分辨同一量级效应所需的每类样本数为 13,098、38,417、64,636,555、1,003,153,317、97,586,175,893；网络各档实际每类 2,000，比所需少 1.3–7.7 个数量级。

网络拓扑是同机两端：服务端在匿名 netns 内，绑到锁频 CPU2；客户端在宿主 netns，绑到 CPU0/1/3，不占 CPU2/CPU6。veth 两端用一对私有 /30 地址；netem 挂在 veth 两端，单向延迟与抖动分别是 100 μs/20 μs、1 ms/200 μs、10 ms/2 ms。这个拓扑能加入协议栈、排队与链路抖动，不能加入真实广域网的拥塞与路由变化。

网络五档的设计是为了分离“协议栈”“排队”与“链路抖动”三项开销。本地直调不走 socket，回环走 veth 与 TCP 但不加 netem，low/mid/high 逐级加入延迟与抖动；五档的检测预算逐级上升，说明每加入一层开销，分辨同一效应所需的样本数就增加。这个阶梯是定性的，不是对真实网络的拟合。

本地直调档的两次运行给出了两种结果：一次 +37.1 ns、区间 [20.4, 53.8]，一次 -7.9 ns、区间 $[-25.0, 9.2]$。两次的噪声水平接近，差别来自效应本身的运行间变动；把两次合并得到的参照效应区间含 0，只能当量级参照。这段结果说明：即使在同一台机器、同一锁频工作点，32 ns 级效应的可分辨性也不是每次都稳定；网络档每类只有 2,000，更不可能稳定分辨。

网络档的 netem 参数是单向 100 µs/20 µs、1 ms/200 µs、10 ms/2 ms，三档对应 low、mid、high。回环档不走 netem，只有 veth 与 TCP 的开销；本地直调档不走 socket，直接调用解封装。五档的设计是为了让“协议栈与排队”逐级加入，而不是一步跳到真实远程。即使最轻

的回环档，所需样本数也比实际多 1.3 个数量级；加上 netem 后，所需样本数上升到 10^7–10^{11}，实际每类 2,000 远远不够。

检测预算按 $n = 2(1.96\,\mathrm{SD}/|\Delta|)^2$ 计算，与 MDE 同口径；本地直调 13,098、回环 38,417、low 64,636,555、mid 1,003,153,317、high 97,586,175,893。网络各档实际每类 2,000，回环差 1.3 个数量级、low 差 4.5 个、mid 差 5.7 个、high 差 7.7 个。预算数字不是“需要这么多才能攻击”，是“需要这么多才能分辨这个量级的差异”；两者不能混。

网络档每类 2,000 的样本量是预算与机时的折中。按本地直调的 13,098 所需样本，2,000 只有 0.15 倍；按 high 档的 97,586,175,893，2,000 差 7.7 个数量级。样本量不足时，未检出是预期结果，不是“网络没有泄漏”的证据；报告把预算与差距一起写出，是为了让读者自己判断这个阴性有多强。

netem 的三档不是“轻度、中度、重度网络”的通用定义，是本作品选的三组参数。low 档 100 μs/20 μs 接近同城链路，mid 档 1 ms/200 μs 接近跨城，high 档 10 ms/2 ms 接近跨洲；这些类比只用于读者建立量级感，不是真实测量。真实远程的排队、拥塞与路由变化不在这个拓扑里。

这是同机模拟，不是真实远程；网络各档的“未检出”只说明在本预算下没有看到可区分差异，不说明远程链路上不存在泄漏，也不说明远程攻击不可行。图 9 证明机制与可检出性有环境边界；它不证明远程攻击可行或不可行，也不证明某个架构绝对安全。在同机模拟里，网络各档的检测预算比实际样本高 1.3–7.7 个数量级；在真实远程里，链路抖动、拥塞与路由变化只会让预算更高，不会更低。这个方向性判断不依赖具体数字，但它是推断，不是实测；本作品不把它写成“远程不可测”。

3.12 多工具交叉验证

单工具会给出方向相反的读法；交叉矩阵的价值不是“工具多”，而是让每个工具的“回答什么、不回答什么”落在同一张表上。图 10 是 9 工具 × 5 模块的矩阵：30 格有真实运行产物，15 格未运行。

矩阵的 9 个工具分三类：族内统计（Welch+Holm）、族外统计（dudect、RTLF、Mona）、定位工具（反汇编、Memcheck、varlat、Microwalk、Binsec）。三类回答的问题不同，矩阵把它们放在同一组模块上，是为了看“同一对象在不同判据下会不会给出相反读法”。15 个未运行格都写了原因：M2 没有完整解封装的 Memcheck 探针，M1/M2 没有 Binsec 抽取内核，M3/M4/M5 没有接入 Microwalk，Mona 只接入 M1/M2。未运行格不填阴性。矩阵按 2026-10-16 冻结的产物登记；M5 上的有界验证结果在矩阵冻结后补充，列在 3.4，不计入这张矩阵的 30 格与 6 工具统计。矩阵的格数不随补充结果改变；未运行格也不因补充结果改填阴性。

三类工具的回答范围不同。族内统计回答“均值差是否可区分”，族外统计回答“分布是否不同”，定位工具回答“哪里依赖秘密”。同一格在族内统计是阴性、在族外统计是阳性，不是矛盾：它们比较的对象不同。矩阵把三类放在一起，是为了让读者看到“同一对象在不同

	M1	M2	M3	M4	M5
dudect	●	△	○	●	○
RTLF	●	△	○	●	○
Welch + Holm	●	○	○	●	○
Memcheck	●	—	●	●	○
KyberSlash varlat	—	●	—	—	○
Binsec	—	—	—	●	—
Microwalk	—	—	●	—	—
Mona	△	△	—	—	—
反汇编	●	●	●	●	○

● 阳性（定位或计时） ○ 阴性 △ 已运行但不可归因或方向不一致 — 未运行
30 格有真实运行产物，15 格未运行；M5 上运行过的 6 种工具误报 0。

图 10: 多工具交叉矩阵

图注：30 格有真实产物，15 格未运行；M5 上 6 工具误报 0。阳性指该工具在其判定层报告了机制或可区分性，阴性指未报告；未运行不等于阴性。

判据下可以给出不同读法”，而不是让它们投票。

M1 是五路同向的例子：反汇编看到按位测试后条件跳转，Memcheck 看到 1 个秘密条件跳转，Welch+Holm 拒绝主类对，RTLF 在未修臂报告差异、在已修臂未报告，dudect 最大 t 在未修臂为 26.41、40.64、14.92、在已修臂为 1.15、1.12、1.31。五类工具回答的问题不同，结论同向。

五路同向增加信心，但不等于“五倍证据”：它们共享同一份产物与同一组输入，不是五个独立实验。

M4 是人工夹具中唯一定位层与计时层都阳性、ct 臂全阴性的正对照。M5 上运行过的 6 种工具全部阴性，误报 0；这个数字只说明这 6 种工具在这个对象、这些输入上没有报假阳性，不能估计总体误报率。

M4 的正对照作用是把“定位阳性”与“计时阳性”同时钉在一个对象上。如果 M4 的 ct 臂也阳性，说明检测链路有假阳性；如果 vulnerable 臂阴性，说明链路测不到已知机制。两种失败都没有出现，M4 才能作为正对照。M5 的阴性对照作用相反：如果 6 种工具在 M5 上报阳性，说明工具有误报；6 种都报 0，说明本次没有误报。

误报 0 的分母是 6 个工具 × 1 个对象，不是 6 个工具 × 全部输入。M5 上每种工具只跑了一组输入或一个探针；换一组输入、换一个优化级别，误报率可能不同。把“6 工具误报 0”写成“工具不会误报”会越过样本边界；写成“本次未出现误报”才是准确的。

单工具误读在 M2 上最清楚。只看 RTLF，会把 M2 读成“两臂都可区分”：主类对 6/6

检出，未修与已修各 3 次；只看 dudect，会因阳性落在已修臂（36.27、30.94、61.91 对 4.68、3.05、3.66）而得出“修复引入泄漏”。两个读法都与端到端 Welch 检验的未检出冲突。[7]

Mona 的不一致原样保留。M1 未修臂全部无分离、已修臂 2/6 次分离，与 Welch、RTLF、dudect 的方向都不一致；它的“置信”字段是批次成功百分比，不是 p 值，不能参与 Holm，也不能用来推翻既有结论。把 Mona 的不一致写成“工具有问题”或“结论有问题”都越界；它只说明这个工具的判据与另外三类不同。

M2 的两个误读来自不同的判据。RTLF 比较分布与分位，主类对使消息位走系统性不同的路径，分布差异可能来自分支而不是除法；dudect 用在线 t 检验，阳性落在已修臂，说明它测到的是修复引入的另一种差异，不是除法。两个工具都没有定位到除法热点；把它们与 Memcheck、varlat 的定位结论交叉，才能得到“定位阳性、计时未检出”的完整读法。

15 个未运行格分布在 5 个工具上:Memcheck 缺 M2 的完整解封装探针,varlat 缺 M1/M3/M4，Binsec 缺 M1/M2/M3/M5，Microwalk 缺 M1/M2/M4/M5，Mona 缺 M3/M4/M5。每一格都写了原因；补上它们需要额外的探针、抽取内核或工具接入。未运行格不是阴性，也不是“无关紧要”；它们只是本轮没有数据。

图 10 证明多工具交叉能消除单工具误读，并把“未运行”与“阴性”分开；它不证明所有工具都同意，也不证明没有假阴性。15 格未运行不等于阴性，未运行格只写原因，不填结论。补上这些格需要额外的工具安装与模型构建，本作品没有做。

交叉矩阵不产生新的统计判定。它把已有工具的结论放在同一张表上，让读者看到“同一对象在不同判据下可以给出不同读法”；它不做投票，也不把多数工具的意见当成结论。主结论仍由预注册 Holm 家族给出，矩阵只负责解释与边界。

3.13 可复现性

可复现不是“代码在仓库”，而是“干净 clone 能跑、包内能复算、每个数字能回到原始行”。一键入口分 quick 与 full 两档：quick 不写 sysfs、不绑核，用固定输入跑功能与统计复算；full 需要锁频控制器与裸金属硬件，按冻结计划重采时间数据。

quick 档的设计目标是“在任何一台能编译的机器上跑通软件流程”。它不写 sysfs，不需要 root，不绑核；它跑功能测试、统计重算与报告生成，不产生新的时间结论。full 档才需要锁频控制器、裸金属与固定的 CPU；它按冻结计划重采，产出原始长表与运行台账。两档分开，是为了让“软件流程可复现”与“时间结论可复现”不混为一谈：前者任何机器都能做，后者需要硬件。

quick 档的输入是固定的，输出是功能测试结果与统计重算结果；它不产生新的时间数据，因此不需要锁频。full 档的输入是冻结计划，输出是原始长表与运行台账；它需要锁频控制器、绑核与恢复核对。两档的入口是同一个脚本，参数不同；把两档分开，是为了让读者知道“跑通”与“复现时间结论”是两件事。

干净 clone 的四个测试集合计 263 项通过、0 失败：platform 88、ordinary 57、sanitize 57、

historical-fixed 61。匿名包解出后 CTest 5/5。随包提供 42 份原始长表、逐次运行台账、冻结计划、产物摘要与统计脚本；每个正文数字可以按附录 B 的运行编号回到原始行，统计判定可以按附录 C 的公式与脚本复算。

42 份原始长表是逐次运行的原始值，不是汇总表；每份长表带运行编号、模块、臂、种子、CPU 与产物摘要。逐次运行台账记录每次控制器运行的进入、采中、退出三段；冻结计划记录主类对、样本量、停止条件与 Holm 家族；统计脚本按附录 C 的公式重算。四类材料合起来，第三方可以从原始值走到正文数字。

263 项测试是软件流程验证，不是计时复现。platform 88 项覆盖平台自身的构建与接口，ordinary 57 项覆盖普通构建，sanitize 57 项覆盖 ASan/UBSan 构建，historical-fixed 61 项覆盖历史修复版；四类合起来说明代码在各构建配置下能跑、功能测试能过，不说明时间结论能在别的机器上复现。263 项证明的是流程，不是时间。

复现边界要写清楚：完整锁频采集需要硬件，包内不能代替；外部 RTLF、Mona、dudect、Binsec 不在包内重跑，只保留输出摘要与调用参数；包内复算只重算已有数据，不产生新实验。这组结果证明软件流程与统计判定可复算，不证明任何机器都能复现时间结论；时间结论依赖本机、本编译产物与锁频工作点。

匿名包解出后 CTest 5/5，是包内复算的入口。它验证包内构建、功能测试与统计脚本能在干净环境里跑通；它不重跑外部工具，也不重采时间数据。5/5 说明包内流程可复算，不覆盖外部工具与时间数据。

4.与现有工作对比及应用前景

4.1 泄漏检测与评估方法对比

软件时间侧信道的评估分为三条路线：统计检测、动态分析与形式化验证。本作品不提出新的检测原语，贡献在于把三条路线组织进同一个以产物为中心、可审计的流程。第 2 章讲这套流程怎么搭，本章讲它与既有路线各自回答什么问题、答案能用到哪一步。

统计检测把程序当黑盒。TVLA[12] 比较固定类与随机类的轨迹分布，dudect[7] 把同一判据搬到在线计时，用最大 t 与上游阈值给二值结论，RTLF[8] 比较分布与分位，并用 bootstrap 控制第一类错误。三者都只回答“两类样本能不能分开”，不回答差异来自哪条指令；类对构造、样本顺序与运行间漂移都会改变结论。第 3.12 节的 M2 是一个现成例子：RTLF 读成两臂都可区分，dudect 的阳性落在已修臂，预注册的均值检验却两个臂都未拒绝。

动态分析在程序内部找秘密依赖。ctgrind 沿用 Valgrind 的秘密标记，报告秘密进入分支条件或访存下标的时刻 [9]; Microwalk 用执行轨迹与互信息找与秘密关联最强的指令 [10]; varlat 检查除法一类可变时延指令的操作数是否含秘密 [4]。这类工具能给出函数级甚至指令级的落点，但只覆盖实际执行过的输入，也不回答“这台机器上能不能测到”。定位阳性与计时阳性是两件事，M2 的三处除法正是两者分离的例子（第 3.4 节、第 3.7 节）。

形式化验证在给定模型与边界内给出另一种结论: 证明控制流与访存不依赖秘密。ct-verif 把常数时间归结为产品程序的安全性，在中间表示层完成这类证明 [13]; Binsec/Rel 用关系符号执行在二进制层做有界检查，并给出反例路径 [11]。它们的成本高，通常不建模指令时延、缓存与流水线，所以能回答“是否依赖秘密”，不能回答“本机计时是否可区分”。第 3.4 节 M4 的不安全判定与 ct 臂的有界安全、M5 的 1088/1088 路径完成，都属于有界结论。

表 12 把三条路线与本作品的定位放在同一组维度上。每一行的最后一列写出这个差异导致的实践后果，而不只是一个勾。

表 12: 泄漏检测与评估方法对比

方法	本机可测性	函数级定位	产物级证明	未检出的量化	多机制/防护复测
TVLA[12]、dudect[7]	是（黑盒计时）	否	否	一般不报，阴性无预算	需另行组织；后果：换产物要整体重判
RTLF[8]	是（第一类错误有界）	否	否	随样本量提高功效，不报 MDE	需另行组织；后果：分布差异无法归因到指令
ctgrind[9]、Microwalk[10]	否（不回答本机可测性）	是（分支、访存、轨迹）	否	不适用（只报依赖）	单一机制类；后果：定位阳性易被读成可利用
varlat[4]	否	是（除法等可变时延指令）	否	不适用	针对单一指令类；后果：只覆盖已知机制，漏掉新形态
ct-verif[13]、Binsec/Rel[11]	否	是（函数或内核）	是（给定边界内；ct-verif 在中间表示层）	不适用（二值结论）	单一函数或内核；后果：不建模指令时延，答不了本机计时
本作品（A-16、A-24、A-26、A-27）	是（预注册 + Holm + MDE）	是（多工具交叉）	部分（M4、M5 有界）	报告 MDE 与样本量	5 模块统一规程、防护前后同规程复测；后果：结论绑定产物、可按层追溯

表注：三条路线回答不同问题：统计检测判“是否可区分”，动态分析判“哪里依赖秘密”，形式化判“给定边界内是否常数时间”。本作品不改变这些判据，只把三类证据登记在同一产物上；“部分”指 M4、M5 的有界验证，不是全实现认证。

五列维度各有用途。本机可测性决定结论能否用于发布判定，函数级定位决定阳性能否落到代码，产物级证明决定结论是否覆盖所有输入，未检出的量化决定阴性有多强，多机制与防护复测决定一次评估能否复用。五列合起来，读者才能判断一条结论该信到哪一步。

本作品把三条路线当作组件来用。同一个二进制既进定位工具，也进计时窗口；定位结果、计时判定与防护复测按证据层分开登记；每个阴性附最小可检测效应，每个未执行单列；再用 90 格排查与 9×5 工具矩阵做交叉（第 3.5 节、第 3.12 节）。这样做的代价是采集与工具

运行的投入增加，收益是每个结论都能回答“在哪一层、对哪个产物、在什么预算下成立”。

这种组织的收益在证据归档，不在灵敏度。定位工具还是那些工具，统计检验还是那套检验；改变的是证据的归档方式：同一次运行的对象、产物摘要、编译条件与判定结果绑在一起，其他人也能按同样的顺序复查。

与单机制研究的分界在目标。KyberSlash 以秘密除法与密钥恢复为目标，clangover 追编译器引入的分支，Azouaoui 等系统量化 Kyber 解密与重加密各步骤的泄漏并指向选择密文攻击 [4, 5, 14]。三者都把一条机制追到攻击终点；本作品在“可区分”处停止，不做密钥恢复，把工作重点放在检测的可审计性与多机制覆盖上。与功耗、电磁研究的关系是互补：它们观测物理量，本作品只观测时间，不覆盖功耗、电磁与故障注入。

定位阳性不等于本机可利用。M2 的三处除法在定位层稳定出现，varlat 报 1280 errors / 3 contexts，端到端与除法热点微基准都未检出，最小可检测效应 24.7–67.1 ns 远大于除法的可能效应。定位阳性只说明机制存在；是否可利用要由测量回答。

锁频的价值是确定性而不是降噪。不锁频与锁频的 CV 比为 0.964、1.006、1.089，方向不一致；锁频提供的是可核对、可恢复的工作点，使“这次采集在什么频率下完成”可被审计。

同一机制在不同优化级别与架构上形态不同。M1 在 -Os 下被编成按位测试后条件跳转，-O2/-O3 的命中位置更多（8、15 个）却计时未检出；aarch64 上位域提取与条件选择取代了分支，机制不复现。分支更多不一定更易检出，结论必须绑定产物与环境。

表 12 证明三条路线各有自己的问题域，也证明本作品的证据组织方式能把这些答案放在同一张表上；它不证明本作品在灵敏度、完备性或成本上优于所有对照路线。在单机、单编译器、时间单通道的条件下，本作品的覆盖面小于专用形式化工具，定位精度也不如把单一工具做到极致的场景。它的差异在于每个阴性带预算、每个未执行单列、每个结论绑定产物，评审者可以沿着证据层往回查。

4.2 应用场景与推广边界

本作品可以作为密码库发布前检查、构建配置回归、防护方案评估与教学评测的本地工具；它不替代独立审计、形式化认证或真实远程评估。四个场景的共同前提是：使用者手里有源码或二进制，能在一台受控机器上重复运行，并愿意记录编译条件与样本量。对 44 名密码库开发者的调查显示，开发者认为时间侧信道的防护并不难实施，落差在工具采用与工程投入 [15]。

后量子密码库发布前检查由库开发者使用。时机是版本冻结、发布候选构建产出之后；输入是待发布的二进制、编译条件与公开向量；输出是定位层的机制清单和固定机器上的计时判定。定位层用反汇编、Memcheck 与 varlat，成本低，可以在每次候选构建上跑；计时层需要锁频与裸金属，按版本或按周在固定机器上跑。

小团队可以只跑定位层，把计时层留给有裸金属的机器；两层的结果分开登记，不互相代替。得到的是“目标机制是否存在、本机是否可区分”，得不到“全库常数时间认证”。第

3.12 节的 9×5 矩阵说明单工具既可能漏掉机制，也可能给出相反读法，发布判定至少要两类工具同向。

编译器与构建配置变更回归由平台维护者使用。时机是升级编译器、改优化级别或换目标架构之前；clangover 表明这些变更会静默改变产物中的分支形态 [5]。本作品按“二进制摘要 + 优化级别”组织判定，先比对新旧产物的反汇编与 Memcheck，再在同一锁频工作点复测计时。得到的是新旧产物的差异，得不到“新版本安全”的结论：新产物可能引入新机制，也可能只是把旧机制挪了位置。面向 CI 的回归门禁尚未实现，列为后续工作。

防护方案的工程评估由实现者与评审者使用。时机是候选补丁进入主线之前；输入是同一份源码的防护前后两版、同一组公开向量与同一锁频规程；输出是定位结果、计时判定与成对开销。M1 的掩码选择使解封装 +1.51%，M2 的除法替代使 −2.92%，前者变慢、后者变快，说明防护代价不能用一个方向概括。得到的是“这个方案在这个产物上的代价与效果”，得不到“所有方案中的最优选择”。

教学与评测把 M3、M4 的人工校准夹具与 M5 阴性对照当作工具测试集。时机是工具版本冻结或课程实验开始之前；教师或工具开发者用已知机制的夹具检查工具能否识别，用阴性对照检查会不会误报；输入是夹具源码与固定输入类，输出是识别、漏报与误报的清单。得到的是“工具能否识别典型机制、会不会误报”，得不到“工具通过了认证”。M5 上运行过的 6 种工具误报 0，分母是 6 个工具 × 1 个对象，只说明本次没有误报。

三类任务不适合本作品。跨机器的绝对耗时比较不在范围内，因为时间证据不跨环境合并；功耗、电磁与故障注入未覆盖，因为本作品只观测时间；真实远程链路未覆盖，因为只有同机网络场景。把这些任务交给本作品，会得到看似相关、实际越界的结论。

推广边界有五条，每条都直接约束上面的场景，也对应一项未覆盖的内容。结论绑定在一台 x86_64 裸金属、clang 18.1.3 与所列优化级别上，不外推到其他 CPU 与编译器。评估在本地白盒下进行，不提供密钥恢复，也不评估功耗、电磁与故障注入。aarch64 只是第二架构，不是第二测量机；网络只是同机模拟，不是真实远程。

真实缺陷采用上游补丁，自主修复只落在 M3、M4 两个人工夹具上；KyberSlash 官方修复提交的残留分支只定位、未修复[2]。题面 12 条按 10 条满足 + 2 条部分登记，缺口与原因见附录 A。

本作品不能替代独立审计、形式化认证或真实远程评估，结果也不构成安全性结论；它能做的是把本地可复现的证据整理成可追溯的记录，让发布者、评审者与教学者知道每一条结论的证据层与适用边界。

[2]该提交号为 272125f，定位与排查见第 3.5 节。

5.结论

5.1 主要结论

本作品在 ML-KEM 解封装上建立了一条“检测—定位—防护—复测”的分层证据链，并在真实历史案例、人工校准与阴性对照上把四段逐一走通。五条判断各自绑定证据层与适用边界；数字回指第 3 章与附录 D。

最直接的一条判断是：数十纳秒量级的真实泄漏可以被检出，也可以在产物层核验修复。M1 未修版本在主类对上的合并均值差为 $+32.311$ ns，95% 区间 $[20.282, 44.339]$，预注册 Holm 家族中 $p = 1.30 \times 10^{-5}$，低于阈值 0.00833；族外的 dudect[7] 与 RTLF[8] 给出同向结论。官方修复后合并差为 -8.150 ns，区间 $[-21.916, 5.616]$，三次独立运行的区间都含 0；解封装开销增加 1.51%。这条判断限定在本机：32.311 ns 只有 2–3 个分辨率单位，依赖每类 10,000 次、三次运行与预注册家族，换产物或换机器要重新判定。

第二条结论：定位层与计时层必须分开登记。M2 的三处秘密除法在定位层稳定出现，可变时延检查报 1,280 errors / 3 contexts，官方修复后归零；同一产物的端到端合并差为 $+4.58$ ns，区间 $[-3.08, 12.25]$，除法热点微基准为 $+0.044$ ns，区间 $[-0.430, 0.517]$，两层都未检出。M2 的检测预算为 25–67 ns，大于除法可能产生的效应；-O2 与 -O3 的产物里秘密分支的命中位置更多，计时同样未检出。定位阳性回答“机制在不在”，计时阳性回答“本机分不分得开”，两者不能互相代替。

修复已知泄漏不等于产物达到常数时间。90 格排查的结果是 5 格存在、75 格不存在、10 格不适用、0 格未检查，覆盖三个实现、五个解封装步骤与六类泄漏来源。KyberSlash[4] 官方修复提交只消除了除法，消息编码函数里与 clangover[5] 同源的分支仍在；未修与已修两侧的指令字节逐字节相同，秘密标记各报 256 errors / 1 context。这是历史产物的机制层观察，未做逐指令时序归因；结论停在机制层，不属于新漏洞。同一份源码的常数时间承诺，只有落到具体产物、具体证据层与具体检测预算上才有内容；修复一条机制，不自动清除同一条路径上的其他机制。

阴性对照没有出现误报。M5 在本机、本编译产物、本类对与每类 10,000 次的预算下，合并差为 -5.30 ns，区间 $[-11.77, 1.16]$ 含 0，预注册家族没有被拒绝；在 M5 上运行过的 6 种工具误报为 0。阴性对照检验的是工具与流程会不会把无泄漏读成泄漏，不是给 M5 发常数时间证书；这条阴性只排除大于该预算的效应，不排除更小的效应，也不覆盖其他产物。分母是 6 个工具与 1 个对象，结论只覆盖本次矩阵。

测量条件决定可检出性。锁频的价值在确定性，不在降噪：不锁频与锁频的变异系数比为 0.964、1.006、1.089，方向不一致，锁频提供的是可核对、可恢复的工作点。M1 的泄漏机制在 aarch64 上不复现，同一份源码被编成位域提取与条件选择；同机网络五档在本预算下全部未检出，分辨同一量级效应所需的样本比实际高 1.3–7.7 个数量级。可检出性由产物、架构与测量条件共同决定，时间证据不跨环境合并。

本作品不提出新检测原语，不主张新漏洞，不做密钥恢复；贡献在产物中心的证据组织与可审计判定。

5.2 局限与未验证项

本作品的结论受测量对象、测量环境、方法覆盖与证据完整度四类限制；这些限制决定结论能用到哪。

第一类是测量对象。M1 是 FIPS 203[1] 初始公开草案（ipd）的实现，M2 是历史 Kyber768[2] 快照，两者都不等同于最终标准；M5 才对应最终标准。M1 的计时数据是同一机器、同一锁频规程下的前期封存数据，不是与 M2–M5 同一批采集。M4 是 32 B 比较夹具，它的前后对照只说明检测链路能走通，不外推到完整解封装。这意味着，本作品的结论按对象分别成立，不能从一个对象推到“ML-KEM 实现”这个整体。

第二类是测量环境。全部时间结论来自一台 x86_64 裸金属与 clang 18.1.3；本机未配置 isolcpus，SMT 开启，后台约 39 个服务在运行。最小步进 10 ns，32.311 ns 的效应只有 2–3 个分辨率单位。网络只有同机 veth 与 netem，没有真实远程链路；aarch64 只构成第二架构，不是第二台测量机，其容器未锁频、非裸金属。三台 WSL2 虚拟化复现只在机制层一致，时间数据没有进入任何 Holm 家族。这意味着，结论不能外推到其他 CPU、其他编译器、隔离更好的机器或真实远程。

第三类是方法覆盖。M2 的主类对不是为除法操作数设计的，message-zero 与 message-ones 只让解密消息的位不同，没有控制除法的被除数；检测预算 25–67 ns 大于除法可能产生的效应。M2 在残留分支存在的情况下，预注册家族仍未拒绝，其原因没有分离。

工具一侧的覆盖同样有限。Binsec[11] 是有界验证，只覆盖给定入口与范围；9×5 工具矩阵有 15 格未运行，Microwalk 只跑过 M3 一个对象；GCC 与 -O1/-Oz/-Og 三个级别未执行。这意味着，阴性的强度受方法覆盖限制；没有跑过的格子是未执行，不是未检出，方法覆盖之外不能推断。

第四类是证据完整度。真实缺陷的修复都采用上游补丁，自主修复只落在 M3、M4 两个人工夹具上；KyberSlash 官方修复版本的残留分支只定位、未修复、未复测。三对修复版本的输出摘要相同，只说明语义不变，不说明产物形态相同。题面要求中第 6、7 条按部分登记，缺口与原因见附录 A。面向 CI 的回归门禁未实现。这意味着，本作品证明的是机制消失与前后可复测；自主防护覆盖真实标准实现仍待补，部分项按部分引用。

四类限制不互相抵消。一条结论同时受几类限制时，按最弱的一类引用；四类分开写，读者才能判断这条结论该信到哪一步。

5.3 后续工作

后续工作按“先补防护、再补功效、再补环境、收尾补工程化”排序；每条给出对应的局限、路径、成功判据与新代价。

第一项是 KyberSlash 官方修复版本的自主修复与复测，补上自主防护没有落到真实标准实现这一缺口。做法是把掩码选择移植到消息编码函数；判据是目标分支从反汇编消失、秘密标记报数由 256 降为 0、KAT 100/100 通过；计时复测视机时；代价是需要重新锁频采集。

第二项针对方法覆盖里主类对的缺口：M2 的输入类与样本量。做法是按除法操作数设计输入类，每类内部变化密文，并按 MDE 扩大样本；判据是能回答“残留分支存在却未检出”的原因；代价是机时增加，预注册家族需要重开。

第三项针对测量环境的量化与隔离缺口。做法是改用 rdtscp、配置 isolcpus、关闭 SMT；判据是单次分辨率与 32 ns 效应的比值改善；代价是需要独占机器。

第四项针对方法覆盖的编译器与级别缺口。做法是补 gcc 13.3.0 与 -O1/-Oz/-Og；判据是每一格都给出产物判定与计时判定，与现有 12 格同口径；代价是每格重新构建与采集。

第五项针对测量环境的远程缺口。做法是从同机 veth 走到真实两端，测出本地与远程的检测预算比；判据是给出可复算的预算比与置信区间；代价是网络环境不可控，需要新的预注册。

第六项针对证据完整度的 CI 缺口。做法是把反汇编、Memcheck 与可变时延检查接入持续集成；判据是每次提交自动跑定位层并给出与基线的差异；代价是维护工具版本与基线，工具升级本身要重新登记。

六项的排序依据是依赖关系。第一项不补，自主防护的缺口仍然存在；第二项不补，M2 的阴性无法解释；环境与工程化放后，是因为它们不改变已有结论的强度，只扩大适用范围。

A.题面要求对照

本附录逐条对照题面的 12 条要求。状态只取“满足 / 部分 / 未执行”三种；第 6、7 条为“部分”，缺口与影响写在同行。第 9 条的必做项为完整实验，可选项分列为优化级别、架构与同机网络三组，按各自覆盖范围表述。

序号	要求	本作品的做法	章节	状态	证据
1	实现或接入符合规范的 ML-KEM 密钥生成、封装与解封装，用公开或主办方向量验证；不得改安全语义换常数时间。	三条实现路径接入统一接口；公开向量、修复等价与隐式拒绝测试；M1 标注为 FIPS 203 初始公开草案（ipd），M2 标注为历史 Kyber768。	2.1、2.4、3.2	满足	A-33 至 A-35
2	建立可重复的高精度计时框架，说明处理器、操作系统、编译器、优化级别、频率管理、测试次数与异常样本处理。	单调时钟计时；锁频控制器 20 次探针与退出恢复核对；28 项环境表；原始长尾全留，失败样本单列。	2.5、2.9、3.1	满足	A-29 至 A-31
3	至少两类有明确区别的解封装输入，重复次数足够；输入设计服务于泄漏分析。	主类对为 message-zero 与 message-ones 等长合法密文；辅以位翻转、随机、格式破损与截断；每类 10000 次、三次运行。	2.1、3.1、3.8	满足	A-01、A-03、A-28
4	用统计方法判断不同类别的执行时间是否显著可区分；均值、方差、分位数、置信区间与检验并用，不得凭少量样本。	Welch 检验、Holm 校正、逆方差固定效应合并、I^2 与 MDE；RTLF、dudect、Mona 作族外交叉。	2.6、3.3、3.7	满足	A-09 至 A-16
5	定位泄漏来源，说明其与解封装流程的对应关系。	反汇编、Memcheck、varlat、Microwalk、Binsec；3 实现 × 5 步骤 × 6 类 = 90 格逐格判定，5 格存在、75 格不存在、10 格不适用、0 格未检查。	2.7、3.4、3.5	满足	A-17 至 A-23
6	对已发现问题采用常数时间比较、掩码选择、固定循环、统一错误处理或消除敏感分支等方式修改；不得通过日志、异常、返回码泄露私钥、中间值或验证结果。	七类手段落在官方补丁与人工校准两臂；1048 个输出与返回调用点逐条审计，122 个接触秘密，秘密内容泄露 0 条。缺口：M2 消息编码分支只定位、未修复；审计不含 vendor 树。	2.8、3.6	部分	A-18、A-23、A-36

表 A1　题面 12 条对照（一）。状态以仓库内可打开的证据文件为判据，不以文稿措辞替代实验完成。

序号	要求	本作品的做法	章节	状态	证据
7	防护后在与防护前相同或等价的测试环境下重采数据，对比不同输入类别的时间分布与统计结果，说明泄露是否明显降低；同时验证功能正确性。	M1 两臂同规程复测；M4 两臂在定位层与计时层都完成前后对照；M3 为 on/off 对照；M2 只在定位层说“机制消失”。缺口：本轮自研防护未落在真实标准实现上，真实实现侧采用上游补丁。	3.6、3.9	部分	A-01、A-02、A-06、A-07、A-35
8	给出防护前后解封装平均时间、典型分位数与性能开销；不得牺牲安全性或正确性。	25 行逐臂逐操作汇总与 11 行成对开销；M1 修复 +1.51%，M2 修复 −2.92%，M4 比较夹具 +26.21% 与 +6.85%。	3.9	满足	A-25
9	至少在一种软件环境完成完整实验；高水平作品可增加优化级别、CPU 架构或本地与网络场景。	12 格优化级别、12 格 aarch64、同机网络 5 档 × 3 臂 × 2 次；aarch64 只复现机制，不构成第二台测量机；网络为同机模拟。	3.10、3.11	满足	A-26 至 A-28
10	明确攻击者可控输入、可观测信息、本地权限与攻击条件；区分本地高精度计时与远程网络时间攻击的边界。	本地白盒模型；只判断两类耗时是否可区分，不做密钥恢复；同机网络只给检测预算，不推断真实远程。	1.2、2.2、5.2	满足	A-28、C-05、C-06
11	提供测试程序、数据生成方法、实验参数、编译选项与统计过程，使评测人员可复现。	一键入口分 quick 与 full 两档；干净 clone 263 项测试通过；vendor 取件两层 SHA-256；逐次运行台账与原始长表随附。	3.13、附录 G	满足	A-37
12	至少在自动化检测、敏感路径定位、常数时间实现、统计方法、低开销防护或跨平台验证之一取得改进，并用可复现实验说明效果。	分层证据与 MDE；预注册锁频；90 格与 9 × 5 工具矩阵；产物与架构依赖对照。	1.4、3.12、4.1、5.1	满足	A-22、A-24、A-26、A-27

表 A1　题面 12 条对照（二）。状态以仓库内可打开的证据文件为判据，不以文稿措辞替代实验完成。10 条满足、2 条部分，缺口见第 6、7 条。

第 6 条与第 7 条判为“部分”，指向同一组事实：本轮自己发现的 M2 消息编码分支只定位、未修复；真实实现侧的防护采用上游官方补丁，自研防护只落在 M3、M4 两条人工校准链上；泄露渠道审计覆盖自有实现与平台驱动，不含实际打补丁的 vendor 树。这个缺口决定两条主张的适用范围：自研防护的完整前后对照落在 M3、M4 两条人工校准链上；真实实现侧，M1 官方修复在定位层与计时层完成前后对照，M2 在定位层机制消失、计时层修复前后均未检出。第 9 条的可选项均已执行，但 aarch64 只构成第二架构，网络只构成同机模拟，二者都不支持跨环境合并时间结论。

状态判定只认仓库内可打开的证据文件：有原始数据、有运行记录、有产物摘要，才写“满足”；证据链缺一段，写“部分”；没有执行，写“未执行”。第 9 条的可选项按覆盖范围计，

不按“做了”计：优化级别有完整 12 格；aarch64 只有机制层 12 格，没有时间分布；网络只有同机模拟 5 档，没有真实远程。按这个口径，12 条落在 10 条满足、2 条部分。按覆盖范围分列后，aarch64 与网络的证据级别与主平台不同：前者只有机制层，后者只有同机模拟。

12 条不是平均用力。第 1 至 5 条是基础层：功能、测量、输入、统计、定位；第 6 至 8 条是防护层：修改、复测、开销；第 9 至 12 条是差异层：环境、威胁模型、复现、改进。基础层每条都有原始数据与产物；防护层第 6、7 条卡在自研防护未落真实实现；差异层第 9 条的可选项已执行但边界不同，第 10 至 12 条有独立证据。这样分组后，两条“部分”不是笼统的保守，而是有具体缺口位置。

证据列用的是台账编号，不是文件路径。A 类编号可在附录 B 至 F 或正文对应节找到原始记录；C 类编号是未执行项，只在边界段引用。编号与章节号并列，是为了让读者既能按章读，也能按台账核。台账编号不替代证据文件，只负责把正文的一句话指回一条可打开的记录；如果编号与文件对不上，以文件为准，并回改正文。

B.实验与运行总表

本附录按运行家族给出逐次记录与失败清单。表 B1 覆盖平台四模块的 54 次锁频控制器运行；表 B2 覆盖 M2 除法热点微基准的 6 次运行；表 B3 覆盖收尾的 117 次锁频运行、同机网络的 30 次运行、aarch64 的 12 格与 M1 封存批次，并把失败尝试单列。控制器的“0/完成/是”表示 controller_exit = 0、reason = worker_finished、restore_match = yes；每次进入的 20 个频率探针均在 ±2% 带内且跨度不超过 50000 kHz。

序	模块	运行	臂	类别	每类样本	日期	控制器	产物 / 异常	
1	M5	s1 31337 CPU2	参考	measure 主类对	10000	10-16	0/完成/是	CSV+meta / —	
2	M5	s1 31337 CPU2	参考	online dudect	10000/批	10-16	0/完成/是	max t 日志 / —	
3	M5	s2 424242 CPU2	参考	measure 主类对	10000	10-16	0/完成/是	CSV+meta / —	
4	M5	s2 424242 CPU2	参考	online dudect	10000/批	10-16	0/完成/是	max t 日志 / —	
5	M5	s3 31337 CPU6	参考	measure 主类对	10000	10-16	0/完成/是	CSV+meta / —	
6	M5	s3 31337 CPU6	参考	online dudect	10000/批	10-16	0/完成/是	max t 日志 / —	
7	M3	s1 31337 CPU2	on	measure 主类对	10000	10-16	0/完成/是	CSV+meta / —	
8	M3	s1 31337 CPU2	off	measure 主类对	10000	10-16	0/完成/是	CSV+meta / —	
9	M3	s2 424242 CPU2	on	measure 主类对	10000	10-16	0/完成/是	CSV+meta / —	
10	M3	s2 424242 CPU2	off	measure 主类对	10000	10-16	0/完成/是	CSV+meta / —	
11	M3	s3 31337 CPU6	on	measure 主类对	10000	10-16	0/完成/是	CSV+meta / —	
12	M3	s3 31337 CPU6	off	measure 主类对	10000	10-16	0/完成/是	CSV+meta / —	
13	M3	s1 31337 CPU2	on	online dudect	10000/批	10-16	0/完成/是	max t 日志 / —	
14	M3	s1 31337 CPU2	off	online dudect	10000/批	10-16	0/完成/是	max t 日志 / —	
15	M3	s2 424242 CPU2	on	online dudect	10000/批	10-16	0/完成/是	max t 日志 / —	
16	M3	s2 424242 CPU2	off	online dudect	10000/批	10-16	0/完成/是	max t 日志 / —	
17	M3	s3 31337 CPU6	on	online dudect	10000/批	10-16	0/完成/是	max t 日志 / —	
18	M3	s3 31337 CPU6	off	online dudect	10000/批	10-16	0/完成/是	max t 日志 / —	

表 B1　平台四模块运行总表（一）。运行编号与正文 3.1、3.3 至 3.7 节一一对应；失败尝试单列于表 B3，不计入正式结果。

序	模块	运行	臂	类别	每类样本	日期	控制器	产物 / 异常	
19	M4	s1 31337 CPU2	vulnerable	measure 主类对	10000	10-16	0/完成/是	CSV+meta / —	
20	M4	s1 31337 CPU2	vulnerable	截断对照	10000	10-16	0/完成/是	CSV+meta / —	
21	M4	s1 31337 CPU2	vulnerable	online dudect	10000/批	10-16	0/完成/是	max t 日志 / —	
22	M4	s1 31337 CPU2	ct	measure 主类对	10000	10-16	0/完成/是	CSV+meta / —	
23	M4	s1 31337 CPU2	ct	截断对照	10000	10-16	0/完成/是	CSV+meta / —	
24	M4	s1 31337 CPU2	ct	online dudect	10000/批	10-16	0/完成/是	max t 日志 / —	
25	M4	s2 424242 CPU2	vulnerable	measure 主类对	10000	10-16	0/完成/是	CSV+meta / —	
26	M4	s2 424242 CPU2	vulnerable	截断对照	10000	10-16	0/完成/是	CSV+meta / —	
27	M4	s2 424242 CPU2	vulnerable	online dudect	10000/批	10-16	0/完成/是	max t 日志 / —	
28	M4	s2 424242 CPU2	ct	measure 主类对	10000	10-16	0/完成/是	CSV+meta / —	
29	M4	s2 424242 CPU2	ct	截断对照	10000	10-16	0/完成/是	CSV+meta / —	
30	M4	s2 424242 CPU2	ct	online dudect	10000/批	10-16	0/完成/是	max t 日志 / —	
31	M4	s3 31337 CPU6	vulnerable	measure 主类对	10000	10-16	0/完成/是	CSV+meta / —	
32	M4	s3 31337 CPU6	vulnerable	截断对照	10000	10-16	0/完成/是	CSV+meta / —	
33	M4	s3 31337 CPU6	vulnerable	online dudect	10000/批	10-16	0/完成/是	max t 日志 / —	
34	M4	s3 31337 CPU6	ct	measure 主类对	10000	10-16	0/完成/是	CSV+meta / —	
35	M4	s3 31337 CPU6	ct	截断对照	10000	10-16	0/完成/是	CSV+meta / —	
36	M4	s3 31337 CPU6	ct	online dudect	10000/批	10-16	0/完成/是	max t 日志 / —	
37	M2	s1 31337 CPU2	未修	measure 主类对	10000	10-16	0/完成/是	CSV+meta / —	
38	M2	s1 31337 CPU2	未修	measure 对照类对	10000	10-16	0/完成/是	CSV+meta / —	
39	M2	s1 31337 CPU2	已修	measure 主类对	10000	10-16	0/完成/是	CSV+meta / —	
40	M2	s1 31337 CPU2	已修	measure 对照类对	10000	10-16	0/完成/是	CSV+meta / —	
41	M2	s1 31337 CPU2	未修	online dudect	10000/批	10-16	0/完成/是	max t 日志 / —	
42	M2	s1 31337 CPU2	已修	online dudect	10000/批	10-16	0/完成/是	max t 日志 / —	
43	M2	s2 424242 CPU2	未修	measure 主类对	10000	10-16	0/完成/是	CSV+meta / —	
44	M2	s2 424242 CPU2	未修	measure 对照类对	10000	10-16	0/完成/是	CSV+meta / —	
45	M2	s2 424242 CPU2	已修	measure 主类对	10000	10-16	0/完成/是	CSV+meta / —	
46	M2	s2 424242 CPU2	已修	measure 对照类对	10000	10-16	0/完成/是	CSV+meta / —	
47	M2	s2 424242 CPU2	未修	online dudect	10000/批	10-16	0/完成/是	max t 日志 / —	
48	M2	s2 424242 CPU2	已修	online dudect	10000/批	10-16	0/完成/是	max t 日志 / —	
49	M2	s3 31337 CPU6	未修	measure 主类对	10000	10-16	0/完成/是	CSV+meta / —	
50	M2	s3 31337 CPU6	未修	measure 对照类对	10000	10-16	0/完成/是	CSV+meta / —	
51	M2	s3 31337 CPU6	已修	measure 主类对	10000	10-16	0/完成/是	CSV+meta / —	
52	M2	s3 31337 CPU6	已修	measure 对照类对	10000	10-16	0/完成/是	CSV+meta / —	
53	M2	s3 31337 CPU6	未修	online dudect	10000/批	10-16	0/完成/是	max t 日志 / —	
54	M2	s3 31337 CPU6	已修	online dudect	10000/批	10-16	0/完成/是	max t 日志 / —	

表 B1　平台四模块运行总表（二）。运行编号与正文 3.1、3.3 至 3.7 节一一对应；失败尝试单列于表 B3，不计入正式结果。

序	运行	臂	种子 / 核	每类样本	控制器	产物 / 异常
1	m2-micro-s1-unfixed	未修	31337 / CPU2	10000	0/完成/是	CSV+meta / —
2	m2-micro-s1-fixed	已修	31337 / CPU2	10000	0/完成/是	CSV+meta / —
3	m2-micro-s2-fixed	已修	424242 / CPU2	10000	0/完成/是	CSV+meta / —
4	m2-micro-s2-unfixed	未修	424242 / CPU2	10000	0/完成/是	CSV+meta / —
5	m2-micro-s3-unfixed	未修	31337 / CPU6	10000	0/完成/是	CSV+meta / —
6	m2-micro-s3-fixed	已修	31337 / CPU6	10000	0/完成/是	CSV+meta / —

表 B2　M2 除法热点微基准（6 次）。窗口只包一次消息编码调用，与完整解封装不可比；6 次运行均通过控制器四判据。

批次	运行构成	次数	日期	样本 / 类	控制器	产物 / 失败尝试
异常输入对照（格式破损）	3 臂 × 3 种子 ×（measure + dudect）	18	10-16	10000	18/18	CSV 与 max t 日志；无失败
性能块	9 臂 × 6 个交错块	54	10-16	decaps 150000; keygen、encaps 30000；M4 120000	54/54	CSV+meta; 18 次无效运行（块号参数错误，退出码 2，无计时值，不计入）
优化级别	12 格 × 3 种子	36	10-16	10000	36/36	CSV+meta；1 次被拒（频率探针跨度 54629 kHz，超过 50000 kHz，无计时值）
锁频对照	3 臂 × 3 种子（锁频臂）	9	10-16	10000	9/9	CSV+meta; 不锁频臂 9 次另列，不计入 117
同机网络	3 臂 × 2 次 × 5 档	30	10-20	本地直调 10000; 网络各档 2000	30/30	客户端 RTT 原始值 + 服务端摘要；5 次频率探针拒绝、2 次客户端缺失，均无计时值
aarch64 格	3 臂 × 4 个优化级别	12	10-21	不采时间	无控制器	Memcheck 日志与反汇编; 无失败
M1 封存批次	2 臂 × 3 次运行	6	10-15/16	10000	6/6	replication.csv; 同一机器与锁频规程，非与 M2 至 M5 同批采集

表 B3　收尾运行、网络、架构与封存批次。117 = 18 + 54 + 36 + 9，均为锁频控制器运行；不锁频 9 次与失败尝试不并入。

同机网络场景的逐格结果见表 B4。五档为本地直调、回环、low、mid、high；网络各档每类 2000 个样本，三次运行中的每一次都单独记录。检测预算按 $n = 2(1.96\,\mathrm{SD}/|\Delta|)^2$ 计算，参照效应取本地直调档两次运行的合并值 14.6 ns，只作量级参照。

臂	次	本地直调	回环	low	mid	high	
M1 未修	r1	+37.1 / 可分辨	+63.1 / 未检出	−801.3 / 未检出	+6515.5 / 未检出	+39569.7 / 未检出	
M1 未修	r2	−7.9 / 未检出	−12.4 / 未检出	+263.0 / 未检出	−1863.5 / 未检出	−51509.7 / 未检出	
M1 已修	r1	+11.9 / 未检出	−52.3 / 未检出	+736.4 / 未检出	−3151.9 / 未检出	+72863.2 / 未检出	
M1 已修	r2	+7.0 / 未检出	+5.9 / 未检出	−942.0 / 未检出	−6885.7 / 未检出	+39109.8 / 未检出	
M5 参考	r1	−11.3 / 未检出	−52.8 / 未检出	−665.1 / 未检出	+2861.4 / 未检出	−30649.4 / 未检出	
M5 参考	r2	+2.7 / 未检出	−57.7 / 未检出	−455.4 / 未检出	+654.6 / 未检出	−61629.6 / 未检出	
分辨 14.6 ns 所需样本 / 类		13098	38417	64636555	1003153317	97586175893	
本轮实际样本 / 类		10000	2000	2000	2000	2000	

表 B4　同机网络五档的均值差（ns）与判定。网络各档在本预算下未检出；这是同机模拟、非真实远程，所需样本比实际高 1.3 至 7.7 个数量级。

臂	-O0	-O2	-O3	-Os	反汇编形态	判定
M1 未修	0 / 0	0 / 0	0 / 0	0 / 0	`sbfx`、`and`、`csel`	四格均无秘密相关分支
M1 已修	0 / 0	0 / 0	0 / 0	0 / 0	掩码选择	四格均无命中
M5 参考	0 / 0	0 / 0	0 / 0	0 / 0	掩码选择 + 值屏障	四格均无命中
x86_64 M1 未修（对照）	0 / 0	512 / 8	512 / 15	512 / 1	`bt`/`jae` 或逐位测试分支	x86_64 三格有命中

表 B5　aarch64 12 格与 x86_64 同格对照（错误数 / 上下文数）。aarch64 只构成第二架构，不是第二台测量机；时间证据不跨架构合并。

表 B1 至表 B5 的共同口径是：一条运行记录只对应一个模块、一个臂、一个种子与一个控制器终态；同一模块的 measure 与 dudect 分开记，不合并成一次。54 次平台家族、6 次微基准与 117 次收尾运行合计 177 次控制器运行；M1 的 6 次主类对属于封存批次，不并入 177。失败尝试单列，不删、不改名、不重算；它们没有计时值，因此既不构成阳性也不构成阴性。表 B4 的网络与表 B5 的 aarch64 不进任何 Holm 家族，时间证据不与主平台合并。

样本预算按主类对设计：每类 10000 次、三次独立运行，换种子与换物理核；微基准只包一个热点函数，与完整解封装不可比；网络档每类 2000 次，检测预算按同一式子反推所需样本。预算不同的表不能横向比较 Δ 的大小，只能各自回答“在本预算下是否可分辨”。这也是表 B3 把 54、6、117 分开列、不合成一个总数的原因。

日期与批次也要一起读。平台四模块、微基准、性能、优化与锁频对照集中在 10 月 16 日；异常输入对照同日；同机网络在 10 月 20 日；aarch64 在 10 月 21 日；M1 的主类对是 10 月 15 日至 16 日的封存批次。同一台机器、同一锁频规程，不等于同一批采集；跨日数据只做同口径对照，不把日期差当成处理效应。表 B1 的 54 次运行全部有控制器终态与原始 CSV，表 B3 的 117 次运行另有逐份 terminal 记录；两类记录都保留失败尝试，避免“只留成功”造成的工作量虚高。

失败尝试单列还有一层作用：它们能区分“设备没跑起来”与“跑了但没有差异”。控制器拒绝、参数错误、客户端缺失都属于前者，没有计时值；它们既不支持阳性，也不支持阴性。把它们写成“失败”而不是删掉，读者才能判断 54、6、117 这些数字是不是从全部尝试里挑出来的。本附录的所有计数都以“成功且四判据通过”为分子，以“成功 + 失败尝试”为分母；分母写在表里，分子写在表注里。

C.统计公式与复算说明

本附录给出统计层用到的七个公式、本次参数与手算例。公式与统计电池、固定效应合并工具的实现一致；dudect 与 RTLF 的工具行为按上游实现 [7, 8]。手算例只用合成数核对公式，不替代正式数据。正式结果见附录 D 与正文 3.3 至 3.7 节。

（1）Welch 检验与自由度。

$$t = \frac{\Delta}{\sqrt{s_0^2/n_0 + s_1^2/n_1}}, \qquad \mathrm{df} = \frac{\left(s_0^2/n_0 + s_1^2/n_1\right)^2}{\dfrac{(s_0^2/n_0)^2}{n_0 - 1} + \dfrac{(s_1^2/n_1)^2}{n_1 - 1}}. \tag{C1}$$

其中 $\Delta = \bar{x}_0 - \bar{x}_1$，$s_0^2$、$s_1^2$ 为样本方差，$n_0 = n_1 = 10000$。本次不称其为有限样本精确 t 区间；p 用 $\mathrm{erfc}(|t|/\sqrt{2})$ 的正态近似。

（2）95% 置信区间。

$$[\Delta - z_{0.975}\,\mathrm{SE},\ \Delta + z_{0.975}\,\mathrm{SE}], \qquad \mathrm{SE} = \sqrt{s_0^2/n_0 + s_1^2/n_1}, \quad z_{0.975} = 1.9599639845. \tag{C2}$$

区间含 0 记“未检出”；区间不含 0 记“可区分”。未检出必须同时给出 MDE。

（3）Holm 逐步校正。

$$p_{(1)} \le p_{(2)} \le \cdots \le p_{(m)}, \qquad \text{第}i\text{项阈值} = \frac{\alpha}{m - i + 1}. \tag{C3}$$

从最小 p 开始逐项比较；第一个不满足即停止，其后各项均不拒绝。全族 $\alpha = 0.05$。M1、M2、M3、M4 的 $m = 6$，M5 的 $m = 3$。

（4）逆方差固定效应合并。

$$w_i = \frac{1}{\mathrm{SE}_i^2}, \qquad \Delta_{\text{pool}} = \frac{\sum_i w_i \Delta_i}{\sum_i w_i}, \qquad \mathrm{SE}_{\text{pool}} = \frac{1}{\sqrt{\sum_i w_i}}. \tag{C4}$$

每次运行的 SE_i 由该次 95% 区间反推：$\mathrm{SE}_i = (\text{上界} - \text{下界})/(2z_{0.975})$。

（5）异质性 I^2。

$$Q = \sum_i w_i \left(\Delta_i - \Delta_{\text{pool}}\right)^2, \qquad I^2 = \max\left(0, \frac{Q - (k-1)}{Q}\right). \tag{C5}$$

k 为成员数。I^2 高时，合并区间只作描述，判定仍以预注册的 Holm 结果为准。

（6）最小可检测效应。

$$\mathrm{MDE} = \left(z_{1-\alpha/2} + z_{1-\beta}\right)\sqrt{\frac{s_0^2}{n_0} + \frac{s_1^2}{n_1}}, \tag{C6}$$

取 $\alpha = 0.05$、$1-\beta = 0.80$，$z_{0.975} = 1.959963985$、$z_{0.80} = 0.841621234$。Holm 口径把 α 换成 $\alpha_{\mathrm{eff}} = \alpha/m$；M1、M2、M3、M4 的 $z_{1-\alpha_{\mathrm{eff}}/2} = 2.638257273$，M5 为 2.393979800。MDE 是事后功效描述，不改变已提交的判定。

（7）RTLF 秩分位数 bootstrap。 RTLF 对两类样本的分位数差做 bootstrap 重采样，$\alpha = 0.01$，固定种子 2026101605，重采样 10000 次，输出二值结论：退出码 10 为未检出、11 为检出。它不产生 p 值，不进入 Holm 家族，也不与均值检验互相替代。

公式	符号	本次取值	用途
C1	t、df、s^2、n	$n_0 = n_1 = 10000$；$\alpha = 0.05$	单次运行的均值差检验与自由度
C2	SE、$z_{0.975}$	$z_{0.975} = 1.9599639845$	每次运行的 95% 区间
C3	$p_{(i)}$、m、α	$m = 6$（M5 为 3）；$\alpha = 0.05$	家族内多重比较
C4	w_i、Δ_{pool}	三次运行按区间反推权重	跨运行合并
C5	Q、k	$k = 6$（M5 为 3）	异质性描述
C6	MDE、$z_{1-\beta}$	$z_{0.80} = 0.841621234$；Holm 口径 $\alpha_{\mathrm{eff}} = \alpha/m$	量化未检出
RTLF	分位数、bootstrap	$\alpha = 0.01$；种子 2026101605；10000 次	族外分布交叉

表 C1　统计公式与参数。公式与统计电池、固定效应合并工具的实现一致；手算例只用于核对，不替代正式数据。

手算例一（Welch 与区间）。 取两组各 3 个数：$x_0 = \{10, 12, 11\}$，$x_1 = \{7, 9, 8\}$。均值 $\bar{x}_0 = 11$、$\bar{x}_1 = 8$，$\Delta = 3$；样本方差 $s_0^2 = s_1^2 = 1$。$\mathrm{SE} = \sqrt{1/3 + 1/3} = 0.8165$，$t = 3.674$，$\mathrm{df} = 4$，双侧 $p \approx 0.021$；95% 区间为 $3 \pm 1.96 \times 0.8165 = [1.40, 4.60]$，不含 0。这个例子只说明式 C1、C2 的算法，不构成任何实验结论。

手算例二（Holm）。 取 $m = 4$ 与 $p = \{0.001, 0.020, 0.030, 0.040\}$。阈值依次为 0.0125、0.0167、0.025、0.05。第一项 $0.001 < 0.0125$，拒绝；第二项 $0.020 > 0.0167$，停止；其后两项均不拒绝。Holm 的停止规则使“前面不拒绝”自动覆盖后面，不需要逐项再比。

手算例三（合并与 I^2）。 三次运行的 $\Delta = \{10, 14, 8\}$，$\mathrm{SE} = \{2, 2, 4\}$。

权重 $w = \{0.25, 0.25, 0.0625\}$，$\sum w = 0.5625$。

$\Delta_{\mathrm{pool}} = (2.5 + 3.5 + 0.5)/0.5625 = 11.556$；$\mathrm{SE}_{\mathrm{pool}} = 1/\sqrt{0.5625} = 1.333$；95% 区间 $11.556 \pm 2.613 = [8.94, 14.17]$。$Q = 0.25(1.556)^2 + 0.25(2.444)^2 + 0.0625(3.556)^2 = 2.89$，$k - 1 = 2$，$I^2 = (2.89 - 2)/2.89 = 30.8\%$。

手算例四（MDE）。 取 $s_0 = 700$ ns、$s_1 = 600$ ns、$n_0 = n_1 = 10000$、$\alpha = 0.05$、功效 0.80。$\mathrm{MDE} = 2.801585 \times \sqrt{700^2/10000 + 600^2/10000} = 2.801585 \times 9.2195 = 25.83$ ns。若改用 $m = 6$ 的 Holm 口径，$z_{1-\alpha_{\mathrm{eff}}/2} = 2.638257$，MDE 变为 $(2.638257 + 0.841621) \times 9.2195 = 32.08$ ns。

口径说明。 M1 族在采集后发生过一次成员调整：dudect 的多批最大 t 不是有效 p 值，因此从参数族移出，原始记录另存。两种口径下的家族级判定一致，正文与附录 D 的判定均未改变。这里不展开成员数，避免把一次口径调整写成新的实验事实。

公式选择的边界有三条。第一，区间与 p 值用大样本正态近似，不称有限样本精确 t 区间；n=10000 时两者接近，但口径必须写明。第二，MDE 是事后功效描述，用本次实测标准差计算，不改变预注册的 Holm 判定；它回答“这套样本能看见多大的效应”，不回答“效应是否为零”。第三，合并区间在 I^2 高时只作描述；M4 的家族合并混合了泄漏臂与不泄漏臂，不能把家族合并值当作单一机制的效应。RTLF 只给二值结论，不转成 p 值，也不与均值检验合并。

Holm 与 MDE 是两件事：Holm 控制家族错误率，MDE 描述可检测性。一个阴性结论要同时给两者，读者才能判断“没拒绝”是因为效应小，还是因为样本不够。手算例只用来核对公式，不替代正式数据；正式数据在附录 D，逐次值与家族判定并列。

D.模块级完整结果

本附录给出 M1 至 M5 的逐次运行结果。Δ 为 class0 减 class1 的均值差；95% 区间为正态近似；p 为双侧原始值；Holm 列为该成员在预注册家族中的判定；dudect 与 RTLF 为族外交叉，不进入 Holm。M1 的逐次 p 由该次 Δ 与区间反推 SE 后按式 C1、C2 复算，属 B 类；其余 p 取自成员表原件。M4 的 $p = 0$ 是双精度下溢，不写“精确为零”。

臂	运行	Δ / ns	95% CI / ns	p	Holm	dudect max $\lvert t\rvert$	RTLF	MDE / ns	
未修	s1 31337 CPU2	+39.337	[21.658, 57.017]	1.30×10^{-5}	族内拒绝	26.41	检出	25.27 / 31.39	
未修	s2 424242 CPU2	+31.117	[6.549, 55.684]	0.0131	族内拒绝	40.64	检出	25.27 / 31.39	
未修	s3 31337 CPU6	+22.337	[0.282, 44.392]	0.0471	族内拒绝	14.92	检出	25.27 / 31.39	
已修	s1 31337 CPU2	−13.209	[−33.238, 6.820]	0.1962	不拒绝	1.15	未检出	28.63 / 35.56	
已修	s2 424242 CPU2	+8.354	[−52.988, 69.696]	0.7895	不拒绝	1.12	未检出	28.63 / 35.56	
已修	s3 31337 CPU6	−4.885	[−24.811, 15.042]	0.6309	不拒绝	1.31	未检出	28.63 / 35.56	
合并（未修）		+32.311	[20.282, 44.339]	1.30×10^{-5}	拒绝	—	—	25.27 / 31.39	
合并（已修）		−8.150	[−21.916, 5.616]	0.196	不拒绝	—	—	28.63 / 35.56	

表 D1　M1 逐次结果。逐次值取自封存原始数据；“未检出”附 MDE。M1 的家族主检验 $p = 1.30 \times 10^{-5}$、阈值 0.00833；已修主检验 $p = 0.196$、阈值 0.05（末位成员）后不拒绝。

臂	运行	Δ / ns	95% CI / ns	p	Holm	dudect max $\lvert t\rvert$	RTLF	MDE / ns	
未修	r1 31337 CPU2	+10.530	[−6.396, 27.455]	0.223	不拒绝	4.68	检出	19.9 至 54.0 / 24.7 至 67.1	
未修	r2 424242 CPU2	+19.126	[0.028, 38.224]	0.0497	不拒绝	3.05	检出	19.9 至 54.0 / 24.7 至 67.1	
未修	r3 31337 CPU6	+9.983	[−3.915, 23.881]	0.159	不拒绝	3.66	检出	19.9 至 54.0 / 24.7 至 67.1	
已修	r1 31337 CPU2	+21.657	[−16.123, 59.436]	0.261	不拒绝	36.27	检出	19.9 至 54.0 / 24.7 至 67.1	
已修	r2 424242 CPU2	−17.080	[−45.991, 11.830]	0.247	不拒绝	30.94	检出	19.9 至 54.0 / 24.7 至 67.1	
已修	r3 31337 CPU6	−15.512	[−31.925, 0.900]	0.0640	不拒绝	61.91	检出	19.9 至 54.0 / 24.7 至 67.1	
合并		+4.58	[−3.08, 12.25]	—	拒绝数 0	—	—	24.7 至 67.1	

表 D2　M2 逐次结果。逐次值由封存原始数据重列；“未检出”附 MDE。dudect 阳性落在已修臂、RTLF 两臂均检出，均不可归因于除法变体，只作族外登记。

臂	运行	Δ / ns	95% CI / ns	p	Holm	dudect max $\lvert t\rvert$	RTLF	MDE / ns	
on	r1 31337 CPU2	-10.723	$[-29.703, 8.258]$	0.268	不拒绝	1.93	未检出	14.2 至 27.1 / 17.7 至 33.7	
on	r2 424242 CPU2	-5.763	$[-17.016, 5.489]$	0.315	不拒绝	3.41	未检出	14.2 至 27.1 / 17.7 至 33.7	
on	r3 31337 CPU6	+7.126	$[-3.497, 17.749]$	0.189	不拒绝	2.81	未检出	14.2 至 27.1 / 17.7 至 33.7	
off	r1 31337 CPU2	-5.464	$[-20.612, 9.683]$	0.480	不拒绝	2.24	未检出	14.2 至 27.1 / 17.7 至 33.7	
off	r2 424242 CPU2	+4.800	$[-5.816, 15.417]$	0.375	不拒绝	3.37	未检出	14.2 至 27.1 / 17.7 至 33.7	
off	r3 31337 CPU6	-3.775	$[-13.732, 6.183]$	0.457	不拒绝	3.03	未检出	14.2 至 27.1 / 17.7 至 33.7	
合并		-0.73	$[-5.56, 4.10]$	—	拒绝数 0	—	—	17.7 至 33.7	

表 D3　M3 逐次结果。逐次值来自封存记录；“未检出”附 MDE。M3 为人工构造校准，不是上游缺陷。

臂	运行	Δ / ns	95% CI / ns	p	Holm	dudect max $\lvert t\rvert$	RTLF	MDE / ns	
vulnerable	r1 31337 CPU2	-4.102	$[-4.232, -3.973]$	下溢 0	拒绝	356.6	检出	0.19 至 1.19 / 1.47	
vulnerable	r2 424242 CPU2	-4.351	$[-5.181, -3.521]$	9.23×10^{-25}	拒绝	396.9	检出	0.19 至 1.19 / 1.47	
vulnerable	r3 31337 CPU6	-3.515	$[-4.226, -2.805]$	3.14×10^{-22}	拒绝	395.6	检出	0.19 至 1.19 / 1.47	
ct	r1 31337 CPU2	+0.415	$[-0.523, 1.353]$	0.386	不拒绝	2.06	未检出	0.19 至 1.19 / 1.47	
ct	r2 424242 CPU2	-0.227	$[-0.800, 0.346]$	0.438	不拒绝	1.90	未检出	0.19 至 1.19 / 1.47	
ct	r3 31337 CPU6	+0.347	$[-0.460, 1.155]$	0.399	不拒绝	2.27	未检出	0.19 至 1.19 / 1.47	
合并（vulnerable）		-4.089	$[-4.215, -3.963]$	—	拒绝 3 项	—	—	1.47（最坏）	
合并（ct）		+0.055	$[-0.363, 0.473]$	—	不拒绝	—	—	1.47（最坏）	

表 D4　M4 分臂逐次结果。逐次值由原始成员表复列；“未检出”附 MDE。$p = 0$ 是双精度下溢，不写“精确为零”；M4 是 32 字节比较夹具，结论不外推到完整解封装。

臂	运行	Δ / ns	95% CI / ns	p	Holm	dudect max $\lvert t\rvert$	RTLF	MDE / ns	
参考	r1 31337 CPU2	-5.596	$[-18.912, 7.720]$	0.410	不拒绝	2.10	未检出	11.7 至 24.5 / 13.5 至 28.3	
参考	r2 424242 CPU2	-6.946	$[-24.103, 10.211]$	0.427	不拒绝	1.98	未检出	11.7 至 24.5 / 13.5 至 28.3	
参考	r3 31337 CPU6	-4.819	$[-13.019, 3.382]$	0.249	不拒绝	2.82	未检出	11.7 至 24.5 / 13.5 至 28.3	
合并		-5.30	$[-11.77, 1.16]$	—	拒绝数 0	—	—	13.5 至 28.3	

表 D5　M5 逐次结果。逐次值取自封存记录；“未检出”附 MDE。M5 在本机、本产物、本类对与本预算下未检出，不证明常数时间。

逐次表的读法与正文不同：正文给分层结论，附录给每次运行的原始量。Δ 的符号由 class0 减 class1 决定，同一模块内方向不一致时，不取绝对值平均；先看每次区间是否含 0，再看 Holm 家族是否拒绝。M2 的未修臂描述性合并为正、已修臂为负，这是臂间漂移，不是泄漏降低；M4 的家族合并被两臂混合拉宽，按臂分开后 vulnerable 与 ct 的区间才可读。M1 的逐次 p 是 B 类复算值，臂合并 p 才是族内判定；两者并列，不互相替代。

五个模块的读法各不相同。M1 是唯一“定位阳性、计时阳性、官方修复后计时阴性”的完整链；M2 是“定位阳性、端到端与微基准均未检出”；M3 是人工校准，定位阳性、端到端未检出；M4 是唯一定位层与计时层都阳性、且 ct 臂全阴的正对照；M5 是阴性对照，六种工具在本机未报阳性。把五个模块压成一句“都测了”会丢掉分层信息，因此每张表后都给一句“这张表证明什么、不证明什么”。

MDE 也要逐模块读。M1 未修的检测预算约 25 至 31 ns，已修约 29 至 36 ns；M2 为 25 至 67 ns；M3 为 18 至 34 ns；M4 最坏 1.47 ns；M5 为 14 至 28 ns。观测效应小于本模块 MDE 时写“未检出”，不写“零效应”。M4 的 MDE 小，是因为比较窗口只有几十纳秒、方差也小；这不表示 M4 的结论可以外推到完整解封装。反过来，M2 的 MDE 大，除法热点即使有几十纳秒的效应也可能落在预算之外，所以定位阳性与计时阴性可以同时成立。

逐次 p 的来源分两类：M2 至 M5 直接取自成员表原件；M1 的逐次 p 由该次 Δ 与区间反推 SE 后按正态近似复算。复算值只用于附录对照，族内判定仍用臂合并 p。把两类值并列，是为了让读者看到“逐次方向一致”与“家族拒绝”不是同一个层次；前者描述三次运行，后者描述预注册检验。

E.定位工具输出摘要

本附录只摘录各工具的关键输出行，不贴完整日志。表 E1 覆盖统计与动态标记工具，表 E2 覆盖形式化、访存与反汇编工具；“未运行”按原因登记，不填阴性。工具版本写在各行括号内；同一工具在不同模块上的结论不可互相替代。

工具（版本）	对象	关键输出	上下文 / 样本	位置 / 地址	结论
dudect（上游头文件快照）	M1	未修 max $\|t\|$ 26.41 / 40.64 / 14.92；已修 1.15 / 1.12 / 1.31	3 次运行，21 批	完整解封装	检出（未修）
dudect	M2	未修 4.68 / 3.05 / 3.66；已修 36.27 / 30.94 / 61.91	3 次运行	完整解封装	未检出；阳性落在已修臂，不可归因
dudect	M3	on 1.93 / 3.41 / 2.81；off 2.24 / 3.37 / 3.03	3 次运行	完整解封装	未检出
dudect	M4	vulnerable 356.6 / 396.9 / 395.6；ct 2.06 / 1.90 / 2.27	3 次运行	32 B 比较调用	检出（vulnerable）
dudect	M5	2.10 / 1.98 / 2.82	3 次运行	完整解封装	未检出
RTLF（R 4.3.3，$\alpha = 0.01$）	M1	未修 2/2 exit 11；已修 2/2 exit 10	2 次运行	完整解封装	检出（未修）
RTLF	M2	主类对 6/6 exit 11（两臂各 3）；对照 6/6 exit 10	6 个数据集	完整解封装	检出；两臂均检出，不可归因
RTLF	M3	主类对与对照共 12 个数据集全部 exit 10	12 个数据集	完整解封装	未检出
RTLF	M4	vulnerable 6/6 exit 11（含截断）；ct 6/6 exit 10	12 个数据集	32 B 比较调用	检出（vulnerable）
RTLF	M5	主类对与对照共 6/6 exit 10	6 个数据集	完整解封装	未检出
Welch + Holm（统计电池）	M1	主检验 $p = 1.30 \times 10^{-5}$，阈值 0.00833；已修 $p = 0.196$	每类 10000	完整解封装	检出（未修）
Welch + Holm	M2	合并 $\Delta = +4.58$ ns，CI $[-3.08, 12.25]$；拒绝 0	6 项	完整解封装	未检出
Welch + Holm	M3	合并 $\Delta = -0.73$ ns，CI $[-5.56, 4.10]$；拒绝 0	6 项	完整解封装	未检出
Welch + Holm	M4	vulnerable 3 项拒绝；ct 0 项	6 项	32 B 比较调用	检出（vulnerable）
Welch + Holm	M5	合并 $\Delta = -5.30$ ns，CI $[-11.77, 1.16]$；拒绝 0	3 项	完整解封装	未检出
Memcheck（Valgrind 3.22.0）	M1	未修 1 error / 1 context；已修 0 / 0	完整解封装	消息编码函数第 179 行	检出（未修）
Memcheck	M2	未运行（完整解封装未跑；仅关 varlat 对照 0/0）	—	—	未运行，不填阴性
Memcheck	M3	on 1280 / 2；off 0 / 0	10 次完整解封装	消息编码函数第 714 行	检出（人工校准）
Memcheck	M4	32 / 1；ct 0 / 0	完整调用	比较内核第 9 行	检出（人工校准）
Memcheck	M5	三模式各 10 次，0 / 0	30 次完整解封装	完整调用链	未检出
varlat（Valgrind 3.25.1）	M1	未运行（机制不在命中类别）	—	—	未运行
varlat	M2	未修 1280 / 3；已修 0 / 0	3 个热点	消息解码、压缩、向量压缩	检出（未修）
varlat	M3	未运行	—	—	未运行
varlat	M4	未运行	—	—	未运行
varlat	M5	varlat=yes 0 / 0	10 次完整解封装	完整调用链	未检出

表 E1　统计与动态标记工具输出摘要。工具输出只证明给定输入下的行为，不证明常数时间；“未运行”不等于“未检出”。

工具（版本）	对象	关键输出	上下文 / 样本	位置 / 地址	结论
Binsec（0.11.3 + Z3 4.15.3）	M1	未运行（无对应抽取内核）	—	—	未运行
Binsec	M2	未运行（div 类来源不在已开检查项）	—	—	未运行
Binsec	M3	未运行	—	—	未运行
Binsec	M4	vulnerable `insecure`，反例 `0x8049047`；ct `bounded secure`	32/32 与 16/16 路径	比较内核第 9 行	检出（人工校准）
Binsec	M5	重加密比较 1088/1088、消息编码 1/1 路径完成，无秘密相关控制流	路径完整	重加密比较与消息编码	未检出（有界）
Microwalk（源码自建 + Pin 3.28）	M1	未运行（官方镜像不可用，Pin 缺失）	—	—	未运行
Microwalk	M2	未运行	—	—	未运行
Microwalk	M3	互信息 1.922 bit（全场最高）；off 0.722 bit	5 个公开用例	两处秘密下标 load	检出（人工校准）
Microwalk	M4	未运行	—	—	未运行
Microwalk	M5	未运行	—	—	未运行
Mona（上游脚本）	M1	未修臂无分离；已修臂 2/6 出现箱体分离	6 份输入	完整解封装	未检出；方向不符
Mona	M2	未修臂无分离；s3 已修臂 1/8 出现分离	6 批	完整解封装	未检出；方向不符
Mona	M3	未运行（未接入）	—	—	未运行
Mona	M4	未运行（未接入）	—	—	未运行
Mona	M5	未运行（未接入）	—	—	未运行
反汇编（发行版 objdump）	M1	`bt+jae` → `bt+setb+call cmov_int16`	两版产物	消息编码函数	检出（未修）
反汇编	M2	三热点各 1 条 `div` → 0；消息编码 `bt/jae` 两臂逐字节相同	两版产物	消息解码、压缩、向量压缩	检出；残留分支
反汇编	M3	两条 `movzwl` 秘密下标 load	on/off 两版	静态偏移 `0x5CED/0x5D15`	检出（人工校准）
反汇编	M4	3 条条件跳转中 1 条数据相关	vulnerable/ct	比较内核	检出（人工校准）
反汇编	M5	无秘密相关分支；消息编码唯一跳转是循环回边；sk 检查为公开数据	完整产物	完整调用链	未发现

表 E2　形式化、访存与反汇编工具输出摘要。工具输出只证明给定输入下的行为，不证明常数时间；“未运行”不等于“未检出”。

45 格中 30 格有真实产物，15 格未运行。未运行的原因分三类：工具没有接入该模块、工具缺少运行依赖、该模块没有对应的抽取内核。三类都写“未运行”，不写“未检出”。单工具读法会出错：只看 RTLF，M2 会被读成两臂都可区分；只看 dudect，M2 的阳性落在已修臂，会被读成修复引入泄漏；只看 Welch+Holm，M2 与 M3 的定位层阳性完全看不到。交叉之后，M2 的分层结论才是“定位阳性、计时未检出”。

工具输出只证明给定输入下的行为。Memcheck 的 0 命中不覆盖算术泄漏与缓存泄漏；Binsec 的 bounded secure 只覆盖给定入口、长度与秘密模型；Microwalk 的互信息只覆盖实际执行的用例；dudect 与 RTLF 只回答分布是否可区分。把这些边界写进表，是为了让“未运行”与“未检出”在版面上就分开，而不是靠读者猜。

F.90 格排查全表

本附录给出 3 实现 × 5 步骤 × 6 类 = 90 格的逐格判定。来源列的数字含义：1 提前返回、2 可变时间比较、3 数据相关循环、4 数据相关访存、5 失败路径、6 异常处理。判定只取“存

在 / 不存在 / 不适用 / 未检查”。全表合计：存在 5、不存在 75、不适用 10、未检查 0。人工夹具 M3、M4 不计入本表；“存在”格中的 M5 一格携带公开信息，按有界例外登记。

实现	步骤	来源	判定	函数	指令或行号	证据	
M1	解密	1 提前返回	不存在	indcpa_dec / poly_tomsg	常数回边	A-17	
M1	解密	2 可变时间比较	不存在	poly_tomsg	乘加、移位，无 `div`	A-17	
M1	解密	3 数据相关循环	不存在	poly_tomsg	32 × 8 常数上界	A-17	
M1	解密	4 数据相关访存	不存在	poly_tomsg	固定顺序写	A-17	
M1	解密	5 失败路径	不存在	crypto_kem_dec	本步不判定失败	A-17	
M1	解密	6 异常处理	不适用	—	无异常机制	A-17	
M1	哈希	1 提前返回	不存在	hash_g / hash_h / rkprf	定长调用	A-17	
M1	哈希	2 可变时间比较	不存在	FIPS202	无比较	A-17	
M1	哈希	3 数据相关循环	不存在	FIPS202	24 轮固定	A-17	
M1	哈希	4 数据相关访存	不存在	FIPS202	常量下标	A-17	
M1	哈希	5 失败路径	不存在	FIPS202	不返回状态	A-17	
M1	哈希	6 异常处理	不存在	FIPS202	无 abort/断言	A-17	
M1	重加密	1 提前返回	存在	poly_frommsg	`0x16e bt` → `0x172 jae`	A-17、A-22	
M1	重加密	2 可变时间比较	不存在	poly_frommsg / 压缩	无比较动作	A-17	
M1	重加密	3 数据相关循环	不存在	poly_frommsg	32 × 8 常数	A-17	
M1	重加密	4 数据相关访存	不存在	poly_frommsg	固定顺序写	A-17	
M1	重加密	5 失败路径	不存在	indcpa_enc	不返回错误	A-17	
M1	重加密	6 异常处理	不存在	indcpa_enc	无异常机制	A-17	
M1	比较	1 提前返回	不存在	verify	遍历 1088 B	A-17	
M1	比较	2 可变时间比较	不存在	verify	累积异或，不短路	A-17	
M1	比较	3 数据相关循环	不存在	verify	长度常数 1088	A-17	
M1	比较	4 数据相关访存	不存在	verify	顺序下标	A-17	
M1	比较	5 失败路径	不存在	verify	结果只作掩码	A-17	
M1	比较	6 异常处理	不适用	—	不可失败	A-17	
M1	隐式拒绝	1 提前返回	不存在	dec / cmov	无条件走完	A-17	
M1	隐式拒绝	2 可变时间比较	不存在	cmov	`0x2b26 sete` → `0x2b37 call`	A-17	
M1	隐式拒绝	3 数据相关循环	不存在	rkprf / cmov	定长 32 B	A-17	
M1	隐式拒绝	4 数据相关访存	不存在	cmov	固定偏移、顺序下标	A-17	
M1	隐式拒绝	5 失败路径	不存在	dec	单点返回，零跳转	A-17	
M1	隐式拒绝	6 异常处理	不适用	—	无异常机制	A-17	
M1 小计：存在 1、不存在 26、不适用 3、未检查 0（30 格）							

表 F1　M1 的 30 格排查。“不存在”仅指给定产物、输入与工具下未发现，不证明常数时间。

实现	步骤	来源	判定	函数	指令或行号	证据	
M2	解密	1 提前返回	不存在	dec / indcpa_dec	常数回边	A-18	
M2	解密	2 可变时间比较	存在	poly_tomsg	`0x36bf div %r9d`	A-18、A-22	
M2	解密	3 数据相关循环	不存在	poly_tomsg	32 × 8 常数	A-18	
M2	解密	4 数据相关访存	不存在	poly_tomsg	顺序读写	A-18	
M2	解密	5 失败路径	不存在	crypto_kem_dec	本步不判定失败	A-18	
M2	解密	6 异常处理	不适用	—	RNG 不在本步	A-18	
M2	哈希	1 提前返回	不存在	sha3 / shake	定长调用	A-18	
M2	哈希	2 可变时间比较	不存在	FIPS202	无比较	A-18	
M2	哈希	3 数据相关循环	不存在	FIPS202	24 轮固定	A-18	
M2	哈希	4 数据相关访存	不存在	FIPS202	常量下标	A-18	
M2	哈希	5 失败路径	不存在	FIPS202	不返回状态	A-18	
M2	哈希	6 异常处理	不存在	FIPS202	无 abort/断言	A-18	
M2	重加密	1 提前返回	存在	poly_frommsg	`0x3668 bt` → `0x366c jae`	A-23、A-22	
M2	重加密	2 可变时间比较	存在	poly_compress / polyvec_compress	`0x3523 div`、`0x38c7 div`	A-18、A-22	
M2	重加密	3 数据相关循环	不存在	压缩函数	常数上界	A-18	
M2	重加密	4 数据相关访存	不存在	压缩函数	定长位移拼装	A-18	
M2	重加密	5 失败路径	不存在	indcpa_enc	不返回错误	A-18	
M2	重加密	6 异常处理	不存在	indcpa_enc	无异常机制	A-18	
M2	比较	1 提前返回	不存在	verify	遍历 1088 B	A-18	
M2	比较	2 可变时间比较	不存在	verify	累积异或	A-18	
M2	比较	3 数据相关循环	不存在	verify	常数 1088	A-18	
M2	比较	4 数据相关访存	不存在	verify	顺序下标	A-18	
M2	比较	5 失败路径	不存在	verify / cmov	结果只作掩码	A-18	
M2	比较	6 异常处理	不适用	—	不可失败	A-18	
M2	隐式拒绝	1 提前返回	不存在	dec / cmov	无条件走完	A-18	
M2	隐式拒绝	2 可变时间比较	不存在	cmov	值屏障在位	A-18	
M2	隐式拒绝	3 数据相关循环	不存在	cmov / KDF	32 B 与定长	A-18	
M2	隐式拒绝	4 数据相关访存	不存在	cmov	固定偏移	A-18	
M2	隐式拒绝	5 失败路径	不存在	dec	同一路径，返回 0	A-18	
M2	隐式拒绝	6 异常处理	不适用	—	无异常机制	A-18	
M2 小计：存在 3、不存在 24、不适用 3、未检查 0（30 格）							

表 F2　M2 的 30 格排查。“不存在”只覆盖给定产物、输入与工具，不证明常数时间；重加密的“存在”格在未修与已修两侧均成立。

实现	步骤	来源	判定	函数	指令或行号	证据
M5	解密	1 提前返回	不存在	indcpa_dec	常数回边	A-21
M5	解密	2 可变时间比较	不存在	poly_tomsg	乘加、移位	A-21
M5	解密	3 数据相关循环	不存在	indcpa_dec	常数上界	A-21
M5	解密	4 数据相关访存	不存在	indcpa_dec	计数器下标	A-21
M5	解密	5 失败路径	不存在	indcpa_dec	仅分配失败清理	A-21
M5	解密	6 异常处理	不适用	—	无异常机制	A-21
M5	哈希	1 提前返回	不存在	hash_g / hash_h / hash_j	定长调用	A-21
M5	哈希	2 可变时间比较	不存在	FIPS202	长度驱动	A-21
M5	哈希	3 数据相关循环	不存在	FIPS202	固定轮数	A-21
M5	哈希	4 数据相关访存	不存在	FIPS202	常量下标	A-21
M5	哈希	5 失败路径	不存在	FIPS202	不返回状态	A-21
M5	哈希	6 异常处理	不存在	FIPS202	无 abort/断言	A-21
M5	重加密	1 提前返回	不存在	poly_frommsg	唯一跳转为循环回边	A-21
M5	重加密	2 可变时间比较	不存在	压缩函数	乘加、位移	A-21
M5	重加密	3 数据相关循环	不存在	poly_frommsg	32 次定长	A-21
M5	重加密	4 数据相关访存	不存在	poly_frommsg	固定顺序	A-21
M5	重加密	5 失败路径	不存在	indcpa_enc	仅分配失败清理	A-21
M5	重加密	6 异常处理	不适用	—	无异常机制	A-21
M5	比较	1 提前返回	不存在	ct_memcmp	循环体内无跳转	A-21
M5	比较	2 可变时间比较	不存在	ct_memcmp	无分支归约	A-21
M5	比较	3 数据相关循环	不存在	ct_memcmp	常数 1088	A-21
M5	比较	4 数据相关访存	不存在	ct_memcmp	顺序 64 B/轮	A-21
M5	比较	5 失败路径	不存在	ct_memcmp	返回不构造分支	A-21
M5	比较	6 异常处理	不适用	—	不可失败	A-21
M5	隐式拒绝	1 提前返回	存在（公开信息）	check_sk	`a404 call → a40e jne ad87`	A-21、A-22
M5	隐式拒绝	2 可变时间比较	不存在	ct_cmov_zero	`ad60` 至 `ad85` 掩码	A-21
M5	隐式拒绝	3 数据相关循环	不存在	KDF / cmov	固定轮数与 32 B	A-21
M5	隐式拒绝	4 数据相关访存	不存在	KDF / cmov	固定偏移	A-21
M5	隐式拒绝	5 失败路径	不存在	dec	成功与失败同路径	A-21
M5	隐式拒绝	6 异常处理	不适用	—	无异常机制	A-21
M5 小计：存在 1、不存在 25、不适用 4、未检查 0（30 格）						

表 F3　M5 的 30 格排查。“不存在”限于给定产物、输入与工具，不证明常数时间；唯一“存在”格判据是公开数据的哈希，按有界例外登记。

三张表合计 90 格：存在 5、不存在 75、不适用 10、未检查 0。五格“存在”分别是 M1 重加密的提前返回、M2 解密的可变时间比较、M2 重加密的可变时间比较、M2 重加密的提前返回（未修与已修两侧），以及 M5 隐式拒绝的公开 sk 检查。十格“不适用”全部是异常处理类与该步骤不可能失败的组合。

90 格的判定边界比计数更重要。“存在”要求同时给出函数、指令或行号与动态或反汇编证据；“不存在”只表示在固定源码、固定编译器与选项、固定产物、给定输入与已用工具下没有发现；“不适用”表示该步骤不存在这类机制或该函数不可能失败；“未检查”单列，本轮为 0。M5 的“存在”格是公开 sk 完整性检查，判据是公开数据的哈希，按有界例外登记；M2 的残留分支在未修与已修两侧都存在；官方修复只消除了除法，消息编码分支仍在。

五个“存在”格分别落在三条机制上：M1 的消息编码分支、M2 的三处秘密除法与消息编码分支、M5 的公开 sk 检查。前四条与秘密相关，第五条只携带公开信息；把五格都写成“漏洞”会混淆可利用性，把第五条删掉又会掩盖一个有意的提前退出。表 F1 至表 F3 逐格保留函数与地址，是为了让读者能回到产物核对，而不是只看一个总数。

十格“不适用”的分布也要能对上：M1 三格、M2 三格、M5 四格，全部是异常处理类与该步骤不可能失败的组合。它们不是“没查”，也不是“查了没有”，而是该步骤没有这类机制可查。三种判定各自计数：“不存在”只用于已查无发现，“不适用”只用于无此类机制，“未检查”单列。这样“不适用”既不会并入阴性，也不会并入缺口。M3 与 M4 是人工校准，不计入 90 格，它们在正文与附录 E 单列，避免与真实实现混算。

G.复现入口与命令

本附录的命令均在 x86_64 Linux 用户态实跑；不写 sysfs、不需要 sudo。实跑环境为 WSL2 虚拟化 Linux（clang 14、CMake 3.31、OpenSSL 3），与锁频采数所用的裸金属不是同一台机器；命令的通过数用于验证软件流程，不产生计时结论。完整锁频采集需要 x86_64 裸金属与频率写权限，不在本附录的实跑范围内。

目标	命令	预期输出	实测结果（权限 / 机器）
普通构建与测试	`make test`	普通 CTest 全通过	57/57 通过，0 失败（普通用户；WSL2，clang 14）
消毒器构建与测试	`make sanitize`	ASan/UBSan CTest 全通过	57/57 通过，0 失败（普通用户；WSL2，clang 14；ASan 关闭泄漏检测并关闭地址随机化）
平台配置	`cmake -S . -B build/platform -DBASELINE_ENABLE_PLATFORM=ON -DA113_ARCHIVED_CASE_ONLY=OFF`	配置成功，生成构建树	配置成功（普通用户，WSL2）
平台构建	`cmake --build build/platform`	构建成功	构建成功（普通用户，WSL2）
平台测试	`ctest --test-dir build/platform --output-on-failure`	平台 CTest 全通过	87/87 通过，0 失败（普通用户；vendor 未启用；quick 档启用后 88/88）
一键 quick	`sh scripts/reproduce_all.sh quick --fetch`	四配置构建与 CTest 全通过，统计演练完成	263/263 通过，0 失败（88 + 57 + 57 + 61；普通用户，WSL2）
vendor 取件	`sh tools/fetch_all_vendors.sh`	四棵 vendor 树就位并逐项 SHA-256 校验	4/4 就位，摘要校验通过（普通用户，WSL2）
full 档演练	`sh scripts/reproduce_all.sh full --modules M2,M3,M4,M5 --control-root < 控制根 >`	只打印采集命令，不执行	打印成功，退出码 0（普通用户，WSL2）

表 G1　复现入口与命令。命令实跑通过；完整锁频采集需要 x86_64 裸金属与频率写权限。quick 档的 263 项是 platform 88 + ordinary 57 + sanitize 57 + historical-fixed 61，失败 0。

quick 档只做配置、构建、CTest 与小样本统计演练：统计演练每类 200 个样本、1 次重复，用于检查链路是否连通；它不锁频、不写 sysfs、不需要 sudo、不绑核，验证软件流程与测试入口，不产生任何模块的正式统计结论。full 档按冻结计划生成采集命令，默认只打印，

加 --execute 才真正采集，且必须给出控制根；计时结论来自锁频采集，仍需裸金属与频率写权限。干净 clone 缺 vendor 与 third_party，M1 与 M2 的 vendor 树缺省不存在，先跑取件脚本；取件按固定摘要校验，不覆盖已有且摘要一致的树。

实跑环境的编译器与采数环境不同：本附录的命令在 WSL2 的 clang 14 与 CMake 3.22 下执行，锁频数据来自另一台 x86_64 裸金属的 clang 18.1.3。两边只共享源码与测试入口，不共享时间结论。把这一点写在表 G1 的表注里，是为了避免读者把 263 项通过当成 263 次计时实验。

命令失败时的排查顺序也写进复现说明：先看编译器与 CMake 版本，再看 vendor 是否就位，然后看 sanitizer 的运行库。ASan 在部分虚拟化环境需要关闭地址随机化或关闭泄漏检测才能跑完；这类环境差异只影响测试入口，不影响源码。复现者若在另一台机器上得到不同的通过数，应先核对配置项与编译器，而不是先怀疑实验结论。

H.术语、缩略语与匿名检查

表 H1 给出全篇统一术语；表 H2 给出提交前的匿名检查项。术语第一次出现时给中文全称与英文原名，之后只用定稿词，不混用同义词。

术语	英文 / 全称	含义	首见
ML-KEM	Module-Lattice-Based Key-Encapsulation Mechanism	FIPS 203 标准化的密钥封装机制；本作品的算法标准为 ML-KEM-768。	1.1
Kyber	CRYSTALS-Kyber	ML-KEM 的前身；本作品的 M2 是 pq-crystals 历史快照 Kyber768。	1.1
FO	Fujisaki-Okamoto	把公钥加密转换为 KEM 的变换；解封装中的重加密与比较由它规定。	1.1
KEM	Key-Encapsulation Mechanism	密钥封装机制。	1.1
PQC	Post-Quantum Cryptography	后量子密码。	1.1
MDE	Minimum Detectable Effect	最小可检测效应；给定 α、功效、样本量与方差下可分辨的最小均值差。	2.6
Holm	Holm-Bonferroni	逐步下降的多重比较校正；本作品四个家族各取全族 $\alpha = 0.05$。	2.6
RTLF	Relative Time Leakage Framework	基于分位数与 bootstrap 的族外分布检验；$\alpha = 0.01$。	2.6
dudect	Dude, is my code constant time?	基于 Welch t 的在线泄漏检测工具；最大 t 不转成 p 值。	2.6
Binsec	Binsec/Rel	二进制级关系符号执行与有界常数时间检查工具。	2.7
Microwalk	MicroWalk	基于动态二进制插桩的访存互信息分析工具。	2.7
Valgrind	Valgrind	动态分析框架；本作品使用 Memcheck 与 varlat 补丁。	2.7
varlat	variable-latency patch	Valgrind 的可变时延指令检查补丁，用于除法操作数含秘密的情形。	2.7
aarch64	ARM 64-bit architecture	ARM 64 位架构；本作品的第二架构对照，不是第二台测量机。	3.11
x86_64	x86-64	本作品的主测量架构。	2.4
ipd	initial public draft	FIPS 203 初始公开草案；M1 的对象口径。	2.4

表 H1　术语与缩略语。术语全篇统一；第一次出现给全称，之后只用定稿词。

检查项	检查方法	结果
单位名称	全文检索单位、学院、实验室名称	0 处
姓名	全文检索成员与指导教师姓名	0 处
主机名	全文检索主机名、机器别名与网络地址	0 处
账号	全文检索用户名、账号、邮箱	0 处
绝对路径	全文检索盘符、用户目录与远端工作根	0 处
内部代号	全文检索内部工作项代号与提交号	0 处（提交号只在脚注与表格）
封面“注意”段	检查封面源文件	已删除
作品类别	检查封面与基本信息表	已勾选“工程实践”
元数据	检查 PDF/DOCX 作者、标题、修订记录	合并时清空作者字段并复查

表 H2　匿名检查。匿名检查逐项打勾；发现即改，不带入提交。本表为本稿扫描结果，合并五章后复扫一次。

匿名是红线，不是格式偏好。表 H2 的九项在每章交稿与合并稿各查一次；提交前再查

PDF 与 DOCX 的元数据。发现单位、姓名、主机名、账号、绝对路径或内部代号，直接改源文件，不在生成稿上涂改。

术语与匿名检查都在写作过程中做，不等到提交前。术语表锁定后，正文、表题、图题、摘要与答辩共用一套词；发现同义词混用，先改源文件，再改生成稿。

提交前最后一遍只做三件事：扫身份信息、对图表编号、对数字来源。身份信息按表 H2；图表编号按正文引用逐张核对；数字来源按台账编号逐句抽查。三件事都过，才生成最终 PDF。术语表与匿名表都在提交前复查一次；复查记录不进正文，只留在工作文件里。

参考文献

[1] NIST. Module-Lattice-Based Key-Encapsulation Mechanism Standard. FIPS 203, 2024. DOI: 10.6028/NIST.FIPS.203.

[2] R. Avanzi, J. Bos, L. Ducas, et al. CRYSTALS-Kyber algorithm specifications and supporting documentation. NIST PQC submission, 2021 (initial version 2018).

[3] Q. Guo, T. Johansson, A. Nilsson. A Key-Recovery Timing Attack on Post-Quantum Primitives Using the Fujisaki-Okamoto Transformation and Its Application on FrodoKEM. CRYPTO 2020, LNCS 12171, 359–386. DOI: 10.1007/978-3-030-56880-1_13.

[4] D. J. Bernstein, et al. KyberSlash: Exploiting secret-dependent division timings in Kyber implementations. IACR TCHES 2025(2): 209–234. DOI: 10.46586/tches.v2025.i2.209-234.

[5] A. Purnal. clangover. GitHub, 2024. https://github.com/antoonpurnal/clangover.

[6] GitHub Security Advisory GHSA-f2v9-5498-2vpp (CVE-2024-36405): liboqs Kyber control-flow timing leak when compiled with Clang. 2024.

[7] O. Reparaz, J. Balasch, I. Verbauwhede. Dude, is my code constant time? DATE 2017, 1697–1702. DOI: 10.23919/DATE.2017.7927267.

[8] M. Dunsche, M. Maehren, N. Erinola, R. Merget, N. Bissantz, J. Somorovsky, J. Schwenk. With Great Power Come Great Side Channels: Statistical Timing Side-Channel Analyses with Bounded Type-I Errors. USENIX Security 2024, 6687–6704. https://www.usenix.org/conference/usenixsecurity24/presentation/dunsche

[9] A. Langley. Checking that functions are constant time with Valgrind. 2010. https://github.com/agl/ctgrind.

[10] J. Wichelmann, A. Moghimi, T. Eisenbarth, B. Sunar. MicroWalk: A Framework for Finding Side Channels in Binaries. ACSAC 2018, 161–173. DOI: 10.1145/3274694.3274741.

[11] L. Daniel, S. Bardin, T. Rezk. Binsec/Rel: Efficient Relational Symbolic Execution for Constant-Time at Binary-Level. IEEE S&P 2020, 1021–1038. DOI: 10.1109/SP40000.2020.00074.

[12] G. Goodwill, B. Jun, J. Jaffe, P. Rohatgi. A testing methodology for side-channel resistance validation. NIST Non-Invasive Attack Testing Workshop, 2011.

[13] J. B. Almeida, M. Barbosa, G. Barthe, F. Dupressoir, M. Emmi. Verifying Constant-Time Implementations. USENIX Security 2016, 53–70.

[14] M. Azouaoui, O. Bronchain, C. Hoffmann, Y. Kuzovkova, T. Schneider, F.-X. Standaert. Systematic Study of Decryption and Re-encryption Leakage: The Case of Kyber. SAC 2021, LNCS 13203, 236–256, 2022. DOI: 10.1007/978-3-030-99766-3_11.

[15] J. Jancar, M. Fourné, D. De Almeida Braga, M. Sabt, P. Schwabe, G. Barthe, P.-A. Fouque, E. Acar. “They’re not that hard to mitigate”: What Cryptographic Library Developers Think About Timing Attacks. IEEE S&P 2022, 632–649. DOI: 10.1109/SP46214.2022.9833713.